Table of contents

MEDSCRIPTAI - Zero-to-Elite Lab Manual (beginner edition, October 2026)

WHERE TO FIND WHAT (read this first)

PART 0 - Start Here

0.1 What MedScriptAI is (in plain words)
0.2 The whole journey in one picture
0.3 What you will be able to do at the end
0.4 Every place you will work
0.5 What each piece does
0.6 What it costs

PART 1 - Prerequisites

1.1 Checklist
1.2 Refreshers and 1-minute checks
1.3 Accounts to create now (some approvals take a day)

PART 2A - Hyperstack GPU machine + SSH + VS Code + WinSCP connection (from zero)

BEFORE YOU START: what you are building in this part
SECTION 2A.1 - Install VS Code on your laptop
SECTION 2A.2 - Install the Remote - SSH extension in VS Code
SECTION 2A.3 - Hugging Face token and Llama licence
SECTION 2A.4 - Hyperstack account and credit
SECTION 2A.5 - Create the SSH key on your laptop
SECTION 2A.6 - Create an environment in Hyperstack
SECTION 2A.7 - Import your SSH key into Hyperstack
SECTION 2A.8 - Deploy the GPU machine (every field)
SECTION 2A.9 - Wait for ACTIVE, copy the IP, check the SSH firewall
SECTION 2A.10 - First connection from PowerShell
SECTION 2A.11 - Connect VS Code to the GPU computer (Remote-SSH)
SECTION 2A.12 - Install and connect WinSCP (drag-and-drop files)
SECTION 2A.13 - Stop paying: hibernate, restore and delete
SECTION 2A.14 - Connection troubleshooting

PART 2B - Project folder + tools on the GPU computer (from an empty disk)

BEFORE YOU START: what you are building in this part
SECTION 2B.1 - Create the project folders
SECTION 2B.2 - Open the project folder in VS Code
SECTION 2B.3 - THE procedure to create any file (practise it once)
SECTION 2B.4 - Store secrets and the model cache location
SECTION 2B.5 - Install system tools and learn tmux (jobs that survive disconnects)
SECTION 2B.6 - Install uv (Python package manager)
SECTION 2B.7 - Create the three requirements files
SECTION 2B.8 - Create the three Python environments
SECTION 2B.9 - GPU test (PyTorch and Docker)
SECTION 2B.10 - Create .gitignore and start a Git repository (for your portfolio)
SECTION 2B.11 - Install FakeCloud (free AWS) and the AWS CLI
SECTION 2B.12 - (Real AWS only) Request GPU quota early
PART 2 RESULT CHECK (all must be true before Part 4)

PART 3 - The Core Idea (read this before you write any code)

3.1 One analogy for the whole project
3.2 Tokens and the chat template
3.3 What "training" actually does
3.4 LoRA in numbers
3.5 QLoRA: making an 8B model fit
3.6 Data: train, validation, test, and leakage
3.7 Synthetic data (Part 5): what it is and why
3.8 DPO (Part 6A): learning from "better vs worse"
3.9 Evaluation (Parts 5-7): five kinds of evidence
3.10 Serving and production (Parts 6A-6B)

PART 4 - BASICS: your first medical fine-tune (data → .jsonl → QLoRA SFT → test → publish)

BEFORE YOU START: what you will build in this part
SECTION 4.1 - Create scripts/common.py (settings shared by every script)
SECTION 4.2 - Create scripts/01_download_data.py
SECTION 4.3 - Run the download
SECTION 4.4 - Create scripts/02_build_sft_jsonl.py (clean + convert)
SECTION 4.5 - Run the conversion
SECTION 4.6 - Create and run scripts/03_validate_jsonl.py
SECTION 4.7 - Create scripts/10_train_sft.py (the training script)
SECTION 4.8 - Run your first training
SECTION 4.9 - Create and run scripts/11_chat_test.py (base vs fine-tuned)
SECTION 4.10 - Create and run scripts/12_push_to_hub.py (publish)
SECTION 4.11 - First cloud step: copy the adapter to object storage (S3)
SECTION 4.12 - Save your progress in Git
PART 4 RESULT CHECK
4.13 Common mistake at this level

PART 5 - INTERMEDIATE: synthetic data, three adapters, real evaluation

BEFORE YOU START: what you will build in this part
SECTION 5.1 - Create serve/start_teacher.sh
SECTION 5.2 - Start the teacher and test it
SECTION 5.3 - Create scripts/20_synth_explanations.py (synthetic data)
SECTION 5.4 - Dry run on 50 rows and inspect quality
SECTION 5.5 - Full synthetic run
SECTION 5.6 - Free the GPU
SECTION 5.7 - Train the three adapters
SECTION 5.8 - Create serve/start_vllm.sh and serve all adapters
SECTION 5.9 - Create and run scripts/30_generate_preds.py (answers on the test set)
SECTION 5.10 - Create and run scripts/31_eval_metrics.py (automatic metrics)
SECTION 5.11 - Create and run scripts/32_judge.py (LLM-as-a-judge)
SECTION 5.12 - Publish your datasets
SECTION 5.13 - Cloud step: reports to S3, token into Secrets Manager
SECTION 5.14 - Commit
PART 5 RESULT CHECK
5.15 Common mistake at this level

PART 6A - ADVANCED: DPO, vLLM multi-LoRA hot-swap, FastAPI + LangSmith

MedScriptAI

Zero-to-Elite Lab Manual: medical LLM post-training (QLoRA SFT + DPO) and production deployment

Llama-3.1-8B-Instruct · Unsloth · TRL · PEFT · distilabel · vLLM multi-LoRA · FastAPI · LangSmith · Docker · FakeCloud / AWS

Version: October 2026

BEFORE YOU START: what you will build in this part
SECTION 6A.1 - Start both servers again
SECTION 6A.2 - Create scripts/21_build_dpo_pairs.py
SECTION 6A.3 - Build the pairs
SECTION 6A.4 - Create scripts/22_train_dpo.py
SECTION 6A.5 - Train DPO
SECTION 6A.6 - Serve five adapters and hot-swap them
SECTION 6A.7 - Evaluate base vs SFT vs DPO
SECTION 6A.8 - Create the API files (api/requirements.txt and api/app.py)
SECTION 6A.9 - Run the API and call it
SECTION 6A.10 - Create the tests and run them without a GPU
SECTION 6A.11 - LangSmith tracing
SECTION 6A.12 - Commit
PART 6A RESULT CHECK
6A.13 Common mistake at this level

PART 6B - ADVANCED: Docker image, Compose stack, and cloud deployment (FakeCloud and AWS side by side)
BEFORE YOU START: the words of this part
SECTION 6B.1 - Create api/Dockerfile
SECTION 6B.2 - Build the image
SECTION 6B.3 - Test the container against the running vLLM
SECTION 6B.4 - Create the Compose files
SECTION 6B.5 - Fill in deploy/.env
SECTION 6B.6 - Run the full stack with one command
SECTION 6B.7 - Create deploy/ec2_user_data.sh (what the AWS machine does on first boot)
SECTION 6B.8 - Deploy step by step (registry, storage, secrets, IAM, logs, firewall, GPU machine)
SECTION 6B.9 - Create deploy/deploy.sh (the whole table as one script)
SECTION 6B.10 - Create deploy/teardown.sh and use both scripts
SECTION 6B.11 - (Optional) SageMaker endpoint objects
SECTION 6B.12 - Commit
PART 6B RESULT CHECK
6B.13 Common mistake at this level

PART 7 - ELITE: prove your results are real (how top labs work)
BEFORE YOU START: why this part exists
7.1 Who defines the top level today
SECTION 7.2 - Leakage check (13-gram near-duplicates)
SECTION 7.3 - Is the gain real? (paired bootstrap)
SECTION 7.4 - Can you trust the judge? (Cohen's kappa)
SECTION 7.5 - Criterion-level scoring for the summary task
SECTION 7.6 - Zero-downtime rollout and rollback
SECTION 7.7 - Report and model card
7.8 What elite teams would try next
SECTION 7.9 - Commit
PART 7 RESULT CHECK
7.10 Common mistake at this level

PART 8 - Tools and Resources Map (checked October 2026)

PART 9 - Keep It Working

PART 10 - Troubleshooting

PART 11 - Practice Projects (goals and success criteria only, no solutions)
11.1 Easy - a fourth adapter: patient-friendly explanations
11.2 Medium - RL with verifiable rewards for QA
11.3 Hard - production hardening on AWS

PART 12 - Cheat Sheet
Part 2 - workspace
Part 4 - data and first SFT (.venv-train, folder scripts)
Part 5 - synthetic data, three adapters, evaluation
Part 6A/6B - DPO, serving, API, Docker, deployment
Part 7 - elite checks
Key numbers and settings

PART 13 - Self-Test

PART 14 - Sources

ANSWER KEY

MEDSCRIPTAI - Zero-to-Elite Lab Manual (beginner edition, October 2026)

Who this is for: a complete beginner in fine-tuning who knows a little Python. Every step is one action. Every file you need is created by you, by hand, in a folder you create. Nothing is skipped.

Assumptions (change anything that is wrong for you): Windows 11 laptop with no usable GPU; goal = a portfolio project you can build, show and defend in AI-engineer interviews; budget = pay-as-you-go GPU on Hyperstack (about US$25-40 for the whole manual), FakeCloud is free, real AWS is optional (about US$3-10 if you follow the teardown steps).

WHERE TO FIND WHAT (read this first)

This document is split into tabs. Click a tab name in the left panel of Google Docs (Document tabs) to open it.

If you are looking for...	Open this tab	Section
What the project is, cost, tools, accounts to create	MEDSCRIPT AI (this tab)	Part 0, Part 1
Hyperstack account, GPU machine, SSH key, connecting VS Code and WinSCP	MS Part 2A - Hyperstack + SSH + VS Code + WinSCP	2A.1 - 2A.14
Creating the project folder, Python environments, FakeCloud, AWS CLI	MS Part 2B - Project Folder + Tools on the VM	2B.1 - 2B.12
The ideas behind everything (read before coding)	MS Part 3 - Core Idea	3.1 - 3.10
Data → .jsonl → first fine-tune → test → publish	MS Part 4 - Basics	4.1 - 4.12
Synthetic data (distilabel), three adapters, evaluation	MS Part 5 - Intermediate	5.1 - 5.14
DPO, vLLM multi-LoRA hot-swap, FastAPI, LangSmith	MS Part 6A - DPO + vLLM + API	6A.1 - 6A.12
Docker, ECR, S3, IAM, EC2 deployment (FakeCloud and AWS side by side)	MS Part 6B - Docker + Cloud Deploy	6B.1 - 6B.12
Contamination, significance tests, judge calibration, rollout	MS Part 7 - Elite	7.1 - 7.8
Tools map, maintenance, troubleshooting, projects, cheat sheet, self-test, sources	MS Parts 8-14 - Reference	8 - 14

How every practical section is written:

Label	Meaning
WHERE:	the exact place you do the work (which app, which window, which folder)
GOAL:	what this section achieves, in one line
Step 1. Step 2. ...	one action per step; do them in order
grey box	text you type or paste EXACTLY (never type the prompt sign $)
You should see:	what appears if the step worked
Watch out:	the mistake beginners make here, and how to avoid it
CHECKPOINT:	what must be true before you continue; If not: the fix
Real AWS (same step) column, light blue	the same cloud step on real AWS; the white column is FakeCloud (free practice)

Golden rules: (1) never skip a CHECKPOINT; (2) if a step fails, stop and fix it before the next step; (3) read the "BEFORE YOU START" box of every part before typing anything.

PART 0 - Start Here

0.1 What MedScriptAI is (in plain words)

MedScriptAI is your own medical AI assistant. You take Meta's Llama-3.1-8B-Instruct (a general chatbot model with 8 billion parameters) and teach it three medical jobs: answer medical exam questions with an explanation (qa), turn a doctor-patient conversation into a clinical note (notes), and turn a clinical note into structured JSON data (summary).

Teaching an existing model new behaviour is called post-training. You will do the two most important kinds: SFT (supervised fine-tuning: show the model thousands of examples of correct answers) and DPO (direct preference optimisation: show the model pairs of answers and tell it which one is better).

Real-world use: AI medical scribes (Nuance DAX, Abridge, Suki), exam-prep tutors, extraction of structured data from health records, hospital copilots. Important: real clinical use needs regulatory approval; this project is an educational portfolio system, never a medical device.

0.2 The whole journey in one picture

laptop (VS Code + WinSCP) --SSH--> Hyperstack GPU computer --> download data --> clean to .jsonl --> synthetic data with a teacher model --> SFT training (3 small "adapters") --> DPO training --> evaluation --> serve with vLLM --> FastAPI web API --> Docker image --> FakeCloud (free AWS practice) --> real AWS (optional) --> tear down.

Part	What you will do	Time estimate
0 Start Here	understand the project, tools and costs	20 min
1 Prerequisites	check your skills, create accounts	1-2 h
2A Hyperstack + connection	GPU machine, SSH key, VS Code, WinSCP	1-2 h
2B Project folder + tools	folders, Python environments, FakeCloud, AWS CLI	1-2 h
3 Core Idea	the ideas behind every step	45 min
4 Basics	data → .jsonl → first fine-tuned adapter → test → publish	3-4 h (about 1.5 GPU-h)
5 Intermediate	synthetic data, three adapters, evaluation	1 day (about 6 GPU-h)
6A Advanced	DPO, vLLM multi-LoRA serving, FastAPI, LangSmith	1 day (about 4 GPU-h)
6B Advanced	Docker, ECR, S3, IAM, EC2, teardown	half a day (about 1 GPU-h + US$3-5 AWS)
7 Elite	prove your results are real	4-6 h
8-14 Reference	tools, maintenance, troubleshooting, projects, cheat sheet, self-test, sources	reference

0.3 What you will be able to do at the end

- Download open medical datasets, clean them and convert them to the chat .jsonl format that training libraries expect.
- Fine-tune Llama-3.1-8B-Instruct with QLoRA (Unsloth + TRL + PEFT), then improve it with DPO, and explain every setting.
- Create synthetic training data and preference pairs with distilabel and a teacher model, with quality filters.
- Evaluate honestly: accuracy, ROUGE-L, BERTScore, LLM-as-a-judge with position swap, confidence intervals, contamination checks.
- Serve one base model with several hot-swappable adapters in vLLM behind a FastAPI web API traced in LangSmith.
- Package it with Docker, push it to (Fake)ECR, run it on an AWS GPU machine, and delete everything so the bill stops.

0.4 Every place you will work

Place	What it is	What you do there
Your laptop: PowerShell	the Windows terminal	create the SSH key, first SSH login
Your laptop: VS Code + Remote-SSH	code editor connected to the GPU computer	create every folder and file, run every command
Your laptop: WinSCP	drag-and-drop file transfer	copy reports and files between laptop and GPU computer
Your laptop: web browser	websites	Hyperstack console, Hugging Face, LangSmith, AWS console

Hyperstack GPU computer (Ubuntu 24.04 + NVIDIA A100 80 GB)	a computer you rent per hour	all training, serving, Docker and FakeCloud
FakeCloud (on the GPU computer, port 4566)	free copy of AWS for practice	practise every AWS command at no cost
Real AWS (region us-east-1)	Amazon's cloud	registry, storage, permissions, secrets, logs, a GPU server
Hugging Face Hub	website hosting models and datasets	download Llama and data; publish your adapters
LangSmith	tracing website	see every request to your API

0.5 What each piece does

Component	Its job
Llama-3.1-8B-Instruct	the base model you improve
Hugging Face datasets library	downloads MedQA, MedMCQA, PubMedQA and clinical notes
.jsonl file	a text file with one JSON object per line: the standard training input
distilabel	builds synthetic-data pipelines (a teacher model writes, a judge model rates)
Qwen2.5-14B-Instruct	the teacher and judge model (bigger and stronger than your 8B model; Apache-2.0 licence)
bitsandbytes	stores the big model in 4-bit so it fits in GPU memory (the "Q" in QLoRA)
PEFT (LoRA)	adds small trainable "adapter" matrices; only about 0.5% of the parameters are trained
Unsloth	makes LoRA training about 2x faster and lighter
TRL	Hugging Face's training library: SFTTrainer and DPOTrainer
vLLM	a very fast model server; serves many adapters on one base model
FastAPI	your web API (POST /v1/generate) in front of vLLM
LangSmith	records every API request (input, output, time)
Docker / Docker Compose	packages the API into an image; starts vLLM + API together
FakeCloud	free local AWS emulator: same aws commands, address http://localhost:4566
AWS ECR / S3 / IAM / Secrets Manager / CloudWatch / EC2	image registry, file storage, permissions, secrets, logs and alarms, the cloud GPU server

0.6 What it costs

Item	Price (October 2026)	How much you need	Approx. cost
Hyperstack 1x A100-80G-PCIe	about US$1.35 per hour [1] (verify on the pricing page)	about 13-15 hours in total	US$18-21
Hyperstack storage while hibernated	about US$0.10 per TB per hour [3]	a few days	under US$3
FakeCloud	free and open source [5]	always	US$0
AWS g6.xlarge (1x NVIDIA L4 24 GB)	about US$0.805 per hour in us-east-1 [12]	2-3 hours	about US$2-3
AWS ECR, S3, Secrets Manager, CloudWatch	cents for this project	Part 6B	under US$2
Hugging Face, LangSmith Developer plan	free (LangSmith: about 5k traces per month [11])	always	US$0

Watch out: Hyperstack keeps billing a machine that is only "stopped" (SHUTOFF). Only HIBERNATED or deleted machines stop the GPU bill [3]. Section 2A.13 shows how.

PART 1 - Prerequisites

1.1 Checklist

Item	MUST or NICE-TO-HAVE
Python basics: variables, lists, dicts, functions, f-strings, reading an error message	MUST
JSON: objects, arrays, double-quoted strings	MUST
Terminal basics: cd, ls, pwd, cat, mkdir	MUST
What a GPU and VRAM are	MUST
HTTP APIs: URL, POST, JSON body, status codes	MUST
Git basics	NICE-TO-HAVE
Docker basics	NICE-TO-HAVE (taught in Part 6B)
Statistics: mean, confidence interval	NICE-TO-HAVE (taught in Part 7)

1.2 Refreshers and 1-minute checks

Python. A dict maps keys to values; a list keeps order. Check: what does this print?

```
row = {"task": "qa", "messages": [1, 2, 3]}
print(len(row["messages"]), row.get("source", "none"))
```

Answer: 3 none.

JSON. Text format: {} objects, [] arrays, strings MUST use double quotes. A .jsonl file has one JSON object per line. Check: which line is NOT valid JSON?

```
{"role": "user", "content": "hi"}
{'role': 'user', 'content': 'hi'}
```

Answer: the second (single quotes are Python, not JSON).

Terminal. pwd = where am I; ls = list files; cd folder = go into folder; cd .. = go up one folder; mkdir -p a/b = make folders; cat file = print a file. Check: after mkdir -p test/one and ls test you should see one.

GPU and VRAM. The GPU does the maths of the model in parallel; VRAM is the GPU's memory. 8 billion parameters x 2 bytes (16-bit) = 16 GB just for the weights; in 4-bit about 5-6 GB. Check: how much VRAM do 14 billion parameters need in 16-bit? Answer: about 28 GB.

HTTP APIs. A client sends a request (method, URL, JSON body); the server answers with a status code (200 OK, 401 not authorised, 422 invalid input, 500 server error) and JSON. Check: which code means "your API key is missing"? Answer: 401.

1.3 Accounts to create now (some approvals take a day)

What	Where	Cost	Which option
Hugging Face account	https://huggingface.co/join	free	normal account
Hugging Face token	https://huggingface.co/settings/tokens	free	Write token (Section 2A.3)
Llama 3.1 licence	https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct	free	fill the form; approval can take hours
Hyperstack account	https://console.hyperstack.cloud	pay as you go	add US$30-40 credit (Section 2A.4)
LangSmith account	https://smith.langchain.com	free Developer plan	you create the API key in Part 6A
AWS account (optional)	https://aws.amazon.com	pay as you go	region us-east-1; request GPU quota early (Section 2B.12)

Watch out: the Llama licence says any model you publish that was fine-tuned from Llama must have a name starting with "Llama" and you must show "Built with Llama" [15]. Your adapters will be called Llama-3.1-8B-MedScriptAI-... for this reason.

CHECKPOINT: you have a Hugging Face account, the Llama licence is requested, and you can log in to Hyperstack.

PART 2A - Hyperstack GPU machine + SSH + VS Code + WinSCP (connection from zero)

BEFORE YOU START: what you are building in this part

Your laptop has no strong GPU, so you rent one. Hyperstack is a cloud company that rents computers with NVIDIA GPUs by the hour. You will: (1) install two programs on your laptop (VS Code and WinSCP), (2) make an SSH key (a digital key pair that lets your laptop log in securely), (3) rent a GPU computer ("VM" = virtual machine) on Hyperstack, and (4) connect to it three ways: PowerShell (a terminal), VS Code (an editor that works ON the VM) and WinSCP (drag-and-drop files).

Word	Plain meaning
VM (virtual machine)	a computer in a data centre that you control over the internet
SSH	the secure way to log in to a Linux computer remotely; it uses port 22
SSH key pair	two files: the private key (stays on your laptop, never share) and the public key (.pub, you give it to Hyperstack)
public IP	the internet address of your VM, for example 38.80.123.45
environment (Hyperstack)	a folder in Hyperstack that holds your VMs and keys in one region
flavor (Hyperstack)	the hardware size: which GPU, how many CPUs and how much memory
image (OS image)	the operating system installed on the VM; ours already has the NVIDIA driver and Docker

Result at the end of this part: VS Code on your laptop shows a terminal that says ubuntu@medscriptai-gpu and nvidia-smi prints "NVIDIA A100 80GB PCIe".

SECTION 2A.1 - Install VS Code on your laptop

WHERE: web browser on your Windows laptop

GOAL: VS Code installed.

Step 1. Open https://code.visualstudio.com/download

Step 2. Click the Windows button (User Installer, x64).

Step 3. Open the downloaded file VSCodeUserSetup-x64-....exe.

Step 4. Click I accept the agreement, then Next.

Step 5. Keep the install folder, click Next, then Next again.

Step 6. On "Select Additional Tasks" tick Add "Open with Code" action to Windows Explorer file context menu and Add to PATH.

Step 7. Click Next, then Install, then Finish.

You should see: VS Code opens with a Welcome tab.

CHECKPOINT: VS Code is open.

SECTION 2A.2 - Install the Remote - SSH extension in VS Code

WHERE: VS Code on your laptop

GOAL: VS Code can open folders on the GPU computer.

Step 1. Press Ctrl+Shift+X (Extensions panel opens on the left).

Step 2. In the search box type Remote - SSH.

Step 3. Click the result named Remote - SSH whose publisher is Microsoft.

Step 4. Click Install.

You should see: a new green-blue "><" icon in the bottom-left corner of VS Code.

Watch out: install the one from Microsoft only; ignore look-alikes.

CHECKPOINT: the extension shows "Installed" (or a gear icon instead of the Install button).

SECTION 2A.3 - Hugging Face token and Llama licence

WHERE: web browser

GOAL: a Write token (password for scripts) and access to Llama 3.1.

Step 1. Open https://huggingface.co/join and create an account (skip if you have one).

Step 2. Open https://huggingface.co/settings/tokens

Step 3. Click Create new token.

Step 4. Click the Write tab.

Step 5. In Token name type medscriptai

Step 6. Click Create token.

Step 7. Click Copy and paste the token into a private Notepad file named secrets.txt on your laptop. It starts with hf_ and is shown only once.

Step 8. Open https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct

Step 9. Fill in the licence form at the top and click Submit.

You should see: "Your request to access this repo has been submitted" (approval later shows "You have been granted access").

Watch out: never paste the token into code files or chats. You will store it on the VM in Section 2B.4.

CHECKPOINT: secrets.txt contains your hf_ token; the licence request is submitted.

SECTION 2A.4 - Hyperstack account and credit

WHERE: web browser

GOAL: a Hyperstack account with money on it.

Step 1. Open https://authui.hyperstack.cloud/register

Step 2. Create the account (e-mail, password) and confirm your e-mail.

Step 3. Open https://console.hyperstack.cloud/billing/activation and activate billing.

Step 4. Open https://console.hyperstack.cloud/billing

Step 5. Add credit: US$30-40 is enough for the whole manual.

You should see: your credit balance on the Billing page.

Watch out: Hyperstack warns about a fake look-alike website; only use addresses ending in hyperstack.cloud [1].

CHECKPOINT: the Billing page shows your credit.

SECTION 2A.5 - Create the SSH key on your laptop

WHERE: Windows laptop > Start menu > type PowerShell > click Windows PowerShell

GOAL: a private key file and a public key file in C:\Users\YOUR_NAME\.ssh

Step 1. Check that SSH is installed. Type and press Enter:

```
ssh -V
```

You should see: OpenSSH_for_Windows_9.x ...

Step 2. Create the key pair. Type and press Enter:

```
ssh-keygen -t ed25519 -f $env:USERPROFILE\.ssh\hyperstack_ed25519 -C "medscriptai"
```

Step 3. When it asks "Enter passphrase (empty for no passphrase)", press Enter.

Step 4. When it asks "Enter same passphrase again", press Enter.

You should see: "Your identification has been saved in C:\Users\YOUR_NAME\.ssh\hyperstack_ed25519" and a small square picture of random characters.

Step 5. Print the PUBLIC key:

```
Get-Content $env:USERPROFILE\.ssh\hyperstack_ed25519.pub
```

You should see: one line starting with ssh-ed25519 and ending with medscriptai

Step 6. Select that whole line with the mouse and press Ctrl+C (copy). Keep PowerShell open.

Placeholder	Replace with	Example

YOUR_NAME	your Windows user folder name (shown in the "saved in" line)	rahul

Watch out: the file WITHOUT .pub is your private key: never upload, e-mail or share it.

CHECKPOINT: the folder C:\Users\YOUR_NAME\.ssh contains hyperstack_ed25519 and hyperstack_ed25519.pub

If not: if "ssh is not recognized", open Settings > System > Optional features > View features > search OpenSSH Client > Install, close and reopen PowerShell, repeat Step 1.

SECTION 2A.6 - Create an environment in Hyperstack

WHERE: web browser > https://console.hyperstack.cloud

GOAL: an environment named medscriptai-env (holds your VM and key).

Step 1. In the left menu click Virtual Machines.

Step 2. Click Deploy New Virtual Machine (the deploy form opens).

Step 3. Scroll to the Environment section.

Step 4. Click Create New Environment.

Step 5. In the name field type medscriptai-env

Step 6. Keep the suggested region and click Create (or Save).

You should see: medscriptai-env selected in the Environment section.

Watch out: the VM and the SSH key must be in the SAME environment, or the key cannot be used.

CHECKPOINT: medscriptai-env exists and is selected. Keep this form open for the next section.

SECTION 2A.7 - Import your SSH key into Hyperstack

WHERE: Hyperstack console > the same Deploy New Virtual Machine form

GOAL: Hyperstack knows your public key.

Step 1. In the Environment / SSH key area click Create New SSH Key.

Step 2. Click the Import SSH key tab.

Step 3. In Environment select medscriptai-env

Step 4. Click in the public key box and press Ctrl+V (the ssh-ed25519 ... line from Section 2A.5).

Step 5. In Name type laptop-ed25519

Step 6. Click Import.

You should see: laptop-ed25519 selected as the SSH key in the form.

CHECKPOINT: laptop-ed25519 is listed for medscriptai-env.

If not: "invalid key" = you pasted the private key or only part of the line; repeat Section 2A.5 Step 5-6.

SECTION 2A.8 - Deploy the GPU machine (every field)

WHERE: Hyperstack console > Deploy New Virtual Machine form

GOAL: one GPU computer that is starting up.

Step 1. Fill the form one field at a time, exactly as in this table.

Field (in the form)	Value	Why
Flavor (GPU card)	A100-80G-PCIe, GPU count 1 (if sold out: L40, count 1)	80 GB lets the teacher (14B) and your model (8B) run together in Parts 5-6
Environment	medscriptai-env	holds your key
OS image (tab OS Images)	Server 24.04 LTS R570 CUDA 12.8 with Docker	NVIDIA driver 570 + CUDA 12.8 + Docker already installed [4]
SSH key	laptop-ed25519	lets you log in
SSH Access	ON (toggle enabled)	opens port 22 in the firewall
Public IP Address	ON (toggle enabled)	you connect over the internet
Number of VMs to deploy	1	one machine

Virtual Machine Name (Additional settings)	medscriptai-gpu	easy to recognise
Jupyter Notebook, Cloud-init Script, Labels, Firewall Rules	leave empty / off	not needed

Step 2. Look at Final Configuration and Price Breakdown and check the hourly price (about US$1.35 per hour for 1x A100 PCIe, verify).

Step 3. Click Deploy.

You should see: the Virtual Machines page with medscriptai-gpu and a status such as CREATING or BUILD.

Watch out: billing starts when the status becomes ACTIVE [2].

If you see...	It means...	Do this next
A100-80G-PCIe is greyed out	sold out in this region	choose 1x L40, or create a second environment in another region and import the key there too
the image "Server 24.04 LTS R570 CUDA 12.8 with Docker" is missing	different images in this region	choose "Server 22.04 LTS R570 CUDA 12.8 with Docker" (same driver)
only "Server 24.04 LTS R595 CUDA 13.2 with Docker" fits	newer driver	use it; later set VLLM_IMAGE=vllm/vllm-openai:v0.32.0 and VLLM_ENABLE_CUDA_COMPATIBILITY=0 (Part 5)

CHECKPOINT: medscriptai-gpu appears in the Virtual Machines list.

SECTION 2A.9 - Wait for ACTIVE, copy the IP, check the SSH firewall

WHERE: Hyperstack console > Virtual Machines

GOAL: the VM is running and reachable on port 22.

Step 1. Wait until the Status column shows ACTIVE (usually 2-10 minutes; refresh the page).

Step 2. In the PUBLIC IP column, copy the address and paste it into secrets.txt as VM_IP = ...

Step 3. Click the VM name medscriptai-gpu (the VM details page opens).

Step 4. Click the Firewall tab.

Step 5. Make sure Enable SSH access is ON.

Step 6. Under Inbound Rules, check there is a TCP port 22 rule with status SUCCESS.

Step 7. Click the Networking tab and find Connect over SSH: it shows a command like ssh ubuntu@38.80.123.45 (you will add your key to it in the next section).

You should see: ACTIVE, a public IP, and the port 22 rule.

CHECKPOINT: you have VM_IP and the port 22 rule shows SUCCESS.

If not: no public IP = Networking tab > turn on Public IP Attached.

SECTION 2A.10 - First connection from PowerShell

WHERE: Windows PowerShell on your laptop

GOAL: log in to the GPU computer and see the GPU.

Step 1. Type (replace VM_IP) and press Enter:

```
ssh -i $env:USERPROFILE\.ssh\hyperstack_ed25519 ubuntu@VM_IP
```

Step 2. When asked "Are you sure you want to continue connecting (yes/no/[fingerprint])?" type yes and press Enter.

You should see: a welcome text and the prompt ubuntu@medscriptai-gpu:~$

Step 3. Type and press Enter:

```
nvidia-smi
```

You should see: a table containing "NVIDIA A100 80GB PCIe", "Driver Version: 570.xx" and "CUDA Version: 12.8".

Step 4. Type and press Enter:

```
df -h / /ephemeral
```

You should see: the root disk / (about 100 GB) and a large disk mounted at /ephemeral [6].

Step 5. Type exit and press Enter (you are back on your laptop).

Placeholder	Replace with	Example
VM_IP	the PUBLIC IP from Section 2A.9	38.80.123.45

Watch out: /ephemeral is fast and big but its data can be lost when the VM is hibernated or deleted [6]. We only put re-downloadable caches there. Your project lives in /home/ubuntu.

CHECKPOINT: nvidia-smi showed the A100 and driver 570 or newer.

If not: see Section 2A.14.

SECTION 2A.11 - Connect VS Code to the GPU computer (Remote-SSH)

WHERE: VS Code on your laptop

GOAL: VS Code opens folders and terminals ON the GPU computer.

Step 1. Press F1 (a command box opens at the top).

Step 2. Type Remote-SSH: Open SSH Configuration File and press Enter.

Step 3. Click the first file: C:\Users\YOUR_NAME\.ssh\config

Step 4. Paste this at the end of the file (replace VM_IP and YOUR_NAME):

```
Host medscriptai
    HostName VM_IP
    User ubuntu
    IdentityFile C:\Users\YOUR_NAME\.ssh\hyperstack_ed25519
    ServerAliveInterval 60
```

Step 5. Press Ctrl+S to save.

Step 6. Press F1.

Step 7. Type Remote-SSH: Connect to Host and press Enter.

Step 8. Click medscriptai.

Step 9. If a box asks "Select the platform of the remote host", click Linux.

Step 10. Wait until the bottom-left corner shows SSH: medscriptai (the first time VS Code installs a small server on the VM, about 1 minute).

Step 11. In the top menu click Terminal > New Terminal.

You should see: a terminal panel at the bottom with the prompt ubuntu@medscriptai-gpu:~$

Step 12. In that terminal type hostname and press Enter.

You should see: medscriptai-gpu

Line in the config	What it means
Host medscriptai	a short nickname you choose
HostName VM_IP	the real address
User ubuntu	the login name on Hyperstack Ubuntu images
IdentityFile ...	which private key to use
ServerAliveInterval 60	send a small "I am here" message every 60 s so the connection does not drop

Watch out: from now on, "VS Code Terminal" in this manual ALWAYS means this terminal connected to the GPU computer (bottom-left shows SSH: medscriptai), not your laptop.

CHECKPOINT: hostname prints medscriptai-gpu.

If not: F1 > Remote-SSH: Kill VS Code Server on Host > medscriptai, then repeat Steps 6-10.

SECTION 2A.12 - Install and connect WinSCP (drag-and-drop files)

WHERE: web browser, then WinSCP on your laptop

GOAL: see the GPU computer's files next to your laptop files.

Step 1. Open https://winscp.net/eng/download.php

Step 2. Click DOWNLOAD WINSCP (the installation package).

Step 3. Run the downloaded WinSCP-x.x.x-Setup.exe, choose Install for me only, accept the licence, choose Typical installation, Commander interface, then Install and Finish.

Step 4. Open WinSCP. The Login window appears.

Step 5. Click New Site.

Step 6. Fill the fields:

Field	Value	Why
File protocol	SFTP	file transfer inside SSH
Host name	VM_IP	your GPU computer
Port number	22	SSH port
User name	ubuntu	the login name
Password	(empty)	you use a key

Step 7. Click Advanced...

Step 8. In the left list click SSH > Authentication.

Step 9. Next to Private key file click the ... button.

Step 10. In the file window change the file type filter (bottom right) to All Files (*.*).

Step 11. Open C:\Users\YOUR_NAME\.ssh\hyperstack_ed25519 (the file WITHOUT .pub).

Step 12. WinSCP asks to convert the key to PuTTY format: click OK, then Save (it creates hyperstack_ed25519.ppk).

Step 13. Click OK to close Advanced.

Step 14. Click Save, type medscriptai as the site name, click OK.

Step 15. Click Login.

Step 16. If asked "Continue connecting to an unknown server?", click Accept.

You should see: two panels: your laptop on the left, /home/ubuntu on the right.

How to copy files: drag a file from the right panel to the left panel = download to your laptop; drag from left to right = upload to the GPU computer.

CHECKPOINT: the right panel shows /home/ubuntu.

If not: "Server refused our key" = choose the .ppk file WinSCP created in Step 12.

SECTION 2A.13 - Stop paying: hibernate, restore and delete

WHERE: Hyperstack console > Virtual Machines > click medscriptai-gpu

GOAL: no GPU bill while you are not working (do this EVERY time you stop).

Step 1. In the VS Code Terminal, check nothing important is running: type tmux ls (you will learn tmux in Part 2B; "no server running" is fine).

Step 2. In the Hyperstack console, open the VM details page (click medscriptai-gpu).

Step 3. Hover over the ⋮ menu in the top-right corner.

Step 4. Click Hibernate this VM.

Step 5. In the dialog, leave "Keep Public IP attached?" OFF (cheaper; you get a new IP later) or turn it ON (small hourly charge, same IP).

Step 6. Click Hibernate this VM to confirm.

You should see: status HIBERNATING, then HIBERNATED.

Step 7. To work again: open the VM details page, hover over ⋮ and click Restore (wait for ACTIVE).

Step 8. If the public IP changed: put the new IP in HostName of C:\Users\YOUR_NAME\.ssh\config (Section 2A.11 Step 1-5) and in the WinSCP site (Edit > Host name).

VM status	GPU billed?	What is kept
ACTIVE	yes	everything
SHUTOFF (Stop)	YES, still billed [3]	everything
HIBERNATED	no (only storage and a kept IP are billed) [3]	root disk (/home/ubuntu); /ephemeral may be lost [6]
Deleted	no	nothing

Watch out: "Stop" is NOT a money saver on Hyperstack. Use Hibernate this VM or Delete.

Watch out: restoring needs the GPU flavor to be in stock; if it is not, try later or deploy a new VM and copy your work from the Hugging Face Hub.

CHECKPOINT: when you finish a work session, the VM shows HIBERNATED.

SECTION 2A.14 - Connection troubleshooting

WHERE: PowerShell, VS Code or WinSCP on your laptop, and the Hyperstack console

GOAL: fix any connection problem in a few minutes.

Step 1. Find your symptom in the table and do the fix in the right column.

Symptom	Likely cause	Exact fix
ssh: connect to host ... port 22: Connection timed out	SSH access off, wrong IP, or VM not ACTIVE	1) VM status must be ACTIVE. 2) Firewall tab > Enable SSH access ON. 3) Copy the IP again from the PUBLIC IP column
Permission denied (publickey)	wrong key file or wrong user	use -i with the file WITHOUT .pub and the user ubuntu; the key must be imported in the SAME environment as the VM
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED	new VM on an old IP	in PowerShell run ssh-keygen -R VM_IP, then connect again
VS Code stuck on "Setting up SSH host"	half-installed VS Code server	F1 > Remote-SSH: Kill VS Code Server on Host, connect again
WinSCP "Server refused our key"	key not in PuTTY format	use the .ppk file from Section 2A.12 Step 12
nvidia-smi: command not found	wrong OS image (no driver)	delete the VM and deploy again with an image whose name contains R570 CUDA 12.8

CHECKPOINT: after the fix, Section 2A.10 Step 1 logs you in again.

PART 2B - Project folder + tools on the GPU computer (from an empty disk)

BEFORE YOU START: what you are building in this part

You now have an empty GPU computer. In this part you create the project folder structure by hand, learn the ONE procedure you will use to create every file in this manual, store your secrets safely, install the tools (uv, tmux, the AWS CLI, FakeCloud), and create three separate Python environments.

Why three Python environments? The libraries fight each other: Unsloth needs transformers 5.17 or older and trl 1.15 or older; distilabel (last release January 2025) needs datasets older than 4 and openai older than 2; the evaluation tools want the newest transformers. An environment (venv) is a private folder of packages, so each tool gets the versions it needs.

Environment	Used for	Parts
.venv-train	data scripts, Unsloth SFT and DPO training, publishing to the Hub	4, 5, 6A
.venv-synth	distilabel synthetic data and preference pairs	5, 6A
.venv-eval	evaluation metrics, the judge, the API and its tests	5, 6A, 7

Result at the end of this part: this folder tree exists, three environments work, the GPU test prints True, and FakeCloud answers on port 4566.

```
/home/ubuntu/medscriptai/
├── scripts/        all Python scripts (data, training, synthetic data, evaluation)
├── serve/          scripts that start the vLLM model servers
├── api/            the FastAPI web app and its Dockerfile
├── deploy/         Docker Compose and cloud deployment scripts
├── tests/          a fake model server and API tests
├── requirements/   the package lists for the three environments
├── data/           datasets (created by the scripts)
└── outputs/        adapters, predictions, reports (created by the scripts)
```

SECTION 2B.1 - Create the project folders

WHERE: VS Code Terminal (bottom-left shows SSH: medscriptai)

GOAL: the empty folder tree above.

Step 1. Go to your home folder:

```
cd ~
```

Step 2. Create the project folder and all sub-folders in one command:

```
mkdir -p medscriptai/scripts medscriptai/serve medscriptai/api medscriptai/deploy medscriptai/tests
medscriptai/requirements medscriptai/data medscriptai/outputs
```

Step 3. Go into the project folder:

```
cd ~/medscriptai
```

Step 4. List what you created:

```
ls
```

You should see: api data deploy outputs requirements scripts serve tests

Piece	What it does
cd ~	go to your home folder /home/ubuntu (~ is a shortcut for it)
mkdir -p	make folders; -p also creates parent folders and does not complain if they exist

CHECKPOINT: ls shows the eight folders.

SECTION 2B.2 - Open the project folder in VS Code

WHERE: VS Code (connected, SSH: medscriptai)

GOAL: the Explorer panel on the left shows your project.

Step 1. In the top menu click File > Open Folder...

Step 2. In the box type /home/ubuntu/medscriptai/ and press Enter (or click OK).

Step 3. If asked "Do you trust the authors of the files in this folder?", click Yes, I trust the authors.

Step 4. Press Ctrl+Shift+E to show the Explorer panel.

Step 5. Open a terminal again: Terminal > New Terminal.

You should see: the Explorer shows MEDSCRIPTAI with the eight folders, and the new terminal starts in ~/medscriptai.

CHECKPOINT: pwd in the terminal prints /home/ubuntu/medscriptai

SECTION 2B.3 - THE procedure to create any file (practise it once)

WHERE: VS Code Explorer panel (left)

GOAL: you can create a file in the right folder with the exact content. Every file in this manual is made this way.

Step 1. In the Explorer, right-click the folder where the file must go (for this practice: the top folder MEDSCRIPTAI).

Step 2. Click New File...

Step 3. Type the file name (for this practice: README.md) and press Enter. The empty file opens in the editor.

Step 4. Copy the content from the grey box in this manual (select it, Ctrl+C).

Step 5. Click inside the empty editor and press Ctrl+V.

Step 6. Press Ctrl+S to save (the dot on the file's tab disappears).

Practice content for README.md:

```
# MedScriptAI
Medical post-training project: Llama-3.1-8B-Instruct + QLoRA SFT + DPO, served with vLLM multi-LoRA behind FastAPI.
Built with Llama. Educational use only; not medical advice.
```

Step 7. Check the file from the terminal:

```
cat README.md
```

You should see: the three lines you pasted.

Watch out: (1) the file name must match EXACTLY, including the extension (.py, .sh, .txt, .yml); (2) right-click the CORRECT folder, otherwise the file lands in the wrong place; (3) Google Docs may turn straight quotes into curly quotes if you edit the document: always copy from the grey box, and if Python later reports "invalid character", re-paste the file.

CHECKPOINT: cat README.md prints the text.

SECTION 2B.4 - Store secrets and the model cache location

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: your Hugging Face token and cache settings are loaded in every new terminal, and never written into code.

Step 1. Create the fast cache folder on the big ephemeral disk:

```
sudo mkdir -p /ephemeral/hf-cache && sudo chown ubuntu:ubuntu /ephemeral/hf-cache
```

Step 2. Tell Hugging Face to use it (added to ~/.bashrc, which runs in every new terminal):

```
echo 'export HF_HOME=/ephemeral/hf-cache' >> ~/.bashrc
```

Step 3. Add your token (replace hf_YOUR_TOKEN with the token from secrets.txt):

```
echo 'export HF_TOKEN=hf_YOUR_TOKEN' >> ~/.bashrc
```

Step 4. Load the new settings into this terminal:

```
source ~/.bashrc
```

Step 5. Check both:

```
echo $HF_HOME && echo ${HF_TOKEN:0:6}
```

You should see: /ephemeral/hf-cache and hf_xxx (only the first 6 characters, so the token is not shown in full).

Piece	What it does
sudo	run as administrator (needed to write in /ephemeral)
chown ubuntu:ubuntu	make the user ubuntu the owner, so scripts can write there
echo '...' >> ~/.bashrc	append a line to the file that runs at every terminal start
source ~/.bashrc	run it now in the current terminal
${HF_TOKEN:0:6}	the first 6 characters of the variable

Watch out: if /ephemeral does not exist on your flavor, use /home/ubuntu/hf-cache instead in Steps 1-2 (without sudo).

CHECKPOINT: both values print.

SECTION 2B.5 - Install system tools and learn tmux (jobs that survive disconnects)

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: tmux, git, jq and unzip installed; you can start, leave and re-enter a tmux session.

Step 1. Install the tools:

```
sudo apt-get update && sudo apt-get install -y tmux git unzip jq
```

Step 2. Start a practice tmux session:

```
tmux new -s practice
```

Step 3. In the tmux session type a long-running command:

```
for i in $(seq 1 300); do echo "tick $i"; sleep 1; done
```

Step 4. Detach (leave it running): press Ctrl+B, release both keys, then press D.

You should see: "[detached (from session practice)]" and your normal prompt.

Step 5. List sessions:

```
tmux ls
```

You should see: practice: 1 windows ...

Step 6. Re-attach:

```
tmux attach -t practice
```

You should see: the ticks still counting.

Step 7. Stop the loop with Ctrl+C, then type exit and press Enter to close the session.

Key or command	Meaning
tmux new -s NAME	start a new session called NAME
Ctrl+B then D	detach: leave the session running in the background
tmux ls	list running sessions
tmux attach -t NAME	go back into a session
exit (inside tmux)	close the session

Why it matters: training runs for 10-60 minutes. If your laptop sleeps or Wi-Fi drops, a normal terminal kills the job; a tmux session keeps it running on the GPU computer.

CHECKPOINT: you detached and re-attached once.

SECTION 2B.6 - Install uv (Python package manager)

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: the uv command works.

Step 1. Install:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Step 2. Load it into this terminal:

```
source ~/.bashrc
```

Step 3. Check:

```
uv --version
```

You should see: uv 0.x.x (any recent version, 0.8 or newer)

Why uv: it creates environments and installs packages 10-100x faster than pip, and it can pick the right PyTorch build for your GPU driver.

CHECKPOINT: uv --version prints a version.

If not: "uv: command not found" = close the terminal (trash-can icon), open Terminal > New Terminal, try again.

SECTION 2B.7 - Create the three requirements files

WHERE: VS Code Explorer, folder requirements

GOAL: three text files listing the exact package versions for each environment.

FILE 1: requirements/train.txt. Create it with the procedure of Section 2B.3:

Step 1. Right-click the folder requirements.

Step 2. Click New File...

Step 3. Type train.txt and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
# Training + data prep venv (.venv-train). Install with:
#   uv pip install -r requirements/train.txt --torch-backend cu126
unsloth==2026.10.7
unsloth_zoo==2026.10.7
jinja2
huggingface_hub
```

FILE 2: requirements/synth.txt

Step 6. Right-click the folder requirements.

Step 7. Click New File...

Step 8. Type synth.txt and press Enter.

Step 9. Paste the grey box below.

Step 10. Press Ctrl+S.

```
# Synthetic-data venv (.venv-synth). distilabel 1.5.3 (Jan 2025) needs these older pins.
distilabel[openai]==1.5.3
datasets<4
openai<2
requests
beautifulsoup4
```

FILE 3: requirements/eval.txt

Step 11. Right-click the folder requirements.

Step 12. Click New File...

Step 13. Type eval.txt and press Enter.

Step 14. Paste the grey box below.

Step 15. Press Ctrl+S.

```
# Evaluation venv (.venv-eval). Install with:
#   uv pip install -r requirements/eval.txt --torch-backend cu126
openai==3.35.1
rouge-score==0.1.2
bert-score==0.3.12
transformers==5.20.2
torch
fastapi==0.156.8
uvicorn==0.54.0
langsmith==0.14.11
```

```
httpx
pytest
```

Line	What it means
# ...	a comment line, ignored by the installer
unsloth==2026.10.7 and unsloth_zoo==2026.10.7	exact Unsloth version; it brings matching trl, peft, transformers, bitsandbytes, datasets and torch versions
jinja2	the template engine used by chat templates (the validator needs it)
huggingface_hub	upload/download to the Hugging Face Hub
distilabel[openai]==1.5.3	distilabel with its OpenAI-compatible client (talks to vLLM)
datasets<4, openai<2, requests, beautifulsoup4	the older versions distilabel 1.5.3 works with; we found during testing that without them distilabel fails to import
openai==3.35.1	the client used to call vLLM in evaluation and the API
rouge-score, bert-score	the two text-similarity metrics
transformers==5.20.2, torch	needed by BERTScore
fastapi, uvicorn, langsmith, httpx, pytest	the web API, its server, tracing, and the test tools

CHECKPOINT: ls requirements shows eval.txt synth.txt train.txt

SECTION 2B.8 - Create the three Python environments

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: .venv-train, .venv-synth and .venv-eval with all packages installed.

Step 1. Create the training environment:

```
uv venv .venv-train --python 3.12
```

You should see: "Creating virtual environment at: .venv-train"

Step 2. Install the training packages (3-6 minutes):

```
uv pip install --python .venv-train -r requirements/train.txt --torch-backend cu126
```

You should see: a list ending with lines such as + torch==2.14.1+cu126, + trl==1.15.0, + transformers==5.17.0, + unsloth==2026.10.7

Step 3. Create the synthetic-data environment:

```
uv venv .venv-synth --python 3.12
```

Step 4. Install its packages:

```
uv pip install --python .venv-synth -r requirements/synth.txt
```

You should see: + distilabel==1.5.3, + datasets==3.6.0, + openai==1.109.1 among the lines.

Step 5. Create the evaluation environment:

```
uv venv .venv-eval --python 3.12
```

Step 6. Install its packages:

```
uv pip install --python .venv-eval -r requirements/eval.txt --torch-backend cu126
```

Step 7. Check all three exist:

```
ls -a
```

You should see: .venv-eval .venv-synth .venv-train among the folders.

How you will use them later (memorise this):

You want to...	Type first
run a data or training script	source ~/medscriptai/.venv-train/bin/activate
run a distilabel script	source ~/medscriptai/.venv-synth/bin/activate

run evaluation, the judge, the API or tests	source ~/medscriptai/.venv-eval/bin/activate
leave any environment	deactivate

The prompt then starts with (.venv-train), (.venv-synth) or (.venv-eval), so you always see which one is active.

Piece	What it does
uv venv NAME --python 3.12	creates a private Python 3.12 folder called NAME (uv downloads Python 3.12 if needed)
--python .venv-train	install INTO that environment without activating it
-r requirements/train.txt	read the package list from the file
--torch-backend cu126	use the PyTorch build for CUDA 12.6, which runs on your driver 570 (CUDA 12.8). PyTorch 2.14 has no cu128 build, so cu126 is the right choice

Watch out: running a script in the wrong environment is the most common error in this project. Look at the prompt before every command.

CHECKPOINT: the three .venv folders exist and Step 2, 4 and 6 ended without red error text.

If not: "No solution found when resolving dependencies" = run uv self update, then repeat the failed step.

SECTION 2B.9 - GPU test (PyTorch and Docker)

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: proof that Python and Docker both see the GPU.

Step 1. Activate the training environment:

```
source .venv-train/bin/activate
```

Step 2. Run the PyTorch check:

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available(), torch.cuda.get_device_name(0),
torch.cuda.is_bf16_supported(), round(torch.cuda.get_device_properties(0).total_memory/1e9), 'GB')"
```

You should see: 2.14.1+cu126 True NVIDIA A100 80GB PCIe True 85 GB

Step 3. Leave the environment:

```
deactivate
```

Step 4. Check Docker can use the GPU:

```
docker run --rm --gpus all ubuntu:24.04 nvidia-smi
```

You should see: the same nvidia-smi table as in Section 2A.10, printed from inside a container.

If you see...	It means...	Do this next
permission denied while trying to connect to the Docker daemon socket	your user is not in the docker group	run sudo usermod -aG docker ubuntu, then close VS Code's remote window (File > Close Remote Connection), reconnect, and repeat Step 4
could not select device driver "" with capabilities: [[gpu]]	NVIDIA container toolkit missing	you picked an image without "with Docker"; redeploy with the right image (Section 2A.8)
torch.cuda.is_available() False	CPU build installed	uv pip install --python .venv-train -r requirements/train.txt --torch-backend cu126 --reinstall

Piece	What it means
torch.cuda.is_available()	True = PyTorch can use the NVIDIA driver
is_bf16_supported()	bfloat16 = the 16-bit number format used for training; A100/L40/H100 support it
--gpus all	give the container access to the GPU

CHECKPOINT: Step 2 ends with True ... 85 GB and Step 4 shows the GPU table.

SECTION 2B.10 - Create .gitignore and start a Git repository (for your portfolio)

WHERE: VS Code Explorer and Terminal, folder ~/medscriptai

GOAL: your project is version-controlled; secrets and big files are never committed.

Step 1. Right-click the top folder MEDSCRIPTAI in the Explorer.

Step 2. Click New File...

Step 3. Type .gitignore and press Enter.

Step 4. Paste the grey box below and press Ctrl+S.

```
# secrets
.env
deploy/.env
*.pem
# big or generated files (adapters/datasets go to the Hugging Face Hub, not git)
data/
outputs/
unsloth_compiled_cache/
# Python environments and caches
.venv*/
__pycache__/
*.pyc
.pytest_cache/
```

Step 5. In the terminal set your name and e-mail for Git (replace both):

```
git config --global user.name "YOUR NAME" && git config --global user.email "you@example.com"
```

Step 6. Start the repository and make the first commit:

```
git init && git add . && git commit -m "Project skeleton"
```

You should see: "[main (root-commit) ...] Project skeleton" and a list of the files.

Line in .gitignore	Why
.env, deploy/.env, *.pem	secret files must never be in Git
data/, outputs/	datasets and adapters are large; they go to the Hugging Face Hub instead
.venv*/	environments can be recreated from requirements/
__pycache__/, *.pyc, .pytest_cache/	temporary Python files

Watch out: commit after every working section (git add . && git commit -m "what you did"). In Part 9 you push the repository to GitHub.

CHECKPOINT: git log --oneline shows "Project skeleton".

SECTION 2B.11 - Install FakeCloud (free AWS) and the AWS CLI

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: the aws command works against FakeCloud now, and against real AWS when you choose.

BEFORE YOU START: what FakeCloud is. FakeCloud is an open-source program (AGPL-3.0) that pretends to be AWS on your own machine at http://localhost:4566. You use the normal aws command; only the address changes. We tested it while writing this manual: ECR (including real docker push), S3, IAM, Secrets Manager, CloudWatch, EC2 and the SageMaker control plane all work. It has no GPU and cannot run your model; it is for practising the AWS steps for free.

Step 1. Download the AWS CLI v2 installer:

```
cd /tmp && curl -s "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
```

Step 2. Unpack and install it:

```
unzip -q awscliv2.zip && sudo ./aws/install && cd ~/medscriptai
```

Step 3. Check:

```
aws --version
```

You should see: aws-cli/2.x.x Python/3.x Linux/...

Step 4. Install FakeCloud version 0.51.0 (pinned; the installer's "latest" lookup can be blocked by GitHub limits):

```
curl -fsSL https://fakecloud.dev/install.sh | bash -s -- --version v0.51.0
```

You should see: "binary: /usr/local/bin/fakecloud" and "Start the emulator: fakecloud"

Step 5. Create a folder for FakeCloud's saved state:

```
mkdir -p ~/fakecloud-data
```

Step 6. Start FakeCloud in the background:

```
nohup fakecloud --storage-mode persistent --data-path ~/fakecloud-data > ~/fakecloud.log 2>&1 &
```

Step 7. Check it is healthy:

```
curl -s http://localhost:4566/_fakecloud/health | head -c 120
```

You should see: {"status":"ok","version":"0.51.0","services":[...

Step 8. Configure the CLI for the column you use. Do every row of your column:

Step	FakeCloud (free practice)	Real AWS (same step)
Credentials	`export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test` (FakeCloud accepts any values)	AWS console > IAM > Users > Create user > name medscriptai-admin > Attach policies directly > AdministratorAccess > Create user > open the user > Security credentials > Create access key > Command Line Interface (CLI) > copy both keys. Then run `aws configure` and paste them
Region	`export AWS_DEFAULT_REGION=us-east-1`	in `aws configure` type us-east-1 and output format json
Endpoint	`export AWS_ENDPOINT_URL=http://localhost:4566`	`unset AWS_ENDPOINT_URL`
Test	`aws sts get-caller-identity`	`aws sts get-caller-identity`
You should see	"Account": "123456789012" (FakeCloud's fake account)	your real 12-digit account number
Safety	nothing to do	AWS console > Billing and Cost Management > Budgets > Create budget > Use a template > Monthly cost budget > US$10 > your e-mail > Create budget. Also enable MFA on the root user

Piece	What it does
nohup ... &	run in the background and keep running after you close the terminal
--storage-mode persistent --data-path	FakeCloud saves what you create, so it survives a restart
> ~/fakecloud.log 2>&1	write its messages into ~/fakecloud.log
AWS_ENDPOINT_URL	the ONE variable that decides FakeCloud or real AWS

Watch out: before every real-AWS step run echo $AWS_ENDPOINT_URL; it must print an empty line. export lines only last for the current terminal: in a new terminal, repeat the FakeCloud exports.

Watch out: never use root-user keys; never put access keys in code or Git.

CHECKPOINT: with the FakeCloud exports, aws sts get-caller-identity returns 123456789012.

If not: "Could not connect to the endpoint URL" = FakeCloud is not running: repeat Step 6 and read tail ~/fakecloud.log.

SECTION 2B.12 - (Real AWS only) Request GPU quota early

WHERE: web browser > AWS console, region us-east-1 (top-right region menu: US East (N. Virginia))

GOAL: permission to start one GPU machine in Part 6B (new accounts have 0; approval takes hours to days).

Step 1. Open https://console.aws.amazon.com/servicequotas

Step 2. Click AWS services.

Step 3. Search and click Amazon Elastic Compute Cloud (Amazon EC2).

Step 4. Search Running On-Demand G and VT instances and click it.

Step 5. Click Request increase at account level.

Step 6. In Increase quota value type 8 and click Request.

You should see: the request with status Pending (later Approved, by e-mail).

CHECKPOINT: the request exists. Continue with Part 3 and Part 4 while you wait.

PART 2 RESULT CHECK (all must be true before Part 4)

Check	Command	Expected
GPU visible	nvidia-smi	NVIDIA A100 80GB PCIe
folders exist	ls ~/medscriptai	api data deploy outputs requirements scripts serve tests
three environments	ls -a ~/medscriptai	.venv-train .venv-synth .venv-eval
PyTorch on GPU	Section 2B.9 Step 2	... True ... 85 GB
Docker on GPU	docker run --rm --gpus all ubuntu:24.04 nvidia-smi	GPU table
token loaded	echo ${HF_TOKEN:0:3}	hf_
FakeCloud	curl -s localhost:4566/_fakecloud/health	"status":"ok"
tmux	tmux ls	no error (or "no server running")

PART 3 - The Core Idea (read this before you write any code)

3.1 One analogy for the whole project

Llama-3.1-8B-Instruct is a smart new graduate doctor who knows a lot but does not write in your hospital's style. SFT (supervised fine-tuning) is residency: you show thousands of worked examples and the doctor imitates them. DPO (preference optimisation) is a senior consultant who looks at two of the doctor's notes and says "this one is better"; the doctor shifts towards the better habits. LoRA means you do not rewrite the doctor's whole brain; you hand them a slim specialty notebook (an adapter). QLoRA compresses the textbook to fit in a backpack (4-bit) while the notebook stays sharp. vLLM multi-LoRA is one doctor carrying three notebooks (qa, notes, summary) and opening the right one for each patient in milliseconds.

3.2 Tokens and the chat template

A model never sees letters. A tokenizer cuts text into tokens (word pieces) and turns each into a number. Llama-3.1 has 128,256 different tokens. "Nitrofurantoin" becomes several tokens; "the" is one.

A chat model also needs to know WHO is speaking. Llama-3.1 uses special tokens around every message. This is the exact text the model sees for one of your training examples (shortened):

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 26 Jul 2024

You are MedScriptAI, a careful medical education assistant. ...<|eot_id|><|start_header_id|>user<|end_header_id|>

A patient presents with symptoms of Hypoglycemia. ... Options:
A) Insulinoma
B) Accidental sulfonylurea ingestion
C) Accidental exogenous Insulin administration
D) Accidental Metformin ingestion<|eot_id|><|start_header_id|>assistant<|end_header_id|>

Answer: C) Accidental exogenous Insulin administration
Explanation: ...<|eot_id|>
```

Token	Meaning
<|begin_of_text|>	start of the whole conversation (must appear exactly once)
<|start_header_id|>role<|end_header_id|>	"the next message is from system / user / assistant"
<|eot_id|>	end of this message
Cutting Knowledge Date ...	Llama-3.1's template adds this to every system message automatically

Rule: training and serving must produce EXACTLY this format. That is why every script imports the same system prompts from one file (scripts/common.py).

3.3 What "training" actually does

The model predicts the next token, one at a time. Training shows it a correct example and measures how surprised it was by each real next token. That surprise is the loss (cross-entropy). An optimizer nudges the weights so that next time the loss is a bit lower.

Tiny example: after "Answer: C) Accidental exogenous", the correct next token is "Insulin". If the model gave "Insulin" a probability of 0.05, the loss for that token is -ln(0.05) = 3.0. After training it gives 0.9, loss -ln(0.9) = 0.1.

Loss masking: we only want the model to learn to WRITE ANSWERS, not to write questions. So the loss is computed only on the assistant part; the system and user tokens are masked (their label is set to -100, which means "ignore").

Word	Plain meaning
step	one update of the weights
batch	how many examples are used for one step (we use 16)
epoch	one full pass through the training data
learning rate (LR)	how big each nudge is (too big = chaos, too small = no learning)

validation loss	the loss on examples the model does NOT train on; if it goes up while training loss goes down, the model is memorising (overfitting)

3.4 LoRA in numbers

One attention matrix in Llama-3.1-8B (q_proj) is 4,096 × 4,096 = 16.8 million numbers. Full fine-tuning would change all of them, in every one of the 32 layers, plus all the other matrices: 8 billion numbers, about 16 GB of weights plus optimizer state that is several times bigger.

LoRA freezes the big matrix W and learns two thin matrices B (4,096 × 16) and A (16 × 4,096) instead. Their product B x A has the same shape as W, but only 2 × 4,096 × 16 = 131,072 trainable numbers. With adapters on 7 matrices in 32 layers you train about 42 million numbers (0.5% of the model). The saved adapter file is only about 80-170 MB.

LoRA setting	Our value	Meaning
r (rank)	16	width of the thin matrices; 8-16 for focused tasks, 32-64 for big behaviour changes
lora_alpha	16	scaling: the update is multiplied by alpha / r = 1.0
target_modules	q, k, v, o, gate, up, down	which matrices get an adapter (all linear layers works best)

3.5 QLoRA: making an 8B model fit

QLoRA stores the frozen base weights in 4-bit (the NF4 format) instead of 16-bit: 16 GB becomes about 5.7 GB. The small LoRA matrices stay in 16-bit and are the only part trained. Quality stays close to full 16-bit LoRA (QLoRA paper, Dettmers et al. 2023). Unsloth adds fast custom GPU code on top, about 2x faster with less memory.

3.6 Data: train, validation, test, and leakage

Split	Used for	Rule
train	the model learns from it	the biggest part
validation (val)	watch the loss during training	never trained on
test	final scores you report	never seen until evaluation; never used to tune

Leakage (contamination) means a test question, or a near-copy of it, is also in the training data. Then the score measures memory, not skill. While building this manual, the full-data run found that the clinical-notes dataset repeats 927 notes up to 5 times under different IDs, so 14-18% of the notes/summary test items had a copy in training. The data script now removes repeated notes before splitting (result: 0.0%). Lesson: always check.

3.7 Synthetic data (Part 5): what it is and why

Synthetic data = training examples written by another AI model instead of by humans. Our problem: MedQA gives only the correct letter, for example "Answer: D) Nitrofurantoin", with no reasoning. A model trained on letters learns to guess. So we ask a bigger, stronger model (the teacher, Qwen2.5-14B-Instruct) to explain WHY the given correct answer is right. This is called rationale generation: the teacher is told the answer, so it explains instead of guessing.

Before and after (one MedQA training target):

```
BEFORE:  Answer: D) Nitrofurantoin
AFTER:   Answer: D) Nitrofurantoin
         Explanation: Nitrofurantoin is a first-line choice for uncomplicated cystitis in pregnancy because ...
```

distilabel is the library that runs this as a pipeline: load rows → send each to the teacher → collect the replies → save. A quality filter then throws away bad explanations (too short, too long, or not mentioning the correct option). Risk: the teacher can be wrong in subtle ways, so you always read a random sample yourself.

3.8 DPO (Part 6A): learning from "better vs worse"

DPO needs triples: a prompt, a chosen answer (better) and a rejected answer (worse). We create them from your own SFT model: it answers each prompt twice at high temperature (so the answers differ), the teacher answers once, the teacher rates all three (1-5), and for qa we also check whether the letter is correct. Best = chosen, worst = rejected.

DPO then raises the probability of chosen answers and lowers rejected ones, while a reference model (a frozen copy of your SFT adapter) keeps it from drifting too far. beta = 0.1 controls the strength of that leash. At step 0 the model equals the reference, so the DPO loss is exactly ln 2 = 0.693; it falls as the model learns the preferences.

3.9 Evaluation (Parts 5-7): five kinds of evidence

Method	Question it answers	Example
accuracy	did it pick the correct letter?	312 correct of 500 = 0.624
ROUGE-L	how much of the reference's word order does the answer share?	reference "acute kidney injury", answer "kidney injury" → high overlap
BERTScore	is the MEANING similar, even with different words?	"heart attack" vs "myocardial infarction" scores high
LLM-as-a-judge	which answer would an expert prefer?	the teacher compares two notes; we ask twice with the order swapped to remove position bias
statistics	is a difference real or luck?	a 95% confidence interval that does not contain 0

3.10 Serving and production (Parts 6A-6B)

vLLM loads the base model once on the GPU and applies a small adapter per request. Clients talk to it with the OpenAI API format; the "model" field chooses the adapter:

```
{"model": "qa-dpo", "messages": [{"role": "user", "content": "Which drug is safe ...?"}]}
```

Piece	Job
FastAPI app	your public endpoint: checks the API key, picks the adapter for the task, adds the system prompt, calls vLLM, returns JSON
LangSmith	records each request so you can debug latency and bad answers
Docker image	the API packaged with its exact dependencies, runs the same everywhere
ECR	AWS's private registry where images are stored
EC2 g6.xlarge	the AWS GPU computer that runs vLLM + API
IAM role, security group, Secrets Manager, CloudWatch	who may do what, which network ports are open, where passwords live, where logs go
FakeCloud	practise all AWS commands for free before paying for real AWS

CHECKPOINT: you can explain in your own words: what a token is, why the loss is masked, what r=16 means, why the test set must not leak, what synthetic data is for, what chosen/rejected mean, and why we ask the judge twice.

PART 4 - BASICS: your first medical fine-tune (data → .jsonl → QLoRA SFT → test → publish)

BEFORE YOU START: what you will build in this part

You will create 6 files by hand, run them in order, and end with your first trained adapter (sft-qa) that answers medical exam questions in a fixed format, published on the Hugging Face Hub and copied to (Fake)S3.

```
01_download_data.py  -> data/raw/*.jsonl        (7 files, untouched copies of the datasets)
02_build_sft_jsonl.py -> data/sft/*.jsonl       (9 files: qa / notes / summary x train / val / test)
                         data/pref/*.jsonl      (3 files of prompts kept aside for DPO in Part 6A)
03_validate_jsonl.py  -> "ALL_OK"               (proof the files are correct before using the GPU)
10_train_sft.py       -> outputs/adapters/sft-qa (your first LoRA adapter)
11_chat_test.py       -> base vs fine-tuned answers side by side
12_push_to_hub.py     -> your adapter on huggingface.co
```

The four datasets (all open, all free):

Dataset	What it contains	Licence	Used for
MedQA (USMLE 4-options)	US medical licensing exam questions, correct letter only	CC-BY-4.0	qa
MedMCQA	Indian medical entrance exam questions with expert explanations	Apache-2.0	qa (+ DPO prompts)
PubMedQA (labelled)	research abstract + yes/no/maybe question + long answer	MIT	qa
augmented-clinical-notes	case reports with a doctor-patient conversation and a JSON summary	MIT	notes and summary

What one training example looks like after Step 4.5 (this is a REAL line from the data):

```
{"task": "qa", "source": "medmcqa", "answer_letter": "C",
 "messages": [
  {"role": "system", "content": "You are MedScriptAI, a careful medical education assistant. ..."},
  {"role": "user", "content": "A patient presents with symptoms of Hypoglycemia. ... C-peptide ... normal levels
...\n\nOptions:\nA) Insulinoma\nB) Accidental sulfonylurea ingestion\nC) Accidental exogenous Insulin administration\nD)
Accidental Metformin ingestion"},
  {"role": "assistant", "content": "Answer: C) Accidental exogenous Insulin administration\nExplanation: ... C peptide
... is suppressed ..."}]}
```

This "messages" format (system, user, assistant) is the conversational format that TRL, Unsloth and vLLM all understand.
Time and cost: about 1.5 GPU-hours (about US$2). The training run itself is roughly 10-15 minutes on an A100 (estimate; your progress bar shows the real speed).

SECTION 4.1 - Create scripts/common.py (settings shared by every script)

WHERE: VS Code Explorer (left panel), folder scripts

GOAL: one file that holds folder paths, model names and the three system prompts.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type common.py and press Enter.

Step 4. Paste the grey box below (Ctrl+V).

Step 5. Press Ctrl+S.

```
"""Shared settings and helpers used by every MedScriptAI script."""
import json                     # read/write JSON text
import os                       # read environment variables
from pathlib import Path        # build file paths that work on any OS

# Project folders. ROOT is the repo folder (one level above scripts/).
ROOT = Path(__file__).resolve().parent.parent
DATA = ROOT / "data"            # all datasets live here
RAW = DATA / "raw"              # untouched downloads
SFT = DATA / "sft"              # chat-format .jsonl for SFT
PREF = DATA / "pref"            # preference pairs for DPO
OUT = ROOT / "outputs"          # adapters, predictions, reports

# Base model. The unsloth mirror is ungated; you still must accept Meta's licence.
BASE_MODEL = os.getenv("BASE_MODEL", "unsloth/Llama-3.1-8B-Instruct")
BASE_MODEL_4BIT = os.getenv("BASE_MODEL_4BIT", "unsloth/Llama-3.1-8B-Instruct-bnb-4bit")
```

```
# One system prompt per task = one LoRA adapter per task.
SYSTEM = {
    "qa": (
        "You are MedScriptAI, a careful medical education assistant. "
        "Answer the multiple-choice question. First line: 'Answer: <letter>) <option>'. "
        "Then give a short explanation. Educational use only, not medical advice."
    ),
    "notes": (
        "You are MedScriptAI, a clinical documentation assistant. "
        "Write a concise clinical case note from the doctor-patient conversation. "
        "Use only facts stated in the conversation. Do not invent findings."
    ),
    "summary": (
        "You are MedScriptAI, a clinical information extractor. "
        "Return ONLY valid JSON that summarises the clinical note. "
        "Include only fields that are stated in the note."
    ),
}


def write_jsonl(path, rows):
    """Write a list of dicts to a .jsonl file (one JSON object per line)."""
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)       # create folders if missing
    with path.open("w", encoding="utf-8") as f:
        for row in rows:
            f.write(json.dumps(row, ensure_ascii=False) + "\n")
    print(f"wrote {len(rows):>6} rows -> {path.relative_to(ROOT)}")


def read_jsonl(path):
    """Read a .jsonl file back into a list of dicts."""
    with Path(path).open(encoding="utf-8") as f:
        return [json.loads(line) for line in f if line.strip()]


def chat(task, user, assistant=None):
    """Build the 'messages' list in the chat format TRL and vLLM understand."""
    msgs = [{"role": "system", "content": SYSTEM[task]},
            {"role": "user", "content": user}]
    if assistant is not None:
        msgs.append({"role": "assistant", "content": assistant})
    return msgs
```

Line	What it does (plain English)
`import json`, `import os`, `from pathlib import Path`	standard Python tools: JSON text, environment variables, file paths that work on every OS
`ROOT = Path(__file__).resolve().parent.parent`	the project folder = two levels above this file (scripts/common.py → medscriptai)
`DATA`, `RAW`, `SFT`, `PREF`, `OUT`	one variable per folder, so no script hard-codes a path
`BASE_MODEL = os.getenv("BASE_MODEL", "unsloth/Llama-3.1-8B-Instruct")`	the 16-bit base model; os.getenv lets you override it with an environment variable without editing code
`BASE_MODEL_4BIT = ...-bnb-4bit`	the same model already compressed to 4-bit NF4 by Unsloth (5.7 GB download instead of 16 GB)
`SYSTEM = {...}`	one system prompt per task; one task = one adapter. The API in Part 6A imports this SAME dict, so training and serving prompts can never differ
`def write_jsonl(path, rows)`	writes a list of dicts, one JSON object per line; `mkdir(parents=True, exist_ok=True)` creates missing folders
`json.dumps(row, ensure_ascii=False)`	dict → JSON text; ensure_ascii=False keeps symbols such as °C readable
`print(f"wrote {len(rows):>6} rows -> ...")`	progress message; `:>6` right-aligns the number in 6 characters
`def read_jsonl(path)`	reads a .jsonl file back into a list of dicts; `if line.strip()` skips empty lines
`def chat(task, user, assistant=None)`	builds the messages list: system, user and (for training rows) the assistant answer

Step 6. Check the file has no typing errors:

```
cd ~/medscriptai && .venv-train/bin/python -c "import sys; sys.path.insert(0,'scripts'); import common;
print(common.BASE_MODEL, list(common.SYSTEM))"
```

You should see: unsloth/Llama-3.1-8B-Instruct ['qa', 'notes', 'summary']

CHECKPOINT: the line above prints.

If not: SyntaxError = the paste was cut or changed; delete everything in the file (Ctrl+A, Delete), paste again, save.

SECTION 4.2 - Create scripts/01_download_data.py

WHERE: VS Code Explorer, folder scripts

GOAL: the script that downloads the four datasets.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 01_download_data.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Step 1: download the open medical datasets and save them untouched as .jsonl."""
import argparse                                  # read command-line options
from datasets import load_dataset                # Hugging Face dataset loader
from common import RAW                           # data/raw folder

# name on the Hub, config, split, file we save, rows to keep as (start, end) after a seeded shuffle
SOURCES = [
    ("GBaker/MedQA-USMLE-4-options", None, "train", "medqa_train", None),
    ("GBaker/MedQA-USMLE-4-options", None, "test", "medqa_test", None),
    ("openlifescienceai/medmcqa", None, "train", "medmcqa_train", (0, 20000)),
    ("openlifescienceai/medmcqa", None, "train", "medmcqa_dpo", (20000, 23000)),  # unseen prompts for DPO
    ("openlifescienceai/medmcqa", None, "validation", "medmcqa_val", None),
    ("qiaojin/PubMedQA", "pqa_labeled", "train", "pubmedqa", None),
    ("AGBonnet/augmented-clinical-notes", None, "train", "clinical_notes", (0, 12000)),
]

parser = argparse.ArgumentParser()
parser.add_argument("--limit", type=int, default=None, help="tiny test run: keep N rows per file")
args = parser.parse_args()

RAW.mkdir(parents=True, exist_ok=True)           # make data/raw if it does not exist
for name, config, split, out_name, rows in SOURCES:
    ds = load_dataset(name, config, split=split)  # downloads once, then uses the local cache
    if rows:                                     # same seed = same order, so slices never overlap
        ds = ds.shuffle(seed=42).select(range(rows[0], min(rows[1], len(ds))))
    if args.limit:                               # --limit: tiny test run
        ds = ds.select(range(min(args.limit, len(ds))))
    path = RAW / f"{out_name}.jsonl"
    ds.to_json(str(path), lines=True, force_ascii=False)  # one JSON object per line
    print(f"{out_name:16} {len(ds):>6} rows  <- {name} [{split}]")
```

Line	What it does (plain English)
`import argparse`	lets you pass options such as --limit on the command line
`from datasets import load_dataset`	Hugging Face's loader: downloads once, caches in HF_HOME, returns a table
`from common import RAW`	the data/raw folder from common.py
`SOURCES = [...]`	the shopping list: Hub name, sub-set name, split, output file name, rows to keep
`("GBaker/MedQA-USMLE-4-options", None, "train", "medqa_train", None)`	MedQA training questions; None = keep all rows
`("openlifescienceai/medmcqa", ..., (0, 20000))`	MedMCQA: keep rows 0-19,999 after shuffling (the full set has 182k; 20k is plenty)
`(..., "medmcqa_dpo", (20000, 23000))`	3,000 DIFFERENT MedMCQA questions kept aside for DPO, so they are new to the SFT model
`("qiaojin/PubMedQA", "pqa_labeled", ...)`	the 1,000 expert-labelled PubMedQA questions
`("AGBonnet/augmented-clinical-notes", ..., (0, 12000))`	12,000 clinical notes with conversations and JSON summaries
`parser.add_argument("--limit", ...)`	optional tiny test run: keep only N rows per file

`RAW.mkdir(parents=True, exist_ok=True)`	create data/raw (no error if it exists)
`ds = load_dataset(name, config, split=split)`	download the first time, load from the cache afterwards
`ds.shuffle(seed=42).select(range(rows[0], ...))`	shuffle with a fixed seed (same order every run) and take a slice; same seed = the SFT slice and the DPO slice never overlap
`ds.to_json(str(path), lines=True, force_ascii=False)`	save as .jsonl (lines=True = one object per line)
`print(f"{out_name:16} ...")`	one progress line per file

CHECKPOINT: ls scripts shows 01_download_data.py and common.py

SECTION 4.3 - Run the download

WHERE: VS Code Terminal, inside a tmux session

GOAL: seven raw .jsonl files in data/raw.

Step 1. Start a tmux session:

```
tmux new -s data
```

Step 2. Go to the scripts folder and activate the training environment:

```
cd ~/medscriptai/scripts && source ../.venv-train/bin/activate
```

Step 3. Run:

```
python 01_download_data.py
```

You should see (2-5 minutes; these are the real numbers):

```
medqa_train       10178 rows  <- GBaker/MedQA-USMLE-4-options [train]
medqa_test         1273 rows  <- GBaker/MedQA-USMLE-4-options [test]
medmcqa_train     20000 rows  <- openlifescienceai/medmcqa [train]
medmcqa_dpo        3000 rows  <- openlifescienceai/medmcqa [train]
medmcqa_val        4183 rows  <- openlifescienceai/medmcqa [validation]
pubmedqa           1000 rows  <- qiaojin/PubMedQA [train]
clinical_notes    12000 rows  <- AGBonnet/augmented-clinical-notes [train]
```

Step 4. List the files:

```
ls -lh ../data/raw
```

You should see: seven .jsonl files (clinical_notes.jsonl is the biggest, roughly 150-250 MB).

Watch out: lines with progress bars ("Generating train split") are normal. "429 Too Many Requests" = run echo ${HF_TOKEN:0:3} (must print hf_) and run the script again.

CHECKPOINT: seven files in data/raw.

SECTION 4.4 - Create scripts/02_build_sft_jsonl.py (clean + convert)

BEFORE YOU START: what this script does. Raw datasets all look different (MedQA has an options dict, MedMCQA has opa/opb/opc/opd columns, PubMedQA has abstracts). This script converts every row into the same "messages" format, removes broken rows, removes duplicates, makes sure no test question is in training, and writes train/val/test files. This "boring" step decides most of the final quality.

WHERE: VS Code Explorer, folder scripts

GOAL: the cleaning and conversion script.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 02_build_sft_jsonl.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Step 2: clean the raw data and convert it to chat-format .jsonl (one file per task and split)."""
import json                                     # parse the JSON summaries
import random                                   # repeatable shuffling
```

```
import re                                          # simple text clean-up
from common import RAW, SFT, PREF, chat, read_jsonl, write_jsonl

random.seed(42)                                    # same split every time you run it
MAX_CHARS = 7000                                   # ~1,750 tokens: fits max_seq_length 2048
LETTERS = "ABCD"


def clean(text):
    """Collapse repeated spaces/newlines and strip the ends."""
    return re.sub(r"[ \t]+", " ", re.sub(r"\n{3,}", "\n\n", str(text or ""))).strip()


def key(text):
    """Normalised text used to find duplicates (lower-case, letters and digits only)."""
    return re.sub(r"[^a-z0-9]", "", text.lower())[:300]


def mcq_prompt(question, options):
    """Question + lettered options, the exact user message format for the qa adapter."""
    lines = [f"{letter}) {text}" for letter, text in options]
    return f"{clean(question)}\n\nOptions:\n" + "\n".join(lines)


def short(text, limit=900):
    """Cut long explanations at the last full stop before `limit` characters."""
    text = clean(text)
    if len(text) <= limit:
        return text
    cut = text[:limit].rsplit(".", 1)[0]
    return cut + "."


# ---------- QA task: MedQA + MedMCQA + PubMedQA ----------
def from_medqa(row):
    opts = sorted(row["options"].items())          # [("A", "..."), ("B", "..."), ...]
    letter = row["answer_idx"]
    answer = f"Answer: {letter}) {clean(row['answer'])}"   # no explanation in MedQA (Part 5 adds one)
    return {"task": "qa", "source": "medqa", "answer_letter": letter,
            "question_key": key(row["question"]),
            "messages": chat("qa", mcq_prompt(row["question"], opts), answer)}


def from_medmcqa(row):
    if row.get("choice_type") != "single" or row.get("cop") not in (0, 1, 2, 3):
        return None                                # skip multi-answer or broken rows
    opts = list(zip(LETTERS, [row["opa"], row["opb"], row["opc"], row["opd"]]))
    letter = LETTERS[row["cop"]]
    answer = f"Answer: {letter}) {clean(opts[row['cop']][1])}"
    if row.get("exp") and len(clean(row["exp"])) > 40:
        answer += f"\nExplanation: {short(row['exp'])}"
    return {"task": "qa", "source": "medmcqa", "answer_letter": letter,
            "question_key": key(row["question"]),
            "messages": chat("qa", mcq_prompt(row["question"], opts), answer)}


def from_pubmedqa(row):
    context = clean(" ".join(row["context"]["contexts"]))
    opts = [("A", "yes"), ("B", "no"), ("C", "maybe")]
    letter = {"yes": "A", "no": "B", "maybe": "C"}[row["final_decision"]]
    question = f"Context: {context}\n\nQuestion: {row['question']}"
    answer = f"Answer: {letter}) {row['final_decision']}\nExplanation: {short(row['long_answer'])}"
    return {"task": "qa", "source": "pubmedqa", "answer_letter": letter,
            "question_key": key(row["question"]),
            "messages": chat("qa", mcq_prompt(question, opts), answer)}


# ---------- notes + summary tasks: augmented clinical notes ----------
def compact(value):
    """Drop empty fields ("None", "", [], {}) so the target JSON is short and factual."""
    if isinstance(value, dict):
        out = {k: compact(v) for k, v in value.items()}
        return {k: v for k, v in out.items() if v not in (None, "", [], {})}
    if isinstance(value, list):
        out = [compact(v) for v in value]
        return [v for v in out if v not in (None, "", [], {})]
```

```
    if isinstance(value, str) and value.strip().lower() in ("none", "n/a", ""):
        return None
    return value


def from_clinical(row):
    """One raw row gives one 'notes' example and one 'summary' example (or None if broken)."""
    conversation, note = clean(row["conversation"]), clean(row["note"])
    try:
        summary = compact(json.loads(row["summary"]))   # ~25% of summaries are invalid JSON
    except (json.JSONDecodeError, TypeError):
        return None
    if not conversation or not note or not summary:
        return None
    summary_text = json.dumps(summary, ensure_ascii=False)
    notes_ex = {"task": "notes", "source": "clinical_notes", "id": row["idx"],
                "messages": chat("notes", conversation, note)}
    summary_ex = {"task": "summary", "source": "clinical_notes", "id": row["idx"],
                  "messages": chat("summary", note, summary_text)}
    return notes_ex, summary_ex


def fits(example):
    """Keep only examples short enough for max_seq_length=2048."""
    return sum(len(m["content"]) for m in example["messages"]) <= MAX_CHARS


def build(rows, fn):
    """Apply a converter, drop None/too-long rows."""
    out = [fn(r) for r in rows]
    return [x for x in out if x is not None]


# ---- QA: build, de-duplicate, split ----
def sample(rows, n):
    """Random n rows (or all of them if there are fewer than n)."""
    return random.sample(rows, min(n, len(rows)))


qa_test = (sample(build(read_jsonl(RAW / "medqa_test.jsonl"), from_medqa), 200)
           + sample(build(read_jsonl(RAW / "medmcqa_val.jsonl"), from_medmcqa), 200))
pubmed = build(read_jsonl(RAW / "pubmedqa.jsonl"), from_pubmedqa)
random.shuffle(pubmed)
qa_test += pubmed[:100]                                  # 100 PubMedQA rows only for testing
qa_pool = (build(read_jsonl(RAW / "medqa_train.jsonl"), from_medqa)
           + build(read_jsonl(RAW / "medmcqa_train.jsonl"), from_medmcqa)
           + pubmed[100:])

test_keys = {x["question_key"] for x in qa_test}          # decontamination: never train on test questions
seen, qa_clean = set(), []
for ex in qa_pool:
    k = ex["question_key"]
    if k in seen or k in test_keys or not fits(ex):
        continue                                         # duplicate, leaked, or too long
    seen.add(k)
    qa_clean.append(ex)
random.shuffle(qa_clean)
write_jsonl(SFT / "qa_val.jsonl", qa_clean[:300])
write_jsonl(SFT / "qa_train.jsonl", qa_clean[300:])
write_jsonl(SFT / "qa_test.jsonl", qa_test)

# DPO prompts: 3,000 MedMCQA questions the SFT model never saw (and not in the test set)
dpo_qa = [x for x in build(read_jsonl(RAW / "medmcqa_dpo.jsonl"), from_medmcqa)
          if x["question_key"] not in seen and x["question_key"] not in test_keys and fits(x)]
write_jsonl(PREF / "qa_dpo.jsonl", dpo_qa)

# ---- notes + summary: one shared split so a test note is never a training note ----
pairs, seen_notes = [], set()
for p in build(read_jsonl(RAW / "clinical_notes.jsonl"), from_clinical):
    k = key(p[1]["messages"][1]["content"])             # the note text: ~13% of rows repeat a note
    if k in seen_notes or not (fits(p[0]) and fits(p[1])):
        continue                                         # duplicate note (would leak into test) or too long
    seen_notes.add(k)
    pairs.append(p)
random.shuffle(pairs)
splits = {"test": pairs[:200], "val": pairs[200:400], "dpo": pairs[400:1400], "train": pairs[1400:]}
```

```
for split, rows in splits.items():
    folder = PREF if split == "dpo" else SFT              # DPO prompts are held out from SFT training
    write_jsonl(folder / f"notes_{split}.jsonl", [p[0] for p in rows])
    write_jsonl(folder / f"summary_{split}.jsonl", [p[1] for p in rows])
```

Line	What it does (plain English)
`random.seed(42)`	fixes the random generator: the split is identical every run (reproducible)
`MAX_CHARS = 7000`	about 1,750 tokens (1 token is roughly 4 characters of English): every example fits max_seq_length 2048
`LETTERS = "ABCD"`	maps answer index 0-3 to a letter
`def clean(text)`	None → ""; 3+ newlines → 2; repeated spaces/tabs → 1; trims both ends
`def key(text)`	a fingerprint for finding duplicates: lower-case, letters and digits only, first 300 characters
`def mcq_prompt(question, options)`	builds the user message: question, blank line, "Options:", then "A) ..." lines. The API expects users to send this same shape
`def short(text, limit=900)`	cuts long explanations at the last full stop before 900 characters
`def from_medqa(row)`	one MedQA row → one example; `sorted(row["options"].items())` gives [("A", ...), ("B", ...)]
`answer = f"Answer: {letter}) {clean(row['answer'])}"`	the target, e.g. Answer: D) Nitrofurantoin (no explanation yet; Part 5 adds one)
`"answer_letter"`, `"question_key"`	kept for scoring accuracy later and for de-duplication
`def from_medmcqa(row)`	one MedMCQA row; `if row.get("choice_type") != "single" ... return None` drops multi-answer and broken rows
`opts = list(zip(LETTERS, [row["opa"], ...]))`	pairs A-D with the four option texts
`if row.get("exp") and len(clean(row["exp"])) > 40`	adds the expert explanation only if it is real text
`def from_pubmedqa(row)`	the abstract becomes "Context:", options A) yes B) no C) maybe, long_answer becomes the explanation
`def compact(value)`	removes empty fields ("None", "", [], {}) from the summary JSON (about 70% of the raw fields are "None")
`def from_clinical(row)`	one clinical row → TWO examples: notes (conversation → note) and summary (note → JSON)
`json.loads(row["summary"])` inside try/except	about 25% of the raw summaries are not valid JSON; those rows are skipped
`def fits(example)`	True if all messages together are at most MAX_CHARS characters
`def build(rows, fn)`	apply a converter to every row, drop the None results
`def sample(rows, n)`	n random rows (or all if fewer)
`qa_test = (sample(... medqa_test ..., 200) + sample(... medmcqa_val ..., 200))`	the QA test set comes ONLY from the official test/validation splits
`qa_test += pubmed[:100]`	100 PubMedQA rows for testing; the other 900 may be used for training
`test_keys = {...}` and `if k in seen or k in test_keys or not fits(ex): continue`	skip duplicates, skip anything identical to a TEST question (decontamination), skip too-long examples
`write_jsonl(SFT / "qa_val.jsonl", qa_clean[:300])`	300 validation rows; the rest is training
`dpo_qa = [...]` → `PREF / "qa_dpo.jsonl"`	unseen MedMCQA prompts for DPO, also checked against train and test
`for p in build(... from_clinical): k = key(p[1]["messages"][1]["content"])`	the full-data test found 927 notes repeated up to 5 times; this keeps only the first copy of each note, so a test note can never also be a training note
`splits = {"test": pairs[:200], "val": pairs[200:400], "dpo": pairs[400:1400], "train": pairs[1400:]}`	ONE shared split for notes and summary
`folder = PREF if split == "dpo" else SFT`	DPO prompts to data/pref, the rest to data/sft

CHECKPOINT: ls scripts shows 02_build_sft_jsonl.py

SECTION 4.5 - Run the conversion

WHERE: VS Code Terminal, folder ~/medscriptai/scripts, (.venv-train) active

GOAL: 9 files in data/sft and 3 in data/pref.

Step 1. Run:

```
python 02_build_sft_jsonl.py
```

You should see (real output, about 1 minute):

```
wrote    300 rows -> data/sft/qa_val.jsonl
wrote  23669 rows -> data/sft/qa_train.jsonl
wrote    500 rows -> data/sft/qa_test.jsonl
wrote   1891 rows -> data/pref/qa_dpo.jsonl
wrote    200 rows -> data/sft/notes_test.jsonl
wrote    200 rows -> data/sft/summary_test.jsonl
wrote    200 rows -> data/sft/notes_val.jsonl
wrote    200 rows -> data/sft/summary_val.jsonl
wrote   1000 rows -> data/pref/notes_dpo.jsonl
wrote   1000 rows -> data/pref/summary_dpo.jsonl
wrote   6367 rows -> data/sft/notes_train.jsonl
wrote   6367 rows -> data/sft/summary_train.jsonl
```

Why 1891 and not 3000 DPO prompts: many MedMCQA questions are repeats or multi-answer; the filters removed them. That is correct behaviour.

Step 2. Look at one QA training example in readable form:

```
head -n 1 ../data/sft/qa_train.jsonl | python -m json.tool
```

You should see: the JSON shown in "BEFORE YOU START" (task, source, answer_letter, question_key, messages).

Step 3. Look at one notes example (the user message is a conversation, the assistant message is the note):

```
head -n 1 ../data/sft/notes_train.jsonl | python -c "import sys,json; m=json.load(sys.stdin)['messages']; print(m[1]
['content'][:400]); print('-----'); print(m[2]['content'][:400])"
```

You should see: lines starting "Doctor: Good morning..." and "Patient: ...", then "-----", then a clinical note such as "A 24-year-old unmarried woman was admitted to the Department of Neurology ...".

Step 4. Look at one summary target (JSON):

```
head -n 1 ../data/sft/summary_train.jsonl | python -c "import sys,json; print(json.load(sys.stdin)['messages'][2]
['content'][:400])"
```

You should see: {"visit motivation": "...", "admission": [...], "patient information": {"age": "24", "sex": "Female"}, ...

CHECKPOINT: the 12 "wrote" lines appeared and the three examples look sensible.

If not: FileNotFoundError data/raw/... = Section 4.3 did not finish; re-run it.

SECTION 4.6 - Create and run scripts/03_validate_jsonl.py

WHERE: VS Code Explorer (create), then VS Code Terminal (run)

GOAL: proof that every line is valid chat data and fits in 2048 tokens.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 03_validate_jsonl.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Step 3: prove every .jsonl file is valid chat data before you pay for GPU time."""
import json                                          # parse each line
import statistics                                    # median / max token counts
import sys                                           # exit code for pass/fail
from pathlib import Path
from common import SFT, BASE_MODEL

ROLES = ["system", "user", "assistant"]
errors = 0

try:                                                 # token counts need the Llama tokenizer
    from transformers import AutoTokenizer
    tok = AutoTokenizer.from_pretrained(BASE_MODEL)
except Exception as exc:                             # still validate structure without it
    tok = None
```

```
    print(f"(tokenizer not loaded, skipping token counts: {exc.__class__.__name__})")

for path in sorted(Path(SFT).glob("*.jsonl")):
    lengths = []
    for n, line in enumerate(path.open(encoding="utf-8"), start=1):
        try:
            row = json.loads(line)                 # 1) each line must be valid JSON
            roles = [m["role"] for m in row["messages"]]
            assert roles == ROLES, f"roles {roles}"  # 2) exactly system -> user -> assistant
            assert all(m["content"].strip() for m in row["messages"]), "empty message"
            if row["task"] == "summary":           # 3) summary targets must be valid JSON
                json.loads(row["messages"][-1]["content"])
            if tok:                                # 4) length in tokens after the chat template
                ids = tok.apply_chat_template(row["messages"], tokenize=True)
                ids = ids["input_ids"] if isinstance(ids, dict) or hasattr(ids, "keys") else ids
                lengths.append(len(ids))
        except Exception as exc:
            errors += 1
            print(f"ERROR {path.name}:{n}: {exc}")
    stats = f" tokens median={statistics.median(lengths):.0f} max={max(lengths)}" if lengths else ""
    print(f"{path.name:22} rows={n:>6}{stats}")

print("ALL_OK" if errors == 0 else f"FAILED with {errors} errors")
sys.exit(1 if errors else 0)
```

Line	What it does (plain English)
`ROLES = ["system", "user", "assistant"]`	the only allowed message order
`AutoTokenizer.from_pretrained(BASE_MODEL)` inside try	loads Llama-3.1's real tokenizer to count tokens; if that fails, structure is still checked
`for path in sorted(Path(SFT).glob("*.jsonl"))`	every .jsonl file in data/sft
`row = json.loads(line)`	check 1: valid JSON
`assert roles == ROLES`	check 2: exactly system → user → assistant
`assert all(m["content"].strip() ...)`	check 3: no empty message
`if row["task"] == "summary": json.loads(...)`	check 4: the summary target is valid JSON
`tok.apply_chat_template(row["messages"], tokenize=True)`	check 5: the real token count after Llama's chat template
`ids = ids["input_ids"] if ... else ids`	newer transformers return a dict, older a list; this handles both
`print("ALL_OK" ...); sys.exit(1 if errors else 0)`	exit code 0 = success, 1 = failure

Step 6. Run it (the first run downloads the tokenizer, a few MB):

```
python 03_validate_jsonl.py
```

You should see (real output):

```
notes_test.jsonl       rows=   200 tokens median=1146 max=1417
notes_train.jsonl      rows=  6367 tokens median=1138 max=1605
notes_val.jsonl        rows=   200 tokens median=1144 max=1540
qa_test.jsonl          rows=   500 tokens median=270 max=810
qa_train.jsonl         rows= 23669 tokens median=253 max=984
qa_val.jsonl           rows=   300 tokens median=245 max=582
summary_test.jsonl     rows=   200 tokens median=1095 max=1493
summary_train.jsonl    rows=  6367 tokens median=1083 max=1659
summary_val.jsonl      rows=   200 tokens median=1090 max=1663
ALL_OK
```

CHECKPOINT: the last line is ALL_OK and every max is below 2048.

If not: an ERROR line names the file and line number; "apply_chat_template requires jinja2" = run uv pip install --python ../.venv-train jinja2.

SECTION 4.7 - Create scripts/10_train_sft.py (the training script)

BEFORE YOU START: what happens when you run it. (1) The 4-bit Llama model (5.7 GB) is downloaded and loaded on the GPU. (2) LoRA adapters (about 42 million trainable numbers) are attached. (3) Each training example is turned into one long text with the chat template. (4) The trainer runs 125 steps (2,000 examples / 16 per step); every 10 steps it prints the loss,

every 100 steps the validation loss. (5) Only the adapter (about 80-170 MB) is saved to outputs/adapters/sft-qa. GPU memory used: roughly 10-16 GB of the 80 GB.

WHERE: VS Code Explorer, folder scripts

GOAL: the QLoRA SFT training script.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 10_train_sft.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Step 4: QLoRA supervised fine-tuning (SFT) of Llama-3.1-8B-Instruct with Unsloth + TRL + PEFT.

One run = one task = one LoRA adapter, e.g.  python scripts/10_train_sft.py --task qa
"""
from unsloth import FastLanguageModel                    # import Unsloth FIRST so it can patch TRL/transformers
from unsloth.chat_templates import train_on_responses_only

import argparse                                          # command-line options
import json                                              # save the training log
from datasets import load_dataset                        # read our .jsonl files
from trl import SFTConfig, SFTTrainer                    # Hugging Face TRL trainer
from common import SFT, OUT, BASE_MODEL_4BIT

p = argparse.ArgumentParser()
p.add_argument("--task", required=True, choices=["qa", "notes", "summary"])
p.add_argument("--max-train", type=int, default=None, help="use only the first N training rows")
p.add_argument("--epochs", type=float, default=1.0)
p.add_argument("--max-steps", type=int, default=-1, help="stop after N steps (-1 = use epochs)")
p.add_argument("--lr", type=float, default=2e-4)        # LoRA learns fast; 2e-4 is the usual QLoRA rate
p.add_argument("--rank", type=int, default=16)          # LoRA rank r: size of the adapter matrices
p.add_argument("--batch", type=int, default=8)          # examples per GPU step
p.add_argument("--grad-accum", type=int, default=2)     # effective batch = batch x grad-accum = 16
p.add_argument("--train-file", default=None, help="override the training .jsonl (Part 5 uses this)")
p.add_argument("--run-name", default=None)
a = p.parse_args()
run = a.run_name or f"sft-{a.task}"
out_dir = OUT / "adapters" / run

# 1) Load the base model already quantised to 4-bit (this is the "Q" in QLoRA).
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=BASE_MODEL_4BIT,     # 4-bit NF4 weights: ~5.7 GB on disk instead of ~16 GB
    max_seq_length=2048,            # longest example we keep is ~1,600 tokens
    load_in_4bit=True,
    dtype=None,                     # None = auto: bfloat16 on A100/L40/H100
)

# 2) Attach small trainable LoRA matrices to every attention and MLP projection.
model = FastLanguageModel.get_peft_model(
    model,
    r=a.rank,
    lora_alpha=a.rank,              # scaling = alpha / r = 1.0
    lora_dropout=0.0,               # 0 is fastest in Unsloth and works well
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
    use_gradient_checkpointing="unsloth",   # recompute activations to save VRAM
    random_state=3407,
)
model.print_trainable_parameters()  # expect ~42M trainable of ~8B (about 0.5%)


# 3) Turn each {"messages": [...]} row into one string using Llama-3.1's own chat template.
#    The template starts with <|begin_of_text|>; the trainer adds it again when tokenising,
#    so we remove it here to end up with exactly ONE begin token (double BOS hurts quality).
def to_text(batch):
    texts = [tokenizer.apply_chat_template(m, tokenize=False).removeprefix(tokenizer.bos_token)
             for m in batch["messages"]]
    return {"text": texts}


train_file = a.train_file or str(SFT / f"{a.task}_train.jsonl")
train = load_dataset("json", data_files=train_file, split="train")
if a.max_train:
    train = train.select(range(min(a.max_train, len(train))))
```

```
val = load_dataset("json", data_files=str(SFT / f"{a.task}_val.jsonl"), split="train").select(range(100))
train = train.map(to_text, batched=True, remove_columns=train.column_names)
val = val.map(to_text, batched=True, remove_columns=val.column_names)
print(train[0]["text"][:600])       # eyeball one formatted example before spending GPU time

# 4) Training settings.
cfg = SFTConfig(
    output_dir=str(OUT / "checkpoints" / run),
    dataset_text_field="text",
    max_length=2048,
    per_device_train_batch_size=a.batch,
    gradient_accumulation_steps=a.grad_accum,
    num_train_epochs=a.epochs,
    max_steps=a.max_steps,
    learning_rate=a.lr,
    lr_scheduler_type="cosine",     # smooth decay to ~0 by the end
    warmup_steps=10,                # ramp the LR up for the first 10 steps
    optim="adamw_8bit",             # 8-bit optimiser states: big VRAM saving
    weight_decay=0.01,
    bf16=True,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=100,
    save_steps=200,
    save_total_limit=2,             # keep only the 2 newest checkpoints on disk
    report_to="none",
    seed=3407,
)
trainer = SFTTrainer(model=model, processing_class=tokenizer, train_dataset=train,
                     eval_dataset=val, args=cfg)

# 5) Only learn from the assistant's answer, not from the system/user text.
trainer = train_on_responses_only(
    trainer,
    instruction_part="<|start_header_id|>user<|end_header_id|>\n\n",
    response_part="<|start_header_id|>assistant<|end_header_id|>\n\n",
)

stats = trainer.train()

# 6) Save ONLY the adapter (~80-170 MB at r=16), not the 16 GB base model.
model.save_pretrained(str(out_dir))
tokenizer.save_pretrained(str(out_dir))
(out_dir / "train_log.json").write_text(json.dumps(trainer.state.log_history, indent=1))
print(f"DONE adapter={out_dir} train_loss={stats.training_loss:.4f}")
```

Line	What it does (plain English)
`from unsloth import FastLanguageModel` (first import)	Unsloth must be imported BEFORE trl/transformers so it can patch them with its fast code
`from unsloth.chat_templates import train_on_responses_only`	masks system/user tokens so the loss is only on the answer
`from trl import SFTConfig, SFTTrainer`	Hugging Face TRL's supervised fine-tuning trainer and its settings
`--task` with `choices=[...]`	which adapter to train; choices prevents typos
`--max-train`	use only the first N training rows (2,000 in this part)
`--epochs 1.0`, `--max-steps -1`	one pass over the data; -1 = stop by epochs
`--lr 2e-4`	learning rate: the standard starting point for QLoRA
`--rank 16`	LoRA rank r
`--batch 8`, `--grad-accum 2`	8 examples per GPU step, gradients added over 2 steps = 16 per update
`--train-file`	train on another file (Part 5 uses the synthetic one)
`FastLanguageModel.from_pretrained(model_name=BASE_MODEL_4B IT, max_seq_length=2048, load_in_4bit=True, dtype=None)`	load Llama-3.1-8B-Instruct in 4-bit with its tokenizer; dtype None = automatic bfloat16
`FastLanguageModel.get_peft_model(model, r=..., lora_alpha=..., lora_dropout=0.0, target_modules=[...], use_gradient_checkpointing="unsloth", random_state=3407)`	attach LoRA to all 7 linear layer types; gradient checkpointing recomputes activations to save memory
`model.print_trainable_parameters()`	prints about 42M trainable of 8B
`def to_text(batch)`	turns each messages list into one text with Llama's chat template

`.removeprefix(tokenizer.bos_token)`	removes the leading <|begin_of_text|> because the trainer adds it again; without this every example would start with TWO begin tokens (this bug was found while testing this manual)
`load_dataset("json", data_files=train_file, split="train")`	read the .jsonl into a table
`train.select(range(...))`	keep the first N rows
`val = ... .select(range(100))`	100 validation rows are enough to watch the validation loss
`train.map(to_text, batched=True, remove_columns=...)`	add the "text" column, drop the columns the trainer does not need
`print(train[0]["text"][:600])`	shows one formatted example before training starts
`SFTConfig(output_dir=..., dataset_text_field="text", max_length=2048, ...)`	where checkpoints go, which column has the text, maximum length
`lr_scheduler_type="cosine"`, `warmup_steps=10`	LR rises for 10 steps, then decreases smoothly to almost 0
`optim="adamw_8bit"`, `weight_decay=0.01`, `bf16=True`	memory-saving 8-bit optimizer, mild regularisation, 16-bit maths
`logging_steps=10`, `eval_strategy="steps"`, `eval_steps=100`	print loss every 10 steps, validation loss every 100
`save_steps=200`, `save_total_limit=2`	checkpoint every 200 steps, keep only 2
`report_to="none"`, `seed=3407`	no external tracker; fixed data order
`SFTTrainer(model=model, processing_class=tokenizer, ...)`	processing_class = the tokenizer (new name in TRL 1.x)
`train_on_responses_only(trainer, instruction_part=..., response_part=...)`	finds the user and assistant headers and sets labels to -100 everywhere except the answer
`stats = trainer.train()`	the training loop
`model.save_pretrained(str(out_dir))`, `tokenizer.save_pretrained(...)`	save ONLY the adapter + tokenizer
`train_log.json`	every logged loss and LR value, for your report

CHECKPOINT: ls scripts shows 10_train_sft.py

SECTION 4.8 - Run your first training

WHERE: VS Code Terminal, new tmux session

GOAL: outputs/adapters/sft-qa with a trained adapter.

Step 1. Start a tmux session:

```
tmux new -s train
```

Step 2. Activate and go to scripts:

```
cd ~/medscriptai/scripts && source ../.venv-train/bin/activate
```

Step 3. Start training on 2,000 QA examples:

```
python 10_train_sft.py --task qa --max-train 2000
```

You should see, in this order:

```
==((====))==  Unsloth 2026.10.7: Fast Llama patching. ... GPU: NVIDIA A100 80GB PCIe ...
trainable params: 41,943,040 || all params: 8,072,204,288 || trainable%: 0.5196   (numbers may differ slightly)
<|start_header_id|>system<|end_header_id|> ... (one formatted example)
  1%|          | 1/125 [...]
{'loss': 1.9..., 'grad_norm': ..., 'learning_rate': ..., 'epoch': 0.08}
...
{'eval_loss': 0.9..., ...}
DONE adapter=/home/ubuntu/medscriptai/outputs/adapters/sft-qa train_loss=0.8...
```

(The loss numbers are illustrative: a healthy run starts around 1.5-2.5 and ends around 0.6-1.0.)

Step 4. Detach while it runs: Ctrl+B, then D.

Step 5. Open a second VS Code terminal (click + in the terminal panel) and watch the GPU:

```
watch -n 5 nvidia-smi
```

You should see: a python process using roughly 10-16 GB and GPU-Util 90-100%. Press Ctrl+C to stop watching.

Step 6. Re-attach to read the end: tmux attach -t train

How to read the log:

What you see	Meaning	Action
loss falls steadily	learning	none
loss is "nan" or jumps above 5	learning rate too high or broken data	stop (Ctrl+C), rerun with --lr 1e-4
eval_loss rises twice in a row while loss falls	overfitting	use fewer epochs or more data
CUDA out of memory	batch too large for the GPU	add --batch 4 --grad-accum 4

CHECKPOINT: ls ../outputs/adapters/sft-qa shows adapter_config.json, adapter_model.safetensors, tokenizer files and train_log.json.

If not: re-attach (tmux attach -t train) and read the last error line; see Part 10.

SECTION 4.9 - Create and run scripts/11_chat_test.py (base vs fine-tuned)

WHERE: VS Code Explorer (create), then VS Code Terminal (run) with (.venv-train)

GOAL: see with your own eyes what fine-tuning changed.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 11_chat_test.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Step 5: ask the same test questions to the base model and to base + your adapter, side by side."""
from unsloth import FastLanguageModel                    # Unsloth first, as always

import argparse
import torch
from common import SFT, OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--task", default="qa", choices=["qa", "notes", "summary"])
p.add_argument("--adapter", default=None, help="adapter folder (default outputs/adapters/sft-<task>)")
p.add_argument("--n", type=int, default=3, help="how many test examples to show")
a = p.parse_args()
adapter = a.adapter or str(OUT / "adapters" / f"sft-{a.task}")

# Loading the ADAPTER folder makes Unsloth read adapter_config.json, load the 4-bit base, then attach the LoRA.
model, tokenizer = FastLanguageModel.from_pretrained(adapter, max_seq_length=2048, load_in_4bit=True)
FastLanguageModel.for_inference(model)                   # switch on Unsloth's fast generation path


def answer(messages):
    """Greedy-decode one reply for a list of chat messages (system + user)."""
    ids = tokenizer.apply_chat_template(messages, add_generation_prompt=True, return_tensors="pt",
                                        return_dict=True).to("cuda")
    out = model.generate(**ids, max_new_tokens=400, do_sample=False)   # do_sample=False = deterministic
    return tokenizer.decode(out[0][ids["input_ids"].shape[1]:], skip_special_tokens=True).strip()


for row in read_jsonl(SFT / f"{a.task}_test.jsonl")[: a.n]:
    prompt = row["messages"][:2]                         # system + user only; hide the reference answer
    with model.disable_adapter():                        # temporarily turn LoRA OFF = plain base model
        base = answer(prompt)
    tuned = answer(prompt)                               # LoRA back ON
    print("=" * 80, "\nUSER:", prompt[1]["content"][:500])
    print("\n--- BASE ---\n", base[:700])
    print("\n--- FINE-TUNED ---\n", tuned[:700])
    print("\n--- REFERENCE ---\n", row["messages"][2]["content"][:700])
    torch.cuda.empty_cache()
```

Line	What it does (plain English)
`FastLanguageModel.from_pretrained(adapter, ...)`	pointing at the ADAPTER folder: Unsloth reads adapter_config.json, loads the 4-bit base and attaches your LoRA
`FastLanguageModel.for_inference(model)`	switch Unsloth to its fast generation mode

`apply_chat_template(messages, add_generation_prompt=True, return_tensors="pt", return_dict=True)`	format system + user and end with the assistant header so the model starts answering
`model.generate(**ids, max_new_tokens=400, do_sample=False)`	greedy decoding: always the most likely token, repeatable
`out[0][ids["input_ids"].shape[1]:]`	keep only the new tokens (cut the prompt)
`prompt = row["messages"][:2]`	system + user; the reference answer is hidden
`with model.disable_adapter():`	temporarily switch LoRA OFF = the plain base model
`torch.cuda.empty_cache()`	free cached GPU memory between examples

Step 6. Run:

```
python 11_chat_test.py --task qa --n 3
```

You should see: three blocks with USER, BASE, FINE-TUNED and REFERENCE. Usually BASE writes a long free answer that often does not start with "Answer:", and FINE-TUNED starts exactly with "Answer: <letter>) <option>".

Watch out: three examples prove the adapter works, not that it is better. Real measurement is Part 5.

CHECKPOINT: the FINE-TUNED answers follow the "Answer: X) ..." format.

SECTION 4.10 - Create and run scripts/12_push_to_hub.py (publish)

WHERE: VS Code Explorer (create), then VS Code Terminal (run) with (.venv-train)

GOAL: your adapter in a private repo on huggingface.co.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 12_push_to_hub.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Step 6: publish an adapter or a dataset folder to the Hugging Face Hub (private by default)."""
import argparse
from huggingface_hub import HfApi                 # official Hub client; reads HF_TOKEN from the environment

p = argparse.ArgumentParser()
p.add_argument("--folder", required=True, help="local folder, e.g. outputs/adapters/sft-qa or data/sft")
p.add_argument("--repo", required=True, help="YOUR_HF_USERNAME/Llama-3.1-8B-MedScriptAI-qa-lora")
p.add_argument("--type", default="model", choices=["model", "dataset"])
p.add_argument("--public", action="store_true", help="make the repo public (default private)")
a = p.parse_args()

api = HfApi()
api.create_repo(a.repo, repo_type=a.type, private=not a.public, exist_ok=True)   # no error if it exists
api.upload_folder(folder_path=a.folder, repo_id=a.repo, repo_type=a.type,
                  ignore_patterns=["checkpoint-*", "*.pt", "optimizer*"])          # skip training leftovers
print(f"https://huggingface.co/{'datasets/' if a.type == 'dataset' else ''}{a.repo}")
```

Line	What it does (plain English)
`from huggingface_hub import HfApi`	the official Hub client; it reads HF_TOKEN from your environment
`--folder`, `--repo`, `--type`, `--public`	what to upload, where, model or dataset, visibility (private by default)
`api.create_repo(..., exist_ok=True)`	create the repo; no error if it exists
`api.upload_folder(..., ignore_patterns=[...])`	upload, skipping training leftovers

Step 6. Run it (replace YOUR_HF_USERNAME):

```
python 12_push_to_hub.py --folder ../outputs/adapters/sft-qa --repo YOUR_HF_USERNAME/Llama-3.1-8B-MedScriptAI-qa-lora
```

You should see: https://huggingface.co/YOUR_HF_USERNAME/Llama-3.1-8B-MedScriptAI-qa-lora

Step 7. Open that URL in your browser.

You should see: adapter_config.json and adapter_model.safetensors, and the label Private.

Placeholder	Replace with	Example

YOUR_HF_USERNAME	your Hugging Face user name	rahulml

Watch out: the name must start with "Llama" (Llama 3.1 licence) [15].

CHECKPOINT: the Hub page shows your adapter files.

If not: 401 Unauthorized = token missing or read-only (make a Write token, Section 2A.3, and repeat Section 2B.4 Step 3-4).

SECTION 4.11 - First cloud step: copy the adapter to object storage (S3)

BEFORE YOU START: S3 is AWS's file storage ("buckets" hold "objects" = files). Production systems keep model files in S3 so any server can download them. You practise on FakeCloud first; the blue column is the same on real AWS.

WHERE: VS Code Terminal, folder ~/medscriptai (type cd ~/medscriptai first)

GOAL: the adapter in a bucket.

Step	FakeCloud (free practice)	Real AWS (same step)
1 Point the CLI	`export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1`	`unset AWS_ENDPOINT_URL` (your aws configure keys are used)
2 Unique bucket name	`export BUCKET=medscriptai-artifacts-$(aws sts get-caller-identity --query Account --output text)-us-east-1`	same command (your account number makes it unique worldwide)
3 Create the bucket	`aws s3 mb s3://$BUCKET`	`aws s3 mb s3://$BUCKET --region us-east-1`
4 Upload the adapter	`aws s3 sync outputs/adapters/sft-qa s3://$BUCKET/adapters/sft-qa`	same command
5 List	`aws s3 ls s3://$BUCKET --recursive --human-readable`	same command
You should see	adapters/sft-qa/adapter_model.safetensors with its size	the same; also in AWS console > S3 > your bucket
Cost	US$0	about US$0.023 per GB-month (verify): under 1 cent per month

What each piece means:

Piece	What it does
`aws s3 mb`	make bucket
`aws s3 sync SRC DST`	copy only new or changed files (safe to repeat)
`--query Account --output text`	print just one field as plain text
`$(...)`	run the inner command and paste its output

CHECKPOINT: aws s3 ls shows the adapter in your column's bucket.

If not: NoSuchBucket = echo $BUCKET is empty in this terminal; repeat Step 2.

SECTION 4.12 - Save your progress in Git

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: a commit with the Part 4 scripts.

Step 1. Run:

```
cd ~/medscriptai && git add . && git commit -m "Part 4: data pipeline and first SFT adapter"
```

You should see: "6 files changed" (or similar). data/ and outputs/ are not committed (they are in .gitignore).

CHECKPOINT: git log --oneline shows two commits.

PART 4 RESULT CHECK

Check	Command	Expected
raw data	ls data/raw | wc -l	7
chat data valid	cd scripts && python 03_validate_jsonl.py | tail -1	ALL_OK

adapter exists	ls outputs/adapters/sft-qa	adapter_config.json adapter_model.safetensors ...
adapter changes behaviour	python 11_chat_test.py --task qa --n 3	FINE-TUNED starts with "Answer:"
published	open your Hub URL	adapter files, Private
in (Fake)S3	aws s3 ls s3://$BUCKET --recursive	adapters/sft-qa/...

4.13 Common mistake at this level

The mistake: training on text formatted differently from what the server sends later (two <|begin_of_text|> tokens, a different system prompt, or a missing chat template). The model then underperforms and you blame the hyper-parameters.

How to spot it: print one formatted training example (10_train_sft.py does this), print what your API sends (Part 6A), and compare them. This project avoids it by importing SYSTEM from scripts/common.py everywhere and by the removeprefix fix.

PART 5 - INTERMEDIATE: synthetic data, three adapters, real evaluation

BEFORE YOU START: what you will build in this part

In Part 4 you trained one adapter on 2,000 examples. Now you do what a real team does next: improve the DATA, scale the training, and MEASURE.

```
Section 5.1-5.2   start the TEACHER model server (Qwen2.5-14B-Instruct) on port 8001
Section 5.3-5.6   synthetic data: the teacher writes explanations for 10,000 MedQA answers (distilabel)
Section 5.7       train three adapters: sft-qa (with the explanations), sft-notes, sft-summary
Section 5.8       serve base + 3 adapters with vLLM on port 8000
Section 5.9-5.11  evaluate: accuracy, ROUGE-L, BERTScore, LLM judge
Section 5.12-5.14 publish datasets, store reports and secrets in (Fake)AWS, commit
```

New words in this part:

Word	Plain meaning
model server	a program that keeps a model loaded on the GPU and answers requests over HTTP, like a website for a model
vLLM	the fastest popular open-source model server; it batches many requests together on the GPU
OpenAI-compatible API	vLLM accepts the same request format as OpenAI's API, so any OpenAI client library works with it
port	a numbered "door" on a computer; 8000 = your model, 8001 = the teacher, 8080 = your API (Part 6A)
teacher model	a bigger model whose answers are used to teach the smaller one
synthetic data	training examples written by a model instead of humans (see Part 3, section 3.7)
distilabel	a library that runs synthetic-data pipelines: load rows → send to a model → collect → save
Docker container	a program running in an isolated box with its own files; vLLM runs in one

Harder than Part 4 because: two model servers, a pipeline framework, three training runs, and metrics you must interpret correctly. Budget: about 6 GPU-hours (about US$8).

SECTION 5.1 - Create serve/start_teacher.sh

WHERE: VS Code Explorer, folder serve

GOAL: a script that starts the teacher model server.

Step 1. Right-click the folder serve.

Step 2. Click New File...

Step 3. Type start_teacher.sh and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
#!/usr/bin/env bash
# Serve the TEACHER / JUDGE model (Qwen2.5-14B-Instruct) on port 8001 under the name "teacher".
# Used by distilabel (Part 5/6) and by the LLM judge. Stop it when you are done: docker rm -f medscriptai-teacher
set -euo pipefail

VLLM_IMAGE="${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0-cu129}"
GPU_UTIL="${GPU_UTIL:-0.85}"                        # 0.85 alone on the GPU; 0.50 next to start_vllm.sh (0.35)

docker rm -f medscriptai-teacher 2>/dev/null || true
docker run -d --name medscriptai-teacher \
  --gpus all --ipc=host \
  -p 8001:8000 \
  -e VLLM_ENABLE_CUDA_COMPATIBILITY="${VLLM_ENABLE_CUDA_COMPATIBILITY:-1}" \
  -v "${HF_HOME:-$HOME/.cache/huggingface}":/root/.cache/huggingface \
  "$VLLM_IMAGE" \
  --model Qwen/Qwen2.5-14B-Instruct \
  --served-model-name teacher \
  --dtype bfloat16 \
  --max-model-len 8192 \
  --gpu-memory-utilization "$GPU_UTIL"

echo "Started. Follow the logs with:  docker logs -f medscriptai-teacher"
```

Line	What it does (plain English)
`#!/usr/bin/env bash`	"run this file with bash"
`set -euo pipefail`	stop at the first error, treat unset variables as errors, fail a pipeline if any part fails
`VLLM_IMAGE="${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0-cu129}"`	the official vLLM image (version 0.32.0, CUDA 12.9 build). `${X:-default}` = use X if it is set, otherwise the default
`GPU_UTIL="${GPU_UTIL:-0.85}"`	fraction of GPU memory this server may use
`docker rm -f medscriptai-teacher 2>/dev/null || true`	remove an old container with the same name; ignore the error if there is none
`docker run -d --name medscriptai-teacher`	start a container in the background with a fixed name
`--gpus all --ipc=host`	give it the GPU; share memory with the host (vLLM needs this)
`-p 8001:8000`	the server listens on 8000 inside the container; you reach it on port 8001
`-e VLLM_ENABLE_CUDA_COMPATIBILITY=1`	your driver (570) supports CUDA 12.8 but the image uses 12.9; NVIDIA's compatibility libraries inside the image bridge this on data-centre GPUs (A100, L40, H100). We read this in vLLM's own Dockerfile
`-v ${HF_HOME}:/root/.cache/huggingface`	share the download cache, so the model downloads only once
`--model Qwen/Qwen2.5-14B-Instruct`	the teacher: 14.7B parameters, Apache-2.0 licence
`--served-model-name teacher`	clients ask for "teacher" instead of the long name
`--dtype bfloat16`	16-bit weights, about 29.5 GB
`--max-model-len 8192`	longest prompt + answer in tokens
`--gpu-memory-utilization "$GPU_UTIL"`	vLLM reserves this share of the GPU and fills the spare part with the KV cache (its memory for running requests)

Step 6. Make the script executable:

```
chmod +x ~/medscriptai/serve/start_teacher.sh
```

CHECKPOINT: ls -l ~/medscriptai/serve shows start_teacher.sh with -rwx at the start.

SECTION 5.2 - Start the teacher and test it

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: the teacher answers on port 8001.

Step 1. Start it:

```
cd ~/medscriptai && bash serve/start_teacher.sh
```

You should see: a long container ID and "Started. Follow the logs with: docker logs -f medscriptai-teacher". The first start downloads the vLLM image (about 11 GB) and the model (about 30 GB): 10-15 minutes.

Step 2. Follow the start-up log:

```
docker logs -f medscriptai-teacher
```

You should see, at the end: "Application startup complete." Press Ctrl+C to stop FOLLOWING (the server keeps running).

Step 3. List the models it serves:

```
curl -s http://localhost:8001/v1/models | jq -r '.data[].id'
```

You should see: teacher

Step 4. Ask a test question:

```
curl -s http://localhost:8001/v1/chat/completions -H "Content-Type: application/json" -d '{"model":"teacher","messages":[{"role":"user","content":"In one sentence: why is nitrofurantoin used for cystitis in pregnancy?"}],"max_tokens":80}' | jq -r '.choices[0].message.content'
```

You should see: one sensible medical sentence.

Piece	What it does
curl -s URL	send an HTTP request quietly
-H "Content-Type: application/json"	"my body is JSON"
-d '{...}'	the JSON body: which model, the messages, max answer length
jq -r '.choices[0].message.content'	pull the answer text out of the JSON reply

If you see...	It means...	Do this next
"CUDA driver version is insufficient" in the log	the compatibility layer was not used	check the VLLM_ENABLE_CUDA_COMPATIBILITY line, or use the R595 OS image with VLLM_IMAGE=vllm/vllm-openai:v0.32.0
"No available memory for the cache blocks"	GPU memory is used by something else	nvidia-smi; stop other containers or training
curl: (7) Failed to connect	still loading	wait for "Application startup complete"

CHECKPOINT: Step 3 prints teacher and Step 4 prints a sentence.

SECTION 5.3 - Create scripts/20_synth_explanations.py (synthetic data)

BEFORE YOU START: how the distilabel pipeline works.

```
qa_train.jsonl (MedQA rows with only "Answer: D) Nitrofurantoin")
      |
      v
LoadDataFromDicts  -> sends rows in batches of 64
      |
      v
TextGeneration     -> for each row asks the teacher: "<question + options> The correct answer is: D) Nitrofurantoin.
      |               Explain step by step why it is correct and briefly why the other options are wrong."
      v
quality filter     -> keep only explanations of 80-1,500 characters that mention the correct option
      |
      v
qa_train_synth.jsonl (same rows, MedQA answers now end with "Explanation: ...")
```

The teacher is GIVEN the correct answer (rationale generation), so it explains rather than guesses. That is much more reliable than letting it answer by itself.

WHERE: VS Code Explorer, folder scripts

GOAL: the synthetic-data script.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 20_synth_explanations.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Part 5: use distilabel + a teacher LLM to write explanations for MedQA answers (synthetic SFT data).

MedQA only gives the correct letter. A model trained on letters alone learns to guess, not to reason.
The teacher (Qwen2.5-14B-Instruct served by vLLM as "teacher") explains WHY the gold answer is right.
"""
import argparse
import os
import re
from distilabel.models import OpenAILLM                 # talks to any OpenAI-compatible server (vLLM)
from distilabel.pipeline import Pipeline
from distilabel.steps import LoadDataFromDicts
from distilabel.steps.tasks import TextGeneration
from common import DATA, SFT, read_jsonl, write_jsonl

p = argparse.ArgumentParser()
p.add_argument("--limit", type=int, default=None, help="only explain the first N MedQA rows (test run)")
a = p.parse_args()

TEACHER_URL = os.getenv("TEACHER_URL", "http://localhost:8001/v1")   # vLLM teacher endpoint
TEACHER = os.getenv("TEACHER_MODEL", "teacher")                       # --served-model-name in vLLM
```

```
TEACHER_SYSTEM = ("You are a senior physician and medical educator. Explain clearly and accurately "
                  "in at most 120 words. Do not change the given correct answer.")

rows = read_jsonl(SFT / "qa_train.jsonl")
todo = [r for r in rows if r["source"] == "medqa" and "Explanation:" not in r["messages"][2]["content"]]
todo = todo[: a.limit] if a.limit else todo
inputs = [{
    "question_key": r["question_key"],
    "gold": r["messages"][2]["content"],                          # e.g. "Answer: D) Nitrofurantoin"
    "instruction": (f"{r['messages'][1]['content']}\n\nThe correct answer is: "
                    f"{r['messages'][2]['content'].removeprefix('Answer: ')}\n"
                    "Explain step by step why it is correct and briefly why the other options are wrong."),
} for r in todo]
print(f"{len(inputs)} MedQA rows need an explanation")

with Pipeline(name="medscriptai-explanations") as pipeline:
    load = LoadDataFromDicts(data=inputs, batch_size=64)          # step 1: feed our rows in
    explain = TextGeneration(                                       # step 2: ask the teacher
        llm=OpenAILLM(model=TEACHER, base_url=TEACHER_URL, api_key="EMPTY",
                      generation_kwargs={"temperature": 0.3, "max_new_tokens": 300}),
        system_prompt=TEACHER_SYSTEM,
        input_batch_size=32,                                          # 32 requests in flight at a time
    )
    load >> explain                                                   # ">>" connects the steps

distiset = pipeline.run(use_cache=False)                            # returns a Hugging Face DatasetDict
generated = distiset["default"]["train"]


def good(text, gold):
    """Quality filter: non-empty, sane length, and it mentions the gold option."""
    if not text or not (80 <= len(text) <= 1500):
        return False
    option = gold.split(") ", 1)[-1].lower()
    letter = gold.removeprefix("Answer: ")[0]
    return option[:25] in text.lower() or f"{letter})" in text


explanations = {g["question_key"]: g["generation"].strip()
                for g in generated if good(g["generation"], g["gold"])}
write_jsonl(DATA / "synth" / "medqa_explanations.jsonl",
            [{"question_key": k, "explanation": v} for k, v in explanations.items()])

for r in rows:                                                      # merge back into the SFT data
    if r["question_key"] in explanations:
        r["messages"][2]["content"] += f"\nExplanation: {explanations[r['question_key']]}"
write_jsonl(SFT / "qa_train_synth.jsonl", rows)
print(f"kept {len(explanations)}/{len(inputs)} explanations after the quality filter")
```

Line	What it does (plain English)
`from distilabel.models import OpenAILLM`	distilabel's client for any OpenAI-compatible server (our vLLM teacher)
`from distilabel.pipeline import Pipeline`	a pipeline = connected steps that pass batches of rows along
`from distilabel.steps import LoadDataFromDicts`	first step: feeds a Python list of dicts into the pipeline
`from distilabel.steps.tasks import TextGeneration`	sends each row's "instruction" to the model and writes the reply into "generation"
`TEACHER_URL`, `TEACHER`	where the teacher listens (port 8001) and its served name
`TEACHER_SYSTEM = (...)`	instructions for the teacher: expert tone, at most 120 words, do not change the answer
`todo = [r for r in rows if r["source"] == "medqa" and "Explanation:" not in ...]`	only MedQA rows without an explanation (MedMCQA and PubMedQA already have one)
`"gold": r["messages"][2]["content"]`	the correct answer line, used by the quality filter
`"instruction": (... "The correct answer is: " ...)`	what the teacher receives: question + options + the correct answer + the request to explain
`with Pipeline(name=...) as pipeline:`	steps created inside this block belong to the pipeline
`LoadDataFromDicts(data=inputs, batch_size=64)`	sends 64 rows at a time

`OpenAILLM(model=TEACHER, base_url=TEACHER_URL, api_key="EMPTY", generation_kwargs={"temperature": 0.3, "max_new_tokens": 300})`	vLLM ignores the key; temperature 0.3 = focused; at most 300 new tokens
`system_prompt=TEACHER_SYSTEM`	added as the system message of every request
`input_batch_size=32`	32 requests in flight at once; vLLM processes them together on the GPU
`load >> explain`	connect the two steps
`distiset = pipeline.run(use_cache=False)`	run everything; use_cache=False forces a fresh run
`distiset["default"]["train"]`	the output table of the last step
`def good(text, gold)`	the quality filter: 80-1,500 characters and it mentions the correct option text or letter
`explanations = {...}`	question_key → explanation, only for rows that passed
`write_jsonl(DATA / "synth" / "medqa_explanations.jsonl", ...)`	the raw synthetic data, kept separately (you publish it in 5.12)
`r["messages"][2]["content"] += f" Explanation: ..."`	append the explanation to the answer
`write_jsonl(SFT / "qa_train_synth.jsonl", rows)`	the improved training file; qa_train.jsonl stays untouched for comparison

CHECKPOINT: ls scripts shows 20_synth_explanations.py

SECTION 5.4 - Dry run on 50 rows and inspect quality

WHERE: VS Code Terminal, tmux, folder ~/medscriptai/scripts, (.venv-synth) active

GOAL: prove the pipeline works and the explanations are good BEFORE the full run.

Step 1. Start tmux:

```
tmux new -s synth
```

Step 2. Activate the synthetic-data environment (NOT .venv-train):

```
cd ~/medscriptai/scripts && source ../.venv-synth/bin/activate
```

Step 3. Run on 50 rows (1-2 minutes):

```
python 20_synth_explanations.py --limit 50
```

You should see: "50 MedQA rows need an explanation", distilabel's colourful pipeline log, then "kept 4x/50 explanations after the quality filter" (most rows should pass).

Step 4. Read three random explanations:

```
shuf -n 3 ../data/synth/medqa_explanations.jsonl | python -m json.tool
```

You should see: three {"question_key": ..., "explanation": ...} objects with sensible medical reasoning that mentions the correct option.

Step 5. Judge them with this checklist (write yes/no for each):

Question	Good sign
Does it support the GIVEN answer (not argue for another option)?	yes
Is it medically plausible (check one claim on Wikipedia or a textbook)?	yes
Is it under about 120 words and to the point?	yes
Does it avoid personal medical advice ("you should take...")?	yes

Watch out: "No module named 'requests'" or datasets errors = wrong environment; the prompt must start with (.venv-synth).

CHECKPOINT: most of the 50 passed and your checklist is mostly "yes".

If not: if explanations argue for another option, the teacher is not following instructions: make TEACHER_SYSTEM stricter and repeat Step 3.

SECTION 5.5 - Full synthetic run

WHERE: VS Code Terminal, tmux session synth, (.venv-synth) active

GOAL: explanations for all MedQA training rows.

Step 1. Run without --limit (roughly 15-30 minutes on an A100, estimate):

```
python 20_synth_explanations.py
```

You should see: "N MedQA rows need an explanation" (about 9,500-10,000), then "kept .../... explanations after the quality filter" and "wrote 23669 rows → data/sft/qa_train_synth.jsonl".

Step 2. Count how many answers now have an explanation, before and after:

```
grep -c "Explanation:" ../data/sft/qa_train.jsonl && grep -c "Explanation:" ../data/sft/qa_train_synth.jsonl
```

You should see: the second number is about 9,000-10,000 larger than the first.

CHECKPOINT: qa_train_synth.jsonl exists with 23669 lines (wc -l ../data/sft/qa_train_synth.jsonl).

SECTION 5.6 - Free the GPU

WHERE: VS Code Terminal

GOAL: no model server using GPU memory (training needs it).

Step 1. Stop the teacher:

```
docker rm -f medscriptai-teacher
```

Step 2. Check the GPU is free:

```
nvidia-smi --query-gpu=memory.used --format=csv
```

You should see: a small number such as 1 MiB.

CHECKPOINT: memory.used is near 0.

SECTION 5.7 - Train the three adapters

WHERE: VS Code Terminal, new tmux session, folder ~/medscriptai/scripts, (.venv-train) active

GOAL: outputs/adapters/sft-qa (retrained with explanations), sft-notes, sft-summary.

Step 1. Start tmux:

```
tmux new -s train3
```

Step 2. Activate the TRAINING environment:

```
cd ~/medscriptai/scripts && source ../.venv-train/bin/activate
```

Step 3. Train all three, one after another (about 2.5-3.5 hours in total on an A100, estimate):

```
python 10_train_sft.py --task qa --train-file ../data/sft/qa_train_synth.jsonl --max-train 12000 && python
10_train_sft.py --task notes && python 10_train_sft.py --task summary
```

Step 4. Detach (Ctrl+B, then D). Come back with tmux attach -t train3.

Run	Training rows	Steps (batch 16)	Rough time on A100
qa (synthetic)	12,000	750	45-70 min
notes	6,367	398	40-60 min
summary	6,367	398	40-60 min

You should see: three "DONE adapter=..." lines. The qa run replaces the Part 4 sft-qa adapter (the Part 4 one is still on the Hugging Face Hub).

Watch out: && means "run the next command only if the previous one succeeded". If one fails, fix it and run only the missing ones.

CHECKPOINT: ls ../outputs/adapters shows sft-qa, sft-notes, sft-summary, each with adapter_model.safetensors.

SECTION 5.8 - Create serve/start_vllm.sh and serve all adapters

BEFORE YOU START: multi-LoRA serving. vLLM loads the 16 GB base model ONCE and keeps your small adapters next to it. Each request names a model: "base" (no adapter), "qa", "notes" or "summary". vLLM applies the right adapter for each request, even inside the same GPU batch. One GPU, many specialised models.

WHERE: VS Code Explorer (create), then VS Code Terminal (run)

GOAL: base + adapters on port 8000, and the judge on port 8001, both on the A100.

Step 1. Right-click the folder serve.

Step 2. Click New File...

Step 3. Type start_vllm.sh and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
#!/usr/bin/env bash
# Serve Llama-3.1-8B-Instruct + all MedScriptAI LoRA adapters from ONE GPU process (multi-LoRA).
# Usage:  bash serve/start_vllm.sh           (from the repo folder)
# Model names clients can use: base, qa, notes, summary, qa-dpo, notes-dpo
set -euo pipefail                                  # stop on the first error

VLLM_IMAGE="${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0-cu129}"   # cu129 build + compat for R570 drivers
GPU_UTIL="${GPU_UTIL:-0.85}"                       # share of GPU memory vLLM may use (0.35 when the teacher also runs)
ADAPTERS="$(pwd)/outputs/adapters"                 # trained adapters on the VM

# Only register adapters that exist, so the script works after Part 4 (qa only) and after Part 6 (all).
LORA_ARGS=()
for pair in qa=sft-qa notes=sft-notes summary=sft-summary qa-dpo=dpo-qa notes-dpo=dpo-notes; do
  name="${pair%%=*}"; dir="${pair#*=}"
  if [ -f "$ADAPTERS/$dir/adapter_config.json" ]; then LORA_ARGS+=("$name=/adapters/$dir"); fi
done
echo "Adapters: ${LORA_ARGS[*]:-none}"
EXTRA=()                                           # becomes: --lora-modules qa=/adapters/sft-qa ...
if [ "${#LORA_ARGS[@]}" -gt 0 ]; then EXTRA=(--lora-modules "${LORA_ARGS[@]}"); fi

docker rm -f medscriptai-vllm 2>/dev/null || true   # remove an old container with the same name
docker run -d --name medscriptai-vllm \
  --gpus all --ipc=host \
  -p 8000:8000 \
  -e HF_TOKEN="${HF_TOKEN:-}" \
  -e VLLM_ENABLE_CUDA_COMPATIBILITY="${VLLM_ENABLE_CUDA_COMPATIBILITY:-1}" \
  -e VLLM_ALLOW_RUNTIME_LORA_UPDATING=True \
  -v "${HF_HOME:-$HOME/.cache/huggingface}":/root/.cache/huggingface \
  -v "$ADAPTERS":/adapters:ro \
  "$VLLM_IMAGE" \
  --model unsloth/Llama-3.1-8B-Instruct \
  --served-model-name base \
  --dtype bfloat16 \
  --max-model-len 4096 \
  --gpu-memory-utilization "$GPU_UTIL" \
  --enable-lora \
  --max-loras 4 \
  --max-lora-rank 16 \
  --max-cpu-loras 8 \
  "${EXTRA[@]}"

echo "Started. Follow the logs with:  docker logs -f medscriptai-vllm"
```

Line	What it does (plain English)
`GPU_UTIL="${GPU_UTIL:-0.85}"`	share of GPU memory; you lower it when the judge runs next to it
`ADAPTERS="$(pwd)/outputs/adapters"`	your adapters folder (run the script from ~/medscriptai)
`for pair in qa=sft-qa notes=sft-notes ... ; do ... done`	for each name=folder pair, register the adapter only if its folder exists; so the script works after Part 4 (qa only) and after Part 6A (all five)
`name="${pair%%=*}"; dir="${pair#*=}"`	bash string cutting: the part before "=" and the part after "="
`EXTRA=(--lora-modules "${LORA_ARGS[@]}")`	builds "--lora-modules qa=/adapters/sft-qa ..." only when adapters exist

`-e VLLM_ALLOW_RUNTIME_LORA_UPDATING=True`	allows adding/removing adapters while running (Part 6A). vLLM warns this is risky in production, so the production Compose file in Part 6B does NOT set it
`-v "$ADAPTERS":/adapters:ro`	your adapters appear inside the container at /adapters, read-only
`--model unsloth/Llama-3.1-8B-Instruct --served-model-name base`	the 16-bit base model, called "base"
`--max-model-len 4096`	longest context per request
`--enable-lora`	turn on LoRA support
`--max-loras 4`	how many DIFFERENT adapters can be active in one GPU batch. The default is 1, which makes requests for different adapters wait for each other: a classic production mistake
`--max-lora-rank 16`	must be at least the rank of your adapters (16); allowed values 1, 8, 16, 32, 64, 128, 256, 320, 512
`--max-cpu-loras 8`	how many adapters are cached in normal memory for fast swapping
`"${EXTRA[@]}"`	adds the --lora-modules list

Step 6. Make it executable:

```
chmod +x ~/medscriptai/serve/start_vllm.sh
```

Step 7. Start the student server with 35% of the GPU:

```
cd ~/medscriptai && GPU_UTIL=0.35 bash serve/start_vllm.sh
```

You should see: Adapters: qa=/adapters/sft-qa notes=/adapters/sft-notes summary=/adapters/sft-summary

Step 8. Wait until it is ready, then list its models:

```
until curl -s localhost:8000/v1/models > /dev/null; do sleep 10; done; curl -s localhost:8000/v1/models | jq -r
'.data[].id'
```

You should see: base, qa, notes, summary (one per line).

Step 9. Start the judge (the teacher) with 50% of the GPU:

```
GPU_UTIL=0.50 bash serve/start_teacher.sh
```

Step 10. Wait for it:

```
until curl -s localhost:8001/v1/models > /dev/null; do sleep 10; done; echo judge ready
```

Watch out: the two GPU_UTIL values must add up to about 0.9 or less, otherwise the second server says "insufficient GPU memory".

CHECKPOINT: Step 8 lists four models and Step 10 prints "judge ready".

SECTION 5.9 - Create and run scripts/30_generate_preds.py (answers on the test set)

WHERE: VS Code Explorer (create), then VS Code Terminal with (.venv-eval)

GOAL: one predictions file per task and model in outputs/preds.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 30_generate_preds.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Evaluation step A: send every test prompt to vLLM for each model/adapter and save the answers.

python scripts/30_generate_preds.py --task qa --models base qa qa-dpo
"base" = Llama-3.1-8B-Instruct with no adapter; "qa" etc. = LoRA adapter names given to vLLM.
"""
import argparse
import os
from concurrent.futures import ThreadPoolExecutor   # many requests at once = vLLM batches them on the GPU
from openai import OpenAI                            # vLLM speaks the OpenAI API
from common import SFT, OUT, read_jsonl, write_jsonl

p = argparse.ArgumentParser()
```

```
p.add_argument("--task", required=True, choices=["qa", "notes", "summary"])
p.add_argument("--models", nargs="+", required=True)
p.add_argument("--n", type=int, default=None, help="only the first N test rows")
p.add_argument("--url", default=os.getenv("VLLM_URL", "http://localhost:8000/v1"))
a = p.parse_args()

client = OpenAI(base_url=a.url, api_key="EMPTY")    # vLLM ignores the key, the SDK needs one
rows = read_jsonl(SFT / f"{a.task}_test.jsonl")[: a.n]
max_tokens = 400 if a.task == "qa" else 900


def ask(model, row):
    resp = client.chat.completions.create(
        model=model,
        messages=row["messages"][:2],               # system + user; never send the reference
        temperature=0.0,                            # greedy = reproducible scores
        max_tokens=max_tokens,
    )
    return resp.choices[0].message.content


for model in a.models:
    with ThreadPoolExecutor(max_workers=16) as pool:
        preds = list(pool.map(lambda r: ask(model, r), rows))
    out = [{"i": i, "source": r.get("source"), "answer_letter": r.get("answer_letter"),
            "prompt": r["messages"][1]["content"], "reference": r["messages"][2]["content"],
            "prediction": pred} for i, (r, pred) in enumerate(zip(rows, preds))]
    write_jsonl(OUT / "preds" / a.task / f"{model}.jsonl", out)
```

Line	What it does (plain English)
`from concurrent.futures import ThreadPoolExecutor`	send many requests at the same time; vLLM batches them
`from openai import OpenAI`	the official OpenAI client, pointed at vLLM
`--models`	names known to vLLM: "base" = no adapter, "qa" = base + qa adapter
`client = OpenAI(base_url=a.url, api_key="EMPTY")`	vLLM ignores the key, the client needs one
`rows = read_jsonl(SFT / f"{a.task}_test.jsonl")[: a.n]`	the held-out TEST set
`max_tokens = 400 if a.task == "qa" else 900`	answer length limit per task
`messages=row["messages"][:2]`	system + user only; the reference is never sent
`temperature=0.0`	greedy decoding: reproducible scores
`pool.map(lambda r: ask(model, r), rows)`	16 parallel workers, results in the original order
`out = [{"i", "source", "answer_letter", "prompt", "reference", "prediction"}]`	everything the metrics and the judge need
`write_jsonl(OUT / "preds" / a.task / f"{model}.jsonl", out)`	e.g. outputs/preds/qa/base.jsonl

Step 6. Activate the EVALUATION environment:

```
cd ~/medscriptai/scripts && source ../.venv-eval/bin/activate
```

Step 7. Generate (about 5-10 minutes):

```
python 30_generate_preds.py --task qa --models base qa && python 30_generate_preds.py --task notes --models base notes
&& python 30_generate_preds.py --task summary --models base summary
```

You should see: six lines such as "wrote 500 rows → outputs/preds/qa/base.jsonl" and "wrote 200 rows → outputs/preds/notes/notes.jsonl".

CHECKPOINT: ls ../outputs/preds/qa ../outputs/preds/notes ../outputs/preds/summary shows two files each.

SECTION 5.10 - Create and run scripts/31_eval_metrics.py (automatic metrics)

BEFORE YOU START: the metrics with a tiny example.

Metric	Reference	Prediction	Score (roughly)	Why
accuracy	Answer: C) ...	Answer: C) ...	1 (correct)	same letter

ROUGE-L	"patient presented with acute kidney injury"	"patient had acute kidney injury"	about 0.8	the longest common word sequence "patient ... acute kidney injury" is most of both
BERTScore F1	"the patient had a heart attack"	"the patient suffered a myocardial infarction"	high (about 0.9)	different words, same meaning
json_valid	(summary task)	{"age": "40"}	1	it parses as JSON

WHERE: VS Code Explorer (create), then VS Code Terminal with (.venv-eval)

GOAL: outputs/reports/metrics.json with every metric for every model.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 31_eval_metrics.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Evaluation step B: automatic metrics for every saved prediction file.

qa      -> accuracy (letter match), ROUGE-L, BERTScore
notes   -> ROUGE-L, BERTScore
summary -> JSON-valid rate, ROUGE-L, BERTScore
"""
import argparse
import json
import os
from pathlib import Path
from rouge_score import rouge_scorer                # Google's reference ROUGE implementation
from bert_score import score as bert_score          # BERTScore (Zhang et al., 2020)
from common import OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--tasks", nargs="+", default=["qa", "notes", "summary"])
p.add_argument("--bert-model", default=os.getenv("BERTSCORE_MODEL", "roberta-large"))
a = p.parse_args()

rouge = rouge_scorer.RougeScorer(["rougeL"], use_stemmer=True)   # stemmer: "infections" ~ "infection"


def letter(text):
    """'Answer: C) ...' -> 'C' (None if the model did not follow the format)."""
    if text and "Answer:" in text:
        tail = text.split("Answer:", 1)[1].strip()
        return tail[:1].upper() or None
    return None


def is_json(text):
    try:
        json.loads(text.strip().removeprefix("```json").removesuffix("```").strip())
        return True
    except (json.JSONDecodeError, AttributeError):
        return False


results = {}
for task in a.tasks:
    for path in sorted(Path(OUT / "preds" / task).glob("*.jsonl")):
        rows = read_jsonl(path)
        preds = [r["prediction"] or "" for r in rows]
        refs = [r["reference"] for r in rows]
        m = {"n": len(rows)}
        m["rougeL"] = sum(rouge.score(ref, pred)["rougeL"].fmeasure for ref, pred in zip(refs, preds)) / len(rows)
        _, _, f1 = bert_score(preds, refs, model_type=a.bert_model, lang="en", verbose=False)
        m["bertscore_f1"] = f1.mean().item()
        if task == "qa":
            m["accuracy"] = sum(letter(p_) == r["answer_letter"] for p_, r in zip(preds, rows)) / len(rows)
            m["format_ok"] = sum(letter(p_) is not None for p_ in preds) / len(rows)
        if task == "summary":
            m["json_valid"] = sum(is_json(p_) for p_ in preds) / len(rows)
        results[f"{task}/{path.stem}"] = {k: round(v, 4) if isinstance(v, float) else v for k, v in m.items()}
        print(f"{task:8} {path.stem:12} {results[f'{task}/{path.stem}']}")

report = OUT / "reports" / "metrics.json"
```

```
report.parent.mkdir(parents=True, exist_ok=True)
report.write_text(json.dumps(results, indent=1))
print(f"saved {report}")
```

Line	What it does (plain English)
`from rouge_score import rouge_scorer`	Google's reference ROUGE implementation
`from bert_score import score as bert_score`	BERTScore (Zhang et al., 2020)
`--bert-model roberta-large`	the standard BERTScore model for English (1.4 GB download on first use)
`RougeScorer(["rougeL"], use_stemmer=True)`	ROUGE-L; the stemmer treats "infections" and "infection" as the same
`def letter(text)`	the letter after "Answer:"
`def is_json(text)`	True if the text parses as JSON (also inside a ```json block)
`rouge.score(ref, pred)["rougeL"].fmeasure`	ROUGE-L F1 per row, then averaged
`bert_score(preds, refs, model_type=..., lang="en")`	BERTScore precision, recall, F1; we keep the mean F1
`m["accuracy"]`, `m["format_ok"]`	share of correct letters; share of answers that used the "Answer: X)" format at all
`m["json_valid"]`	share of summary outputs that are valid JSON
`report.write_text(...)`	saves outputs/reports/metrics.json

Step 6. Run:

```
python 31_eval_metrics.py
```

You should see: six lines such as

```
qa       base         {'n': 500, 'rougeL': ..., 'bertscore_f1': ..., 'accuracy': ..., 'format_ok': ...}
qa       qa           {'n': 500, 'rougeL': ..., 'bertscore_f1': ..., 'accuracy': ..., 'format_ok': ...}
...
saved /home/ubuntu/medscriptai/outputs/reports/metrics.json
```

How to read your numbers:

Metric	What a good change looks like	Trap
format_ok (qa)	fine-tuned close to 1.0	read it BEFORE accuracy: unparseable answers count as wrong
accuracy (qa)	fine-tuned higher than base	a few points can be noise: test it in Part 7
ROUGE-L	fine-tuned higher on notes/summary	copying the reference style raises it even when facts are wrong
BERTScore F1	values cluster around 0.80-0.90	compare models with each other, never with a fixed "good" value
json_valid (summary)	fine-tuned close to 1.0	valid JSON can still contain invented facts (the judge checks)

CHECKPOINT: cat ../outputs/reports/metrics.json shows six entries.

SECTION 5.11 - Create and run scripts/32_judge.py (LLM-as-a-judge)

BEFORE YOU START: an LLM judge is a strong model asked to grade answers against a rubric. In score mode it gives each answer 1-5. In pairwise mode (Part 6A) it compares two answers; because judges prefer whichever answer comes first, we ask twice with the order swapped and only count a win if it wins both times.

WHERE: VS Code Explorer (create), then VS Code Terminal with (.venv-eval)

GOAL: a 1-5 quality score per model on 100 test items per task.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 32_judge.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Evaluation step C: LLM-as-a-judge.

--mode score     one answer at a time, 1-5 rubric score           (Part 5)
--mode pairwise  model A vs model B, run twice with order swapped    (Part 6)
                 -> removes position bias: a "win" must win in BOTH orders
"""
import argparse
```

```
import json
import os
import re
from concurrent.futures import ThreadPoolExecutor
from openai import OpenAI
from common import OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--task", required=True, choices=["qa", "notes", "summary"])
p.add_argument("--mode", default="score", choices=["score", "pairwise"])
p.add_argument("--models", nargs="+", required=True, help="score: any number; pairwise: exactly two (A B)")
p.add_argument("--n", type=int, default=100)
a = p.parse_args()

judge = OpenAI(base_url=os.getenv("JUDGE_URL", "http://localhost:8001/v1"), api_key="EMPTY")
JUDGE_MODEL = os.getenv("JUDGE_MODEL", "teacher")
RUBRIC = {
    "qa": "correct final answer (vs the reference), sound medical reasoning, no unsafe advice",
    "notes": "faithful to the conversation (no invented facts), complete key findings, clear clinical style",
    "summary": "valid JSON, every value supported by the note, no important fact missing",
}


def ask_json(prompt):
    """Call the judge at temperature 0 and pull the first {...} object out of the reply."""
    text = judge.chat.completions.create(
        model=JUDGE_MODEL, temperature=0.0, max_tokens=300,
        messages=[{"role": "user", "content": prompt}]).choices[0].message.content
    match = re.search(r"\{.*\}", text, re.DOTALL)
    try:
        return json.loads(match.group(0)) if match else {}
    except json.JSONDecodeError:
        return {}


def score_prompt(row):
    return (f"You are a strict medical expert grader. Criteria: {RUBRIC[a.task]}.\n\n"
            f"TASK INPUT:\n{row['prompt'][:4000]}\n\nREFERENCE ANSWER:\n{row['reference'][:3000]}\n\n"
            f"MODEL ANSWER:\n{row['prediction'][:3000]}\n\n"
            'Return ONLY a JSON object: {"score": <1-5>, "reason": "<one sentence>"}')


def pair_prompt(row, first, second):
    return (f"You are a strict medical expert judge. Criteria: {RUBRIC[a.task]}.\n\n"
            f"TASK INPUT:\n{row['prompt'][:4000]}\n\nREFERENCE ANSWER:\n{row['reference'][:3000]}\n\n"
            f"ANSWER A:\n{first[:3000]}\n\nANSWER B:\n{second[:3000]}\n\n"
            'Which answer is better? Return ONLY a JSON object: {"winner": "A" or "B" or "tie", "reason": "..."}')


preds = {m: read_jsonl(OUT / "preds" / a.task / f"{m}.jsonl")[: a.n] for m in a.models}
report = {}
with ThreadPoolExecutor(max_workers=16) as pool:
    if a.mode == "score":
        for m, rows in preds.items():
            scores = [r.get("score") for r in pool.map(lambda row: ask_json(score_prompt(row)), rows)]
            valid = [s for s in scores if isinstance(s, (int, float))]
            report[m] = {"mean_score": round(sum(valid) / max(len(valid), 1), 3), "judged": len(valid)}
    else:
        ma, mb = a.models
        rows_a, rows_b = preds[ma], preds[mb]

        def duel(i):
            ab = ask_json(pair_prompt(rows_a[i], rows_a[i]["prediction"], rows_b[i]["prediction"]))
            ba = ask_json(pair_prompt(rows_a[i], rows_b[i]["prediction"], rows_a[i]["prediction"]))
            first, second = ab.get("winner"), ba.get("winner")
            if first == "A" and second == "B":
                return ma                                # A won in both orders
            if first == "B" and second == "A":
                return mb
            return "tie"                                 # disagreement across orders = tie

        outcomes = list(pool.map(duel, range(min(len(rows_a), len(rows_b)))))
        n = len(outcomes)
        report = {f"{ma}_wins": round(outcomes.count(ma) / n, 3),
                  f"{mb}_wins": round(outcomes.count(mb) / n, 3),
                  "ties": round(outcomes.count("tie") / n, 3), "n": n,
```

```
                  "per_item": outcomes}                    # kept for judge-vs-human agreement (Part 7)

print(json.dumps({k: v for k, v in report.items() if k != "per_item"}, indent=1))
path = OUT / "reports" / f"judge_{a.task}_{a.mode}_{'_vs_'.join(a.models)}.json"
path.parent.mkdir(parents=True, exist_ok=True)
path.write_text(json.dumps(report, indent=1))
```

Line	What it does (plain English)
`judge = OpenAI(base_url=os.getenv("JUDGE_URL", "http://localhost:8001/v1"), ...)`	the judge is the teacher server on port 8001
`RUBRIC = {...}`	what "good" means per task; for notes, faithfulness (no invented facts) comes first
`def ask_json(prompt)`	asks at temperature 0 and pulls the first {...} out of the reply; {} if the judge broke the format
`def score_prompt(row)`	task input + reference + model answer + "Return ONLY a JSON object {score, reason}"; long texts are cut at 3,000-4,000 characters
`def pair_prompt(row, first, second)`	the pairwise version with ANSWER A and ANSWER B
`if a.mode == "score":`	grade each answer alone and average the valid scores
`def duel(i)`	pairwise: ask A-then-B and B-then-A
`if first == "A" and second == "B": return ma`	a model wins only if it is preferred in BOTH orders, otherwise "tie": this cancels position bias (Zheng et al., 2023)
`report = {... "per_item": outcomes}`	win rates plus each verdict (Part 7 compares them with your own labels)
`print(json.dumps({... if k != "per_item"}))`	prints only the summary
`path = OUT / "reports" / f"judge_..."`	one report file per run

Step 6. Run score mode for notes and summary:

```
python 32_judge.py --task notes --mode score --models base notes --n 100 && python 32_judge.py --task summary --mode score --models base summary --n 100
```

You should see: two JSON blocks like {"base": {"mean_score": ..., "judged": 100}, "notes": {"mean_score": ..., "judged": 100}}.

Watch out: the judge is also the teacher that wrote the synthetic explanations, so it may favour its own style (self-preference bias). Say this in interviews; Part 7 measures judge reliability.

CHECKPOINT: outputs/reports contains judge_notes_score_base_notes.json and judge_summary_score_base_summary.json.

SECTION 5.12 - Publish your datasets

WHERE: VS Code Terminal, folder ~/medscriptai/scripts, (.venv-train) active

GOAL: two private dataset repos on the Hub.

Step 1. Activate the training environment:

```
source ../.venv-train/bin/activate
```

Step 2. Publish the SFT data:

```
python 12_push_to_hub.py --type dataset --folder ../data/sft --repo YOUR_HF_USERNAME/medscriptai-sft
```

Step 3. Publish the synthetic explanations:

```
python 12_push_to_hub.py --type dataset --folder ../data/synth --repo YOUR_HF_USERNAME/medscriptai-medqa-explanations
```

Step 4. On each dataset page click Edit dataset card and write: the sources and licences (MedQA CC-BY-4.0, MedMCQA Apache-2.0, PubMedQA MIT, augmented-clinical-notes MIT), how you cleaned and de-duplicated, that the explanations are synthetic from Qwen2.5-14B-Instruct, and "not for clinical use".

Watch out: CC-BY-4.0 requires attribution: credit the MedQA authors in the card.

CHECKPOINT: both dataset pages show your files.

SECTION 5.13 - Cloud step: reports to S3, token into Secrets Manager

BEFORE YOU START: Secrets Manager is AWS's safe for passwords and tokens. Servers read secrets from it at start-up instead of having them written in files or code.

WHERE: VS Code Terminal, folder ~/medscriptai (cd ~/medscriptai)

GOAL: practise storing artifacts and secrets like production systems.

Step	FakeCloud (free practice)	Real AWS (same step)
1 Point the CLI	`export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1`	`unset AWS_ENDPOINT_URL`
2 Bucket name	`export BUCKET=medscriptai-artifacts-$(aws sts get-caller-identity --query Account --output text)-us-east-1`	same
3 Upload reports	`aws s3 sync outputs/reports s3://$BUCKET/reports`	same
4 Upload all adapters	`aws s3 sync outputs/adapters s3://$BUCKET/adapters --exclude "*/checkpoint-*"`	same
5 Store the HF token	`aws secretsmanager create-secret --name medscriptai/HF_TOKEN --secret-string "$HF_TOKEN"`	same (about US$0.40 per secret per month, verify; deleted in teardown)
6 Read it back	`aws secretsmanager get-secret-value --secret-id medscriptai/HF_TOKEN --query SecretString --output text`	same
You should see	your hf_ token	your hf_ token; also listed (masked) in AWS console > Secrets Manager

What each piece means:

Piece	What it does
`--exclude "*/checkpoint-*"`	skip training checkpoints
`create-secret --name medscriptai/HF_TOKEN`	store an encrypted secret; the slash only groups names
`--query SecretString --output text`	print only the value

CHECKPOINT: aws s3 ls s3://$BUCKET/adapters/ lists sft-qa, sft-notes, sft-summary.

SECTION 5.14 - Commit

WHERE: VS Code Terminal

GOAL: Part 5 scripts saved in Git.

Step 1. Run:

```
cd ~/medscriptai && git add . && git commit -m "Part 5: synthetic data, three adapters, evaluation"
```

CHECKPOINT: git log --oneline shows the new commit.

PART 5 RESULT CHECK

Check	Command	Expected
synthetic file	wc -l data/sft/qa_train_synth.jsonl	23669
three adapters	ls outputs/adapters	sft-notes sft-qa sft-summary
six prediction files	ls outputs/preds/*	base.jsonl + adapter file per task
metrics	cat outputs/reports/metrics.json	six task/model entries; fine-tuned format_ok and json_valid near 1.0
judge	ls outputs/reports/judge_*	two score files
artifacts in (Fake)S3	aws s3 ls s3://$BUCKET --recursive | head	reports/... and adapters/...

5.15 Common mistake at this level

The mistake: celebrating a metric that does not measure what you think: ROUGE-L rises because the model copies the reference style while facts get worse; accuracy rises because test questions leaked into training; the judge prefers the fine-tuned model because it writes like the judge.

How to spot it: read 10 random predictions next to their references; check format_ok before accuracy; keep the de-duplication (Section 4.4) and run the contamination check (Part 7); compare at least two independent metrics; run the significance test before claiming a win.

PART 6A - ADVANCED: DPO, vLLM multi-LoRA hot-swap, FastAPI + LangSmith

BEFORE YOU START: what you will build in this part

```
Section 6A.1-6A.3  build preference pairs (chosen vs rejected) from your own SFT model, with distilabel
Section 6A.4-6A.5  train DPO on top of the SFT adapters -> dpo-qa, dpo-notes
Section 6A.6       serve 5 adapters on one GPU and add/remove adapters without restarting
Section 6A.7       evaluate base vs SFT vs DPO (metrics + pairwise judge)
Section 6A.8-6A.11 your web API (FastAPI), its tests, and LangSmith tracing
```

DPO in one paragraph (details in Part 3, section 3.8): SFT taught the model to imitate good answers. DPO teaches it to PREFER better answers over worse ones. You give it (prompt, chosen, rejected) triples; it increases the probability of the chosen answer and decreases the rejected one, while a frozen copy of the SFT model (the reference) stops it from drifting too far.

Where the pairs come from (real example shape from this pipeline):

```
prompt   : "Pediatric airway differs from adult - Options: A) Large tongue  B) Short epiglottis ..."
candidate 1 (your SFT model, temperature 0.9): "Answer: B) Short epiglottis ..."   -> wrong letter,  rating 2
candidate 2 (your SFT model, temperature 0.9): "Answer: A) Large tongue ..."       -> right letter,  rating 4
candidate 3 (teacher)                        : "Answer: A) Large tongue ..."       -> right letter,  rating 5
chosen   = candidate 3 (score 10 + 5 = 15)     rejected = candidate 1 (score 0 + 2 = 2)
```

Score = 10 points if the letter is correct + the judge's 1-5 rating. Ground truth always beats the judge.

Word	Plain meaning
preference pair	one prompt with a better (chosen) and a worse (rejected) answer
reference model	a frozen copy of the SFT model that DPO compares against
beta	how strongly DPO is pulled back towards the reference (0.1 = standard)
hot-swap	adding or removing an adapter on a running server without restarting it
API endpoint	a URL your program offers, e.g. POST /v1/generate
tracing	recording each request's input, output and timing (LangSmith)

Budget: about 4 GPU-hours (about US$5-6).

SECTION 6A.1 - Start both servers again

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: student (your SFT adapters) on port 8000 and teacher/judge on port 8001.

Step 1. Check whether they are still running from Part 5:

```
curl -s localhost:8000/v1/models | jq -r '.data[].id'; curl -s localhost:8001/v1/models | jq -r '.data[].id'
```

You should see: base, qa, notes, summary and teacher. If you do, skip to Section 6A.2.

Step 2. If not, start the student:

```
cd ~/medscriptai && GPU_UTIL=0.35 bash serve/start_vllm.sh
```

Step 3. Start the teacher:

```
GPU_UTIL=0.50 bash serve/start_teacher.sh
```

Step 4. Wait for both:

```
until curl -s localhost:8000/v1/models >/dev/null && curl -s localhost:8001/v1/models >/dev/null; do sleep 10; done;
echo both ready
```

CHECKPOINT: "both ready".

SECTION 6A.2 - Create scripts/21_build_dpo_pairs.py

WHERE: VS Code Explorer, folder scripts

GOAL: the preference-pair builder.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 21_build_dpo_pairs.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Part 6: build DPO preference pairs (prompt, chosen, rejected) with distilabel.

For each held-out prompt we collect 3 candidate answers:
  2 samples from YOUR SFT adapter (served by vLLM under the adapter name, e.g. "qa")
  1 answer from the teacher model
The teacher then rates all 3 (UltraFeedback, 1-5). For qa we ALSO check the letter against the gold answer,
because a verifiable signal beats any judge. Best candidate = chosen, worst = rejected.
"""
import argparse
import os
from distilabel.models import OpenAILLM
from distilabel.pipeline import Pipeline
from distilabel.steps import GroupColumns, LoadDataFromDicts
from distilabel.steps.tasks import TextGeneration, UltraFeedback
from common import PREF, SYSTEM, read_jsonl, write_jsonl

p = argparse.ArgumentParser()
p.add_argument("--task", default="qa", choices=["qa", "notes", "summary"])
p.add_argument("--limit", type=int, default=1500)
a = p.parse_args()

STUDENT_URL = os.getenv("STUDENT_URL", "http://localhost:8000/v1")  # vLLM with your SFT adapters
TEACHER_URL = os.getenv("TEACHER_URL", "http://localhost:8001/v1")  # vLLM with the teacher
TEACHER = os.getenv("TEACHER_MODEL", "teacher")

rows = read_jsonl(PREF / f"{a.task}_dpo.jsonl")[: a.limit]
inputs = [{"instruction": r["messages"][1]["content"],                # the user message
           "gold_letter": r.get("answer_letter")} for r in rows]      # None for notes/summary


def llm(model, url, temperature):
    return OpenAILLM(model=model, base_url=url, api_key="EMPTY",
                     generation_kwargs={"temperature": temperature, "max_new_tokens": 700})


with Pipeline(name=f"medscriptai-dpo-{a.task}") as pipeline:
    load = LoadDataFromDicts(data=inputs, batch_size=64)
    student_1 = TextGeneration(name="student_1", llm=llm(a.task, STUDENT_URL, 0.9),
                               system_prompt=SYSTEM[a.task], input_batch_size=32)
    student_2 = TextGeneration(name="student_2", llm=llm(a.task, STUDENT_URL, 0.9),
                               system_prompt=SYSTEM[a.task], input_batch_size=32)
    teacher = TextGeneration(name="teacher", llm=llm(TEACHER, TEACHER_URL, 0.3),
                             system_prompt=SYSTEM[a.task], input_batch_size=32)
    group = GroupColumns(columns=["generation", "model_name"],          # 3 single answers -> 1 list
                         output_columns=["generations", "generation_models"])
    rate = UltraFeedback(llm=llm(TEACHER, TEACHER_URL, 0.0), aspect="overall-rating", input_batch_size=16)
    load >> [student_1, student_2, teacher] >> group >> rate            # fan out, then join

data = pipeline.run(use_cache=False)["default"]["train"]


def letter_of(text):
    """Read the letter after 'Answer:' ('Answer: C) ...' -> 'C')."""
    if text and "Answer:" in text:
        tail = text.split("Answer:", 1)[1].strip()
        return tail[:1].upper() if tail else None
    return None


pairs = []
for row in data:
    scored = []
    for text, rating in zip(row["generations"], row["ratings"]):
        if not text or rating is None:
            continue
        correct = 1 if row["gold_letter"] and letter_of(text) == row["gold_letter"] else 0
```

```
        scored.append((correct * 10 + rating, text))    # correctness dominates, rating breaks ties
    if len(scored) < 2:
        continue
    scored.sort(key=lambda s: s[0])
    (low, rejected), (high, chosen) = scored[0], scored[-1]
    if high - low < 2:                                  # too similar = weak training signal, skip
        continue
    system = {"role": "system", "content": SYSTEM[a.task]}
    pairs.append({"prompt": [system, {"role": "user", "content": row["instruction"]}],
                  "chosen": [{"role": "assistant", "content": chosen}],
                  "rejected": [{"role": "assistant", "content": rejected}],
                  "score_chosen": high, "score_rejected": low})

write_jsonl(PREF / f"{a.task}_pairs.jsonl", pairs)
print(f"{len(pairs)} preference pairs from {len(inputs)} prompts")
```

Line	What it does (plain English)
`from distilabel.steps import GroupColumns, LoadDataFromDicts`	GroupColumns merges the outputs of parallel steps into one list
`from distilabel.steps.tasks import TextGeneration, UltraFeedback`	UltraFeedback = the rating task from the UltraFeedback paper: the judge rates several answers 1-5
`STUDENT_URL` (port 8000)	vLLM with YOUR SFT adapters; model "qa" = base + sft-qa
`rows = read_jsonl(PREF / f"{a.task}_dpo.jsonl")[: a.limit]`	the held-out DPO prompts from Section 4.5 (never used in SFT)
`"gold_letter": r.get("answer_letter")`	the correct letter for qa; None for notes
`def llm(model, url, temperature)`	a small helper so every step uses the same settings
`student_1`, `student_2` at temperature 0.9	two DIFFERENT samples from your SFT model; high temperature creates variety, including mistakes worth un-learning
`teacher` at temperature 0.3	a third, usually stronger, candidate
`GroupColumns(columns=["generation", "model_name"], output_columns=["generations", "generation_models"])`	three single answers → one list of three
`UltraFeedback(llm=..., aspect="overall-rating")`	the teacher rates all three together
`load >> [student_1, student_2, teacher] >> group >> rate`	fan out to three generators, join, rate
`def letter_of(text)`	the letter after "Answer:"
`correct = 1 if ... letter_of(text) == row["gold_letter"] else 0`	the verifiable check: is the letter right?
`scored.append((correct * 10 + rating, text))`	correctness dominates; the rating breaks ties
`if high - low < 2: continue`	skip near-ties (no real difference = noisy signal)
`pairs.append({"prompt": [...], "chosen": [...], "rejected": [...]})`	exactly TRL's conversational preference format

CHECKPOINT: ls scripts shows 21_build_dpo_pairs.py

SECTION 6A.3 - Build the pairs

WHERE: VS Code Terminal, tmux, folder ~/medscriptai/scripts, (.venv-synth) active

GOAL: data/pref/qa_pairs.jsonl and data/pref/notes_pairs.jsonl

Step 1. Start tmux:

```
tmux new -s pairs
```

Step 2. Activate the synthetic-data environment:

```
cd ~/medscriptai/scripts && source ../.venv-synth/bin/activate
```

Step 3. Build QA pairs from 1,500 prompts (about 20-40 minutes, estimate):

```
python 21_build_dpo_pairs.py --task qa --limit 1500
```

You should see: "wrote N rows → data/pref/qa_pairs.jsonl" and "N preference pairs from 1500 prompts". N is usually several hundred: prompts where all three answers were equally good are skipped on purpose.

Step 4. Build notes pairs from 600 prompts:

```
python 21_build_dpo_pairs.py --task notes --limit 600
```

Step 5. Look at one pair:

```
head -n 1 ../data/pref/qa_pairs.jsonl | python -m json.tool
```

You should see: "prompt" (system + user), "chosen" (often the correct letter), "rejected" (often a wrong letter), "score_chosen", "score_rejected".

Step 6. Check the share of pairs where chosen is correct and rejected is not:

```
python -c "import json; p=[json.loads(l) for l in open('../data/pref/qa_pairs.jsonl')]; print(len(p), 'pairs;',
sum(x['score_chosen']>=10 and x['score_rejected']<10 for x in p), 'correct-vs-wrong pairs')"
```

CHECKPOINT: at least about 300 QA pairs and 150 notes pairs.

If not: if almost all prompts are skipped, your SFT model and the teacher agree on nearly everything: use more prompts (--limit 1891) or lower the threshold 2 to 1 in the script.

SECTION 6A.4 - Create scripts/22_train_dpo.py

BEFORE YOU START: what you should see in the DPO log. At step 0 the model equals the reference, so the loss is exactly 0.693 (= ln 2; we verified this on a test run). As training works, loss falls, rewards/margins (how much more the model prefers chosen over rejected) rises, and rewards/accuracies (share of pairs where chosen wins) goes above 0.5.

WHERE: VS Code Explorer, folder scripts

GOAL: the DPO training script.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 22_train_dpo.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Part 6: DPO preference alignment that continues from your SFT adapter.

python scripts/22_train_dpo.py --task qa     ->  outputs/adapters/dpo-qa
"""
from unsloth import FastLanguageModel, PatchDPOTrainer   # Unsloth first
PatchDPOTrainer()                                         # no-op in current Unsloth; kept for older versions

import argparse
import json
from datasets import load_dataset
from trl import DPOConfig, DPOTrainer
from common import PREF, OUT

p = argparse.ArgumentParser()
p.add_argument("--task", default="qa", choices=["qa", "notes", "summary"])
p.add_argument("--beta", type=float, default=0.1)       # how strongly to stay close to the SFT model
p.add_argument("--lr", type=float, default=5e-6)        # DPO needs a MUCH smaller LR than SFT
p.add_argument("--epochs", type=float, default=1.0)
a = p.parse_args()
sft_dir = OUT / "adapters" / f"sft-{a.task}"
out_dir = OUT / "adapters" / f"dpo-{a.task}"

# 1) Load base + your SFT adapter as TRAINABLE. DPO will keep improving these same LoRA weights.
model, tokenizer = FastLanguageModel.from_pretrained(str(sft_dir), max_seq_length=2048, load_in_4bit=True)
tokenizer.padding_side = "left"                           # TRL's DPO collator expects left padding

# 2) Preference data: {"prompt": [...], "chosen": [...], "rejected": [...]} (conversational format).
pairs = load_dataset("json", data_files=str(PREF / f"{a.task}_pairs.jsonl"), split="train")
pairs = pairs.select_columns(["prompt", "chosen", "rejected"])
split = pairs.train_test_split(test_size=0.05, seed=42)  # 5% held out to watch reward accuracy

cfg = DPOConfig(
    output_dir=str(OUT / "checkpoints" / f"dpo-{a.task}"),
    beta=a.beta,
    loss_type=["sigmoid"],                # the original DPO loss
    max_length=2048,                      # prompt + answer are truncated to this many tokens
    per_device_train_batch_size=2,        # each example = 2 sequences (chosen + rejected)
    gradient_accumulation_steps=8,        # effective batch 16 pairs
```

```
    num_train_epochs=a.epochs,
    learning_rate=a.lr,
    lr_scheduler_type="cosine",
    warmup_steps=10,
    optim="adamw_8bit",
    bf16=True,
    logging_steps=10,
    eval_strategy="steps",
    eval_steps=50,
    save_steps=200,
    save_total_limit=2,
    report_to="none",
    seed=3407,
)

# ref_model=None + a model that already has an adapter -> TRL copies it into a frozen "ref" adapter,
# so the reference policy is your SFT model (textbook DPO) without loading a second 8B model.
trainer = DPOTrainer(model=model, ref_model=None, args=cfg, processing_class=tokenizer,
                     train_dataset=split["train"], eval_dataset=split["test"])
print("adapters inside the model:", list(trainer.model.peft_config.keys()))   # expect ['default', 'ref']

trainer.train()
metrics = trainer.evaluate()
print({k: round(v, 4) for k, v in metrics.items() if "rewards" in k})   # accuracies should be > 0.5

# Save ONLY the trained adapter ("default"), not the frozen "ref" copy.
model.save_pretrained(str(out_dir), selected_adapters=["default"])
tokenizer.save_pretrained(str(out_dir))
(out_dir / "dpo_eval.json").write_text(json.dumps(metrics, indent=1))
print(f"DONE adapter={out_dir}")
```

Line	What it does (plain English)
`from unsloth import FastLanguageModel, PatchDPOTrainer` and `PatchDPOTrainer()`	Unsloth first; in Unsloth 2026.10 PatchDPOTrainer() does nothing (its source just returns) and is kept for older versions
`--beta 0.1`, `--lr 5e-6`	standard DPO strength; a learning rate 40x smaller than SFT, because DPO breaks fluency easily
`sft_dir = OUT / "adapters" / f"sft-{a.task}"`	start from your SFT adapter
`FastLanguageModel.from_pretrained(str(sft_dir), ...)`	loads 4-bit base + SFT LoRA as trainable
`tokenizer.padding_side = "left"`	TRL's DPO expects left padding
`select_columns(["prompt", "chosen", "rejected"])`	keep only what DPO needs
`train_test_split(test_size=0.05, seed=42)`	5% of pairs held out to measure reward accuracy
`loss_type=["sigmoid"]`	the original DPO loss
`per_device_train_batch_size=2`, `gradient_accumulation_steps=8`	each pair is 2 sequences; effective batch 16 pairs
`DPOTrainer(model=model, ref_model=None, ...)`	with ref_model=None and an existing adapter, TRL 1.15 copies your SFT adapter into a frozen adapter called "ref" and uses it as the reference (we checked this in TRL's source and in a test run)
`print("adapters inside the model:", ...)`	must print ['default', 'ref']
`metrics = trainer.evaluate()`	reward metrics on the 5% held-out pairs
`model.save_pretrained(str(out_dir), selected_adapters=["default"])`	save only the trained adapter, not the frozen copy

CHECKPOINT: ls scripts shows 22_train_dpo.py

SECTION 6A.5 - Train DPO

WHERE: VS Code Terminal, tmux, folder ~/medscriptai/scripts, (.venv-train) active

GOAL: outputs/adapters/dpo-qa and dpo-notes.

Step 1. Free the GPU:

```
docker rm -f medscriptai-vllm medscriptai-teacher
```

Step 2. Start tmux:

```
tmux new -s dpo
```

Step 3. Activate the training environment:

```
cd ~/medscriptai/scripts && source ../.venv-train/bin/activate
```

Step 4. Train (about 20-40 minutes per task, estimate):

```
python 22_train_dpo.py --task qa && python 22_train_dpo.py --task notes
```

You should see:

```
adapters inside the model: ['default', 'ref']
{'loss': 0.6931, ... 'rewards/accuracies': 0.0, 'rewards/margins': 0.0, ...}     <- step 0: exactly ln 2
{'loss': 0.6..., ... 'rewards/accuracies': 0.6..., 'rewards/margins': 0.1..., ...}
...
{'eval_rewards/accuracies': ..., 'eval_rewards/margins': ...}
DONE adapter=/home/ubuntu/medscriptai/outputs/adapters/dpo-qa
```

What you see	Meaning	Action
loss falls below 0.69, accuracies above 0.5, margins rising	DPO is learning the preferences	none
rewards/chosen AND rewards/rejected both fall fast	known DPO failure: pushing everything down	rerun with --lr 2e-6 or --beta 0.2
only ['default'] printed	the reference is the base model, not SFT	still works; check your TRL version is 1.15 (pip show trl)

CHECKPOINT: ls ../outputs/adapters shows dpo-qa and dpo-notes; eval_rewards/accuracies above 0.5.

SECTION 6A.6 - Serve five adapters and hot-swap them

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: base + sft-qa + sft-notes + sft-summary + dpo-qa + dpo-notes on one GPU; add and remove adapters while it runs.

Step 1. Start the student server with more memory:

```
cd ~/medscriptai && GPU_UTIL=0.60 bash serve/start_vllm.sh
```

You should see: Adapters: qa=/adapters/sft-qa notes=/adapters/sft-notes summary=/adapters/sft-summary qa-dpo=/adapters/dpo-qa notes-dpo=/adapters/dpo-notes

Step 2. Wait and list:

```
until curl -s localhost:8000/v1/models > /dev/null; do sleep 10; done; curl -s localhost:8000/v1/models | jq -r
'.data[].id'
```

You should see: base, qa, notes, summary, qa-dpo, notes-dpo

Step 3. Ask the SAME question to two different "models" on the same server:

```
for m in base qa-dpo; do curl -s localhost:8000/v1/chat/completions -H "Content-Type: application/json" -d "
{\"model\":\"$m\",\"max_tokens\":120,\"messages\":[{\"role\":\"user\",\"content\":\"A pregnant woman has cystitis. Which
is safest?\n\nOptions:\nA) Doxycycline\nB) Nitrofurantoin\nC) Ciprofloxacin\nD) Tetracycline\"}]}" | jq -r
'.choices[0].message.content'; echo ---; done
```

You should see: two different answers; the qa-dpo one starts with "Answer: B)".

Step 4. Remove an adapter while the server runs:

```
curl -s -X POST localhost:8000/v1/unload_lora_adapter -H "Content-Type: application/json" -d '{"lora_name": "notes-
dpo"}'
```

Step 5. Add it back under a new version name:

```
curl -s -X POST localhost:8000/v1/load_lora_adapter -H "Content-Type: application/json" -d '{"lora_name": "notes-dpo-v2", "lora_path": "/adapters/dpo-notes"}'
```

Step 6. List again:

```
curl -s localhost:8000/v1/models | jq -r '.data[].id'
```

You should see: notes-dpo gone, notes-dpo-v2 present: no restart, no downtime.

Step 7. Put the original name back (the evaluation uses it):

```
curl -s -X POST localhost:8000/v1/load_lora_adapter -H "Content-Type: application/json" -d '{"lora_name": "notes-dpo", "lora_path": "/adapters/dpo-notes"}'
```

How it works: the 16 GB base stays on the GPU once; each adapter is about 80-170 MB; per request vLLM reads the "model" name and applies that adapter's small matrices (the batched-LoRA idea from the Punica and S-LoRA papers). Note: the adapters were trained on the 4-bit base and are served on the 16-bit base; that is standard practice, and it is why you evaluate on THIS server, not inside the training script.

CHECKPOINT: Step 6 showed notes-dpo-v2 and Step 7 succeeded.

SECTION 6A.7 - Evaluate base vs SFT vs DPO

WHERE: VS Code Terminal, folder ~/medscriptai/scripts, (.venv-eval) active; student server on 8000

GOAL: numbers that show whether DPO helped.

Step 1. Start the judge next to the student (0.60 + 0.30 = 0.90):

```
cd ~/medscriptai && GPU_UTIL=0.30 bash serve/start_teacher.sh
```

Step 2. Activate the evaluation environment:

```
cd ~/medscriptai/scripts && source ../.venv-eval/bin/activate
```

Step 3. Answers from the DPO adapters:

```
python 30_generate_preds.py --task qa --models qa-dpo && python 30_generate_preds.py --task notes --models notes-dpo
```

Step 4. Recompute all metrics:

```
python 31_eval_metrics.py
```

Step 5. Wait for the judge:

```
until curl -s localhost:8001/v1/models > /dev/null; do sleep 10; done; echo ready
```

Step 6. Pairwise judge, DPO vs SFT, with position swap:

```
python 32_judge.py --task qa --mode pairwise --models qa-dpo qa --n 200 && python 32_judge.py --task notes --mode pairwise --models notes-dpo notes --n 100
```

You should see: {"qa-dpo_wins": ..., "qa_wins": ..., "ties": ..., "n": 200} and the same for notes.

If you see...	It means...	Do this next
qa-dpo accuracy higher and qa-dpo_wins > qa_wins	DPO helped	test significance in Part 7 before claiming it
same accuracy, more wins	better explanations, same correctness	report both honestly
lower accuracy	DPO hurt correctness	--lr 2e-6, --beta 0.2, check pair quality (6A.3 Step 6)
ties above 60%	the judge flips with order: weak signal	read 10 duels yourself

If the judge fails to start ("insufficient GPU memory"): run GPU_UTIL=0.45 bash serve/start_vllm.sh, then repeat Step 1.

CHECKPOINT: metrics.json has qa/qa-dpo and notes/notes-dpo; two pairwise judge files exist.

SECTION 6A.8 - Create the API files (api/requirements.txt and api/app.py)

BEFORE YOU START: what the API does. Clients (a website, a mobile app, a hospital system) should not talk to vLLM directly. Your API sits in front: it checks an API key, validates the input, picks the adapter for the task (qa → qa-dpo, notes → notes, summary → summary), adds the right system prompt, calls vLLM, measures latency, and returns clean JSON with a disclaimer. Request and response look like this:

```
POST /v1/generate   header X-API-Key: dev-key
{"task": "qa", "input": "A pregnant woman has cystitis. Which is safest?\n\nOptions:\nA) ...\nB) Nitrofurantoin ..."}

{"request_id": "a6f4...", "task": "qa", "model": "qa-dpo", "output": "Answer: B) Nitrofurantoin ...",
 "parsed_json": null, "latency_ms": 640, "disclaimer": "AI-generated. For education/documentation support only. Not
medical advice."}
```

WHERE: VS Code Explorer, folder api

GOAL: the two API files.

FILE 1: api/requirements.txt

Step 1. Right-click the folder api.

Step 2. Click New File...

Step 3. Type requirements.txt and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
fastapi==0.156.8
uvicorn==0.54.0
openai==3.35.1
langsmith==0.14.11
```

Line	Why
fastapi	the web framework
uvicorn	the web server that runs FastAPI
openai	the client that talks to vLLM
langsmith	tracing

FILE 2: api/app.py

Step 6. Right-click the folder api.

Step 7. Click New File...

Step 8. Type app.py and press Enter.

Step 9. Paste the grey box below.

Step 10. Press Ctrl+S.

```
"""MedScriptAI API: one endpoint, three tasks, each routed to its own LoRA adapter inside vLLM."""
import json
import os
import time
import uuid

from fastapi import FastAPI, Header, HTTPException
from langsmith import traceable                  # turns a function call into a LangSmith trace
from langsmith.wrappers import wrap_openai       # also traces every OpenAI-style call inside it
from openai import OpenAI
from pydantic import BaseModel, Field

from common import SYSTEM                        # SAME system prompts as training (no train/serve skew)

VLLM_URL = os.getenv("VLLM_URL", "http://localhost:8000/v1")
API_KEY = os.getenv("API_KEY", "")               # empty = no auth (local only); set it in production
TASK_TO_MODEL = {                                # which adapter answers which task (change without code edits)
    "qa": os.getenv("MODEL_QA", "qa-dpo"),
    "notes": os.getenv("MODEL_NOTES", "notes"),
    "summary": os.getenv("MODEL_SUMMARY", "summary"),
}

llm = wrap_openai(OpenAI(base_url=VLLM_URL, api_key="EMPTY", timeout=120))
app = FastAPI(title="MedScriptAI", version="1.0.0")
```

```
class GenerateRequest(BaseModel):
    task: str = Field(pattern="^(qa|notes|summary)$", examples=["qa"])
    input: str = Field(min_length=1, max_length=20000)
    max_tokens: int = Field(default=700, ge=1, le=2000)
    temperature: float = Field(default=0.0, ge=0.0, le=1.5)


class GenerateResponse(BaseModel):
    request_id: str
    task: str
    model: str
    output: str
    parsed_json: dict | None = None              # filled for task="summary" when the output is valid JSON
    latency_ms: int
    disclaimer: str = "AI-generated. For education/documentation support only. Not medical advice."


@traceable(name="medscriptai.generate", run_type="chain")
def run_generation(task: str, text: str, max_tokens: int, temperature: float) -> dict:
    model = TASK_TO_MODEL[task]
    resp = llm.chat.completions.create(
        model=model,
        messages=[{"role": "system", "content": SYSTEM[task]}, {"role": "user", "content": text}],
        max_tokens=max_tokens,
        temperature=temperature,
    )
    return {"model": model, "output": resp.choices[0].message.content or ""}


@app.get("/health")
def health():
    """Liveness: the API process is up."""
    return {"status": "ok"}


@app.get("/ready")
def ready():
    """Readiness: vLLM answers and every adapter we route to is loaded."""
    try:
        served = {m.id for m in llm.models.list().data}
    except Exception as exc:
        raise HTTPException(503, f"vLLM not reachable: {exc.__class__.__name__}")
    missing = sorted(set(TASK_TO_MODEL.values()) - served)
    if missing:
        raise HTTPException(503, f"adapters not loaded in vLLM: {missing}")
    return {"status": "ready", "models": sorted(served)}


@app.post("/v1/generate", response_model=GenerateResponse)
def generate(req: GenerateRequest, x_api_key: str = Header(default="")):
    if API_KEY and x_api_key != API_KEY:
        raise HTTPException(401, "invalid or missing X-API-Key header")
    start = time.perf_counter()
    try:
        result = run_generation(req.task, req.input, req.max_tokens, req.temperature)
    except Exception as exc:
        raise HTTPException(502, f"model backend error: {exc.__class__.__name__}")
    parsed = None
    if req.task == "summary":
        try:
            parsed = json.loads(result["output"])
        except json.JSONDecodeError:
            parsed = None
    return GenerateResponse(request_id=str(uuid.uuid4()), task=req.task, model=result["model"],
                            output=result["output"], parsed_json=parsed,
                            latency_ms=int((time.perf_counter() - start) * 1000))
```

Line	What it does (plain English)
`from fastapi import FastAPI, Header, HTTPException`	the app object, header reading, HTTP errors
`from langsmith import traceable`	a decorator that records a function call as a LangSmith "run"
`from langsmith.wrappers import wrap_openai`	wraps the OpenAI client so every model call is recorded with tokens and latency

`from pydantic import BaseModel, Field`	typed request/response models with automatic validation and documentation
`from common import SYSTEM`	the SAME system prompts as in training
`VLLM_URL = os.getenv(...)`	where vLLM is; inside Docker Compose it becomes http://vllm:8000/v1
`API_KEY = os.getenv("API_KEY", "")`	shared secret for clients; empty = no check (local tests only)
`TASK_TO_MODEL = {"qa": os.getenv("MODEL_QA", "qa-dpo"), ...}`	which adapter answers which task, changeable without code edits (Part 7 uses this for rollouts)
`llm = wrap_openai(OpenAI(base_url=VLLM_URL, api_key="EMPTY", timeout=120))`	one shared client for all requests; 120 s timeout
`app = FastAPI(title="MedScriptAI", version="1.0.0")`	the web app; title and version appear in /docs
`class GenerateRequest(BaseModel)`	the request schema
`task: str = Field(pattern="^(qa|notes|summary)$")`	only these three values pass; anything else gets HTTP 422 automatically
`input: str = Field(min_length=1, max_length=20000)`	no empty or huge inputs
`max_tokens ... ge=1, le=2000`, `temperature ... ge=0.0, le=1.5`	numeric limits checked automatically
`class GenerateResponse(BaseModel)`	the response schema
`parsed_json: dict | None = None`	for summary: the parsed JSON, or null if the model wrote invalid JSON
`disclaimer: str = "AI-generated ..."`	always tell users this is not medical advice
`@traceable(name="medscriptai.generate", run_type="chain")`	each call becomes a LangSmith trace
`def run_generation(...)`	picks the adapter, sends system + user messages to vLLM, returns model name and text
`@app.get("/health")`	liveness: the process is alive (Docker uses it)
`@app.get("/ready")`	readiness: vLLM answers AND every adapter we route to is loaded; otherwise 503
`served = {m.id for m in llm.models.list().data}`	the names vLLM currently serves
`missing = sorted(set(TASK_TO_MODEL.values()) - served)`	routed adapters that are not loaded
`@app.post("/v1/generate", response_model=GenerateResponse)`	the main endpoint
`x_api_key: str = Header(default="")`	reads the X-API-Key header
`if API_KEY and x_api_key != API_KEY: raise HTTPException(401, ...)`	wrong or missing key → 401
`except Exception: raise HTTPException(502, ...)`	vLLM failed → 502 Bad Gateway, never a stack trace
`if req.task == "summary": parsed = json.loads(...)`	ready-to-use JSON for summary clients
`request_id=str(uuid.uuid4())`, `latency_ms=...`	a unique ID per request and the measured time

CHECKPOINT: Is api shows app.py and requirements.txt

SECTION 6A.9 - Run the API and call it

WHERE: VS Code Terminal (two terminals), folder ~/medscriptai/api, (.venv-eval) active; student vLLM on port 8000

GOAL: your API answers on port 8080.

Step 1. In terminal 1, activate and start the API:

```
cd ~/medscriptai/api && source ../.venv-eval/bin/activate && VLLM_URL=http://localhost:8000/v1 API_KEY=dev-key
PYTHONPATH=../scripts uvicorn app:app --host 0.0.0.0 --port 8080
```

You should see: "Uvicorn running on http://0.0.0.0:8080"

Step 2. Open a second terminal (click + in the terminal panel) and check readiness:

```
curl -s localhost:8080/ready | jq
```

You should see: {"status": "ready", "models": ["base", "notes", "notes-dpo", "qa", "qa-dpo", "summary"]}

Step 3. Call the API:

```
curl -s -X POST localhost:8080/v1/generate -H "Content-Type: application/json" -H "X-API-Key: dev-key" -d
'{"task":"qa","input":"A pregnant woman has cystitis. Which is safest?\n\nOptions:\nA) Doxycycline\nB)
Nitrofurantoin\nC) Ciprofloxacin\nD) Tetracycline"}' | jq
```

You should see: request_id, "task": "qa", "model": "qa-dpo", an output starting with "Answer: B)", latency_ms and the disclaimer.

Step 4. Try a wrong key (should be refused):

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST localhost:8080/v1/generate -H "Content-Type: application/json" -H "X-
API-Key: wrong" -d '{"task":"qa","input":"x"}'
```

You should see: 401

Step 5. Open the interactive documentation: in VS Code click the PORTS tab (next to TERMINAL), click Forward a Port, type 8080, press Enter, then open http://localhost:8080/docs in your laptop browser.

You should see: Swagger UI with GET /health, GET /ready, POST /v1/generate; you can click "Try it out".

Step 6. Stop the API in terminal 1 with Ctrl+C.

Piece	What it does
PYTHONPATH=../scripts	lets app.py find common.py
uvicorn app:app	run the object "app" from the file app.py
--host 0.0.0.0 --port 8080	listen on all network interfaces, port 8080

CHECKPOINT: Step 3 returned "model": "qa-dpo" and Step 4 returned 401.

SECTION 6A.10 - Create the tests and run them without a GPU

BEFORE YOU START: why a fake server. Tests must run anywhere (your laptop, a CI server) without a GPU. tests/mock_openai.py is a tiny fake vLLM that answers in the same SHAPE as the real one. We used exactly these files to test the pipelines and the API while writing this manual.

WHERE: VS Code Explorer (create), then VS Code Terminal with (.venv-eval)

GOAL: 5 passing API tests.

FILE 1: tests/mock_openai.py

Step 1. Right-click the folder tests.

Step 2. Click New File...

Step 3. Type mock_openai.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""A fake OpenAI-compatible server (like vLLM) for testing pipelines and the API WITHOUT a GPU.

Run:  uvicorn tests.mock_openai:app --port 8000
It answers /v1/models and /v1/chat/completions with small, predictable replies.
"""
import re
import time
from fastapi import FastAPI, Request

app = FastAPI()
MODELS = ["base", "qa", "notes", "summary", "qa-dpo", "notes-dpo", "teacher"]


def reply_for(model, text):
    """Pick a fake answer that has the same SHAPE as a real one."""
    n_texts = len(re.findall(r"<text \d+>", text))
    if n_texts:                                          # UltraFeedback judge request
        blocks = [f"#### Output for Text {i + 1}\nRating: {5 - i if i < 4 else 1}\nRationale: mock rationale."
                  for i in range(n_texts)]
        return "\n\n".join(blocks)
    if "The correct answer is:" in text:                 # teacher explanation request (Part 5)
        gold = text.split("The correct answer is:", 1)[1].splitlines()[0].strip()
        return (f"{gold} is correct because it best fits the key findings in the stem. "
                "The other options do not explain the presentation as well. (mock teacher)")
    if '"score": <1-5>' in text:                         # 32_judge.py --mode score
        return '{"score": 4, "reason": "mock grade"}'
```

```
    if '"winner"' in text:                                  # 32_judge.py --mode pairwise
        answer_a = text.split("ANSWER A:", 1)[1].split("ANSWER B:", 1)[0]
        return '{"winner": "%s", "reason": "mock"}' % ("A" if "dpo" in answer_a else "B")
    if "Return ONLY valid JSON" in text:                    # summary adapter
        return '{"visit motivation": "mock", "patient information": {"age": "40"}}'
    if "clinical case note" in text:                        # notes adapter
        return "A 40-year-old patient presented with mock symptoms."
    letter = "B" if model.startswith("qa") else "A"         # qa adapters / base / teacher
    return f"Answer: {letter}) mock option\nExplanation: mock explanation from {model}."


@app.get("/v1/models")
def models():
    return {"object": "list", "data": [{"id": m, "object": "model"} for m in MODELS]}


@app.get("/health")
def health():
    return {"status": "ok"}


@app.post("/v1/chat/completions")
async def chat(request: Request):
    body = await request.json()
    text = "\n".join(str(m.get("content", "")) for m in body.get("messages", []))
    content = reply_for(body.get("model", "base"), text)
    return {
        "id": "mock-1", "object": "chat.completion", "created": int(time.time()), "model": body.get("model"),
        "choices": [{"index": 0, "finish_reason": "stop",
                     "message": {"role": "assistant", "content": content}}],
        "usage": {"prompt_tokens": len(text) // 4, "completion_tokens": len(content) // 4,
                  "total_tokens": (len(text) + len(content)) // 4},
    }
```

Line	What it does (plain English)
`MODELS = [...]`	the names /v1/models reports, like a real vLLM with adapters
`def reply_for(model, text)`	returns an answer with the right shape for each kind of request (judge ratings, explanation, JSON, note, "Answer: X)")
`re.findall(r"<text \d+>", text)`	detects UltraFeedback rating prompts
`@app.post("/v1/chat/completions")`	the OpenAI chat endpoint, returning choices[0].message.content and usage

FILE 2: tests/test_api.py

Step 6. Right-click the folder tests.

Step 7. Click New File...

Step 8. Type test_api.py and press Enter.

Step 9. Paste the grey box below.

Step 10. Press Ctrl+S.

```
"""API tests that run on any laptop/CPU: start the mock first (uvicorn tests.mock_openai:app --port 8000)."""
import os
import sys

sys.path[:0] = ["api", "scripts"]                # import app.py and common.py
os.environ.setdefault("VLLM_URL", "http://localhost:8000/v1")
os.environ["API_KEY"] = "test-key"

from fastapi.testclient import TestClient          # noqa: E402
import app as api                                  # noqa: E402

client = TestClient(api.app)
HEADERS = {"X-API-Key": "test-key"}


def test_health():
    assert client.get("/health").json() == {"status": "ok"}


def test_ready_lists_adapters():
    body = client.get("/ready").json()
```

```
    assert body["status"] == "ready" and "qa-dpo" in body["models"]


def test_qa_routes_to_dpo_adapter():
    r = client.post("/v1/generate", headers=HEADERS, json={"task": "qa", "input": "Q?\n\nOptions:\nA) x\nB) y"})
    assert r.status_code == 200
    assert r.json()["model"] == "qa-dpo" and r.json()["output"].startswith("Answer:")


def test_summary_returns_parsed_json():
    r = client.post("/v1/generate", headers=HEADERS, json={"task": "summary", "input": "A 40-year-old ..."})
    assert r.json()["parsed_json"]["patient information"]["age"] == "40"


def test_auth_and_validation():
    assert client.post("/v1/generate", json={"task": "qa", "input": "x"}).status_code == 401
    assert client.post("/v1/generate", headers=HEADERS, json={"task": "bad", "input": "x"}).status_code == 422
```

Line	What it does (plain English)
`sys.path[:0] = ["api", "scripts"]`	lets the test import app.py and common.py
`os.environ["API_KEY"] = "test-key"`	set BEFORE importing app (app reads it at import time)
`client = TestClient(api.app)`	calls the app in-process, no real network
the five `test_...` functions	health, readiness, qa routed to qa-dpo, summary JSON parsed, auth (401) and validation (422)

Step 11. Start the fake server on port 8009 (8000 is your real vLLM):

```
cd ~/medscriptai && source .venv-eval/bin/activate && (uvicorn tests.mock_openai:app --port 8009 > /tmp/mock.log 2>&1 &)
&& sleep 2 && curl -s localhost:8009/health
```

You should see: {"status":"ok"}

Step 12. Run the tests:

```
VLLM_URL=http://localhost:8009/v1 python -m pytest -q tests/test_api.py
```

You should see: 5 passed

CHECKPOINT: 5 passed.

SECTION 6A.11 - LangSmith tracing

WHERE: web browser (smith.langchain.com), then VS Code Terminal

GOAL: every API request visible as a trace.

Step 1. Open https://smith.langchain.com and sign in (or sign up, free Developer plan).

Step 2. Click Settings (gear icon, bottom left).

Step 3. Click API Keys, then Create API Key, then Create.

Step 4. Copy the key (starts with lsv2_) into secrets.txt.

Step 5. Save it on the GPU computer (replace lsv2_YOUR_KEY):

```
echo 'export LANGSMITH_API_KEY=lsv2_YOUR_KEY' >> ~/.bashrc && source ~/.bashrc
```

Step 6. Start the API with tracing on:

```
cd ~/medscriptai/api && source ../.venv-eval/bin/activate && LANGSMITH_TRACING=true LANGSMITH_PROJECT=medscriptai
VLLM_URL=http://localhost:8000/v1 API_KEY=dev-key PYTHONPATH=../scripts uvicorn app:app --port 8080
```

Step 7. In the second terminal, repeat the curl from Section 6A.9 Step 3 three times.

Step 8. In LangSmith click Projects (left menu), then medscriptai.

You should see: three runs named medscriptai.generate; click one to see the input, the output, a child ChatOpenAI run with model qa-dpo, tokens and latency.

Step 9. Stop the API with Ctrl+C.

Variable	What it does
LANGSMITH_TRACING=true	turns tracing on (we checked the SDK source: it reads LANGSMITH_TRACING or the older LANGCHAIN_TRACING_V2)

LANGSMITH_API_KEY	authenticates the SDK
LANGSMITH_PROJECT	the project name (created automatically)
LANGSMITH_ENDPOINT	only for EU accounts: https://eu.api.smith.langchain.com

Watch out: traces contain the full input text. With real patient data that is a privacy problem; here the data is public.

CHECKPOINT: the medscriptai project shows your runs.

SECTION 6A.12 - Commit

Step 1. Run:

```
cd ~/medscriptai && git add . && git commit -m "Part 6A: DPO, multi-LoRA serving, API and tests"
```

CHECKPOINT: git log --oneline shows the new commit.

PART 6A RESULT CHECK

Check	Command	Expected
pairs	wc -l data/pref/qa_pairs.jsonl data/pref/notes_pairs.jsonl	hundreds each
DPO adapters	ls outputs/adapters	dpo-notes dpo-qa (plus the three sft-*)
DPO learned	cat outputs/adapters/dpo-qa/dpo_eval.json	eval_rewards/accuracies above 0.5
five adapters served	curl -s localhost:8000/v1/models | jq -r '.data[].id'	base qa notes summary qa-dpo notes-dpo
DPO vs SFT judged	ls outputs/reports/judge_*pairwise*	two files
API	curl localhost:8080/ready (API running)	"status": "ready"
tests	pytest -q tests/test_api.py (mock on 8009)	5 passed
traces	LangSmith > Projects > medscriptai	your runs

6A.13 Common mistake at this level

The mistake: calling vLLM directly from clients, leaving runtime LoRA loading open to the internet, or creating a new model client per request.

How to spot it: only the API (port 8080) should ever be reachable from outside; Part 6B's Compose file publishes 8080 only. In app.py the client is created once at the top, not inside the endpoint.

PART 6B - ADVANCED: Docker image, Compose stack, and cloud deployment (FakeCloud and AWS side by side)

BEFORE YOU START: the words of this part

Word	Plain meaning
Docker image	a frozen package: an operating system layer + Python + your code + its libraries. Built once, runs the same everywhere
container	a running copy of an image
Dockerfile	the recipe to build an image, step by step
registry	an online store for images (Docker Hub is public; AWS ECR is your private one)
tag	a version label of an image, e.g. medscriptai-api:v1
Docker Compose	starts several containers together (vLLM + API) from one YAML file
ECR	Elastic Container Registry: AWS's private image registry
S3	AWS file storage (buckets)
IAM role	a set of permissions a machine can use without any password on its disk
Secrets Manager	AWS's safe for passwords and tokens
CloudWatch	AWS's logs and alarms
security group	AWS's firewall: which ports are open, to whom
EC2	AWS virtual machines; g6.xlarge = 1x NVIDIA L4 24 GB GPU, about US$0.805 per hour
AMI	the disk image an EC2 machine starts from; we use AWS's Deep Learning AMI (Ubuntu 24.04, NVIDIA driver 595, Docker included)
user data	a script the EC2 machine runs automatically on its first boot

The plan of this part:

```
6B.1-6B.3  build and test the API Docker image on Hyperstack
6B.4-6B.6  create the Compose files and run vLLM + API together on Hyperstack
6B.7       the boot script the AWS machine runs on first start
6B.8       deploy step by step: ECR -> S3 -> Secrets -> IAM -> CloudWatch -> firewall -> EC2 (two columns)
6B.9-6B.10 the same deployment as one script, and the teardown script
6B.11      optional: SageMaker objects
```

SECTION 6B.1 - Create api/Dockerfile

WHERE: VS Code Explorer, folder api

GOAL: the recipe for the API image.

Step 1. Right-click the folder api.

Step 2. Click New File...

Step 3. Type Dockerfile (no extension, capital D) and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
# Small CPU-only image for the API. The GPU work happens in the separate vLLM container.
# Build from the REPO ROOT:  docker build -f api/Dockerfile -t medscriptai-api:v1 .
# Official Python image via the AWS ECR Public mirror (avoids Docker Hub pull rate limits).
# Switch registry if one is rate-limited:  --build-arg BASE_IMAGE=python:3.12-slim
ARG BASE_IMAGE=public.ecr.aws/docker/library/python:3.12-slim
FROM ${BASE_IMAGE}

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app
COPY api/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY scripts/common.py api/app.py ./

RUN useradd --create-home appuser
USER appuser

EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')" || exit 1

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080", "--workers", "2"]
```

Line	What it does (plain English)
`ARG BASE_IMAGE=public.ecr.aws/docker/library/python:3.12-slim`	the official Python image through AWS's ECR Public mirror; we hit Docker Hub's anonymous download limit (HTTP 429) while testing, so this avoids it. Switch with --build-arg BASE_IMAGE=python:3.12-slim
`FROM ${BASE_IMAGE}`	start from a small Debian Linux with Python 3.12
`ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1`	no .pyc files; log lines appear immediately
`WORKDIR /app`	all following commands run in /app inside the image
`COPY api/requirements.txt .` then `RUN pip install --no-cache-dir -r requirements.txt`	install the pinned libraries in their own layer, so code changes rebuild fast
`COPY scripts/common.py api/app.py ./`	only the two files the API needs; this is why you build from the project root
`RUN useradd --create-home appuser` and `USER appuser`	the container does not run as root (security)
`EXPOSE 8080`	documents the port
`HEALTHCHECK ... urlopen('http://localhost:8080/health')`	Docker marks the container healthy or unhealthy
`CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080", "--workers", "2"]`	start the API with 2 worker processes

CHECKPOINT: ls api shows Dockerfile, app.py, requirements.txt

SECTION 6B.2 - Build the image

WHERE: VS Code Terminal, folder ~/medscriptai (the project ROOT)

GOAL: an image called medscriptai-api:v1

Step 1. Build:

```
cd ~/medscriptai && docker build -f api/Dockerfile -t medscriptai-api:v1 .
```

You should see: build steps [1/7] ... [7/7] and "naming to docker.io/library/medscriptai-api:v1".

Step 2. List it:

```
docker images medscriptai-api
```

You should see: medscriptai-api v1 ... about 290MB

Piece	What it does
-f api/Dockerfile	which recipe file
-t medscriptai-api:v1	the image name and tag
. (the final dot)	the build context = the current folder (the project root), so COPY scripts/common.py works
If you see...	Do this
429 Too Many Requests	wait 10 minutes, or add --build-arg BASE_IMAGE=python:3.12-slim
COPY failed: file not found	you are not in ~/medscriptai: run cd ~/medscriptai and build again

CHECKPOINT: docker images shows medscriptai-api v1.

SECTION 6B.3 - Test the container against the running vLLM

WHERE: VS Code Terminal; student vLLM running on port 8000 (Section 6A.6)

GOAL: the containerised API works and is healthy.

Step 1. Run it:

```
docker run -d --name api-test --network host -e VLLM_URL=http://localhost:8000/v1 -e API_KEY=dev-key medscriptai-api:v1
```

Step 2. Check readiness (wait 5 seconds first):

```
sleep 5 && curl -s localhost:8080/ready | jq -r .status
```

You should see: ready

Step 3. Check Docker's health status after 30 seconds:

```
sleep 30 && docker inspect --format '{{.State.Health.Status}} user={{.Config.User}}' api-test
```

You should see: healthy user=appuser (this exact result was produced when we tested the image)

Step 4. Remove the test container:

```
docker rm -f api-test
```

CHECKPOINT: you saw ready and healthy.

SECTION 6B.4 - Create the Compose files

WHERE: VS Code Explorer, folder deploy

GOAL: docker-compose.yml, compose.aws.yml and .env.example

FILE 1: deploy/docker-compose.yml

Step 1. Right-click the folder deploy.

Step 2. Click New File...

Step 3. Type docker-compose.yml and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
# The full MedScriptAI stack on ONE GPU machine (Hyperstack VM or AWS EC2 g6).
# Start:  docker compose --env-file .env up -d        (run inside the deploy/ folder)
services:
  vllm:
    image: ${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0}
    command: >
      --model unsloth/Llama-3.1-8B-Instruct --served-model-name base --dtype bfloat16
      --max-model-len 4096 --gpu-memory-utilization 0.90
      --enable-lora --max-loras 4 --max-lora-rank 16 --max-cpu-loras 8
      --lora-modules qa=/adapters/sft-qa notes=/adapters/sft-notes summary=/adapters/sft-summary
      qa-dpo=/adapters/dpo-qa
    environment:
      HF_TOKEN: ${HF_TOKEN:-}
      VLLM_ENABLE_CUDA_COMPATIBILITY: ${VLLM_ENABLE_CUDA_COMPATIBILITY:-0}
    volumes:
      - ${ADAPTERS_DIR:?set ADAPTERS_DIR in .env}:/adapters:ro
      - ${HF_HOME:?set HF_HOME in .env}:/root/.cache/huggingface
    ipc: host
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    healthcheck:
      test: ["CMD", "python3", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 30s
      timeout: 5s
      retries: 40
      start_period: 120s
    restart: unless-stopped
```

```
    # No "ports:" on purpose: vLLM is reachable ONLY by the api container, never from the internet.

  api:
    image: ${API_IMAGE:-medscriptai-api:v1}
    environment:
      VLLM_URL: http://vllm:8000/v1
      API_KEY: ${API_KEY:?set API_KEY in .env}
      MODEL_QA: ${MODEL_QA:-qa-dpo}
      MODEL_NOTES: ${MODEL_NOTES:-notes}
      MODEL_SUMMARY: ${MODEL_SUMMARY:-summary}
      LANGSMITH_TRACING: ${LANGSMITH_TRACING:-false}
      LANGSMITH_API_KEY: ${LANGSMITH_API_KEY:-}
      LANGSMITH_PROJECT: ${LANGSMITH_PROJECT:-medscriptai}
      LANGSMITH_ENDPOINT: ${LANGSMITH_ENDPOINT:-https://api.smith.langchain.com}
    ports:
      - "8080:8080"
    depends_on:
      vllm:
        condition: service_healthy
    restart: unless-stopped
```

Line	What it does (plain English)
`services: vllm: ... api: ...`	two containers that Compose starts and connects
`image: ${VLLM_IMAGE:-vllm/vllm-openai:v0.32.0}`	the vLLM image; .env sets the -cu129 build for your driver
`command: > --model ... --lora-modules ...`	the same arguments as start_vllm.sh ("> " folds the lines into one)
`${ADAPTERS_DIR:?set ADAPTERS_DIR in .env}`	Compose STOPS with this message if the variable is missing
`deploy.resources.reservations.devices: driver nvidia, count all, capabilities [gpu]`	give the container the GPU
`healthcheck: ... urlopen('http://localhost:8000/health')`, `start_period: 120s`	vLLM is healthy when /health answers; 2 minutes grace to load
`restart: unless-stopped`	restart after crashes and reboots
no `ports:` under vllm	vLLM is NOT reachable from outside, only by the api container (by the name vllm)
`VLLM_URL: http://vllm:8000/v1`	Compose's internal DNS: a service name works as a host name
`ports: "8080:8080"`	only the API is published
`depends_on: vllm: condition: service_healthy`	start the API only after vLLM is healthy

FILE 2: deploy/compose.aws.yml (extra settings used only on AWS)

Step 6. Right-click the folder deploy.

Step 7. Click New File...

Step 8. Type compose.aws.yml and press Enter.

Step 9. Paste the grey box below.

Step 10. Press Ctrl+S.

```
# Extra settings used ONLY on AWS EC2: send container logs to CloudWatch Logs.
# docker compose -f docker-compose.yml -f compose.aws.yml --env-file .env up -d
services:
  api:
    logging:
      driver: awslogs
      options:
        awslogs-region: ${AWS_REGION:-us-east-1}
        awslogs-group: /medscriptai/api
        awslogs-create-group: "true"
  vllm:
    logging:
      driver: awslogs
      options:
        awslogs-region: ${AWS_REGION:-us-east-1}
        awslogs-group: /medscriptai/vllm
        awslogs-create-group: "true"
```

Line	What it does (plain English)
`logging: driver: awslogs`	Docker sends container logs to CloudWatch Logs

`awslogs-group: /medscriptai/api`	the log group name
`awslogs-create-group: "true"`	create the group if missing

FILE 3: deploy/.env.example (a template; the real .env is never committed)

Step 11. Right-click the folder deploy.

Step 12. Click New File...

Step 13. Type .env.example and press Enter.

Step 14. Paste the grey box below.

Step 15. Press Ctrl+S.

```
# Copy to deploy/.env and fill in.  NEVER commit the real .env (it is in .gitignore).
ADAPTERS_DIR=/home/ubuntu/medscriptai/outputs/adapters
HF_HOME=/ephemeral/hf-cache
HF_TOKEN=hf_REPLACE_ME
API_KEY=REPLACE_WITH_A_LONG_RANDOM_STRING
VLLM_IMAGE=vllm/vllm-openai:v0.32.0-cu129
VLLM_ENABLE_CUDA_COMPATIBILITY=1
API_IMAGE=medscriptai-api:v1
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=lsv2_REPLACE_ME
LANGSMITH_PROJECT=medscriptai
AWS_REGION=us-east-1
```

CHECKPOINT: ls -a deploy shows .env.example, compose.aws.yml, docker-compose.yml

SECTION 6B.5 - Fill in deploy/.env

WHERE: VS Code Terminal and editor

GOAL: your real settings file.

Step 1. Copy the template:

```
cd ~/medscriptai && cp deploy/.env.example deploy/.env
```

Step 2. Make a long random API key:

```
openssl rand -hex 24
```

Step 3. In the Explorer, click deploy > .env to open it.

Step 4. Replace hf_REPLACE_ME with your HF token, REPLACE_WITH_A_LONG_RANDOM_STRING with the key from Step 2, and lsv2_REPLACE_ME with your LangSmith key.

Step 5. Press Ctrl+S.

Line in .env	Value	Why
ADAPTERS_DIR	/home/ubuntu/medscriptai/outputs/adapters	where your adapters are
HF_HOME	/ephemeral/hf-cache	the model cache from Section 2B.4
VLLM_IMAGE	vllm/vllm-openai:v0.32.0-cu129	matches your R570 driver
VLLM_ENABLE_CUDA_COMPATIBILITY	1	needed with R570 + the cu129 image
API_IMAGE	medscriptai-api:v1	your image

Watch out: .env is in .gitignore; never commit it.

CHECKPOINT: grep -c REPLACE deploy/.env prints 0.

SECTION 6B.6 - Run the full stack with one command

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: vLLM + API started together by Compose.

Step 1. Stop the standalone servers (Compose starts its own vLLM):

```
docker rm -f medscriptai-vllm medscriptai-teacher
```

Step 2. Start the stack:

```
cd ~/medscriptai/deploy && docker compose --env-file .env up -d && cd ..
```

Step 3. Watch the status (2-4 minutes until healthy; Ctrl+C to stop watching):

```
watch -n 10 docker compose -f deploy/docker-compose.yml ps
```

You should see: vllm ... (healthy) and api ... Up.

Step 4. Call the summary task through the stack:

```
curl -s -X POST localhost:8080/v1/generate -H "Content-Type: application/json" -H "X-API-Key: $(grep ^API_KEY
deploy/.env | cut -d= -f2)" -d '{"task":"summary","input":"A 54-year-old man with type 2 diabetes presented with chest
pain for 2 hours. ECG showed ST elevation. He underwent primary PCI."}' | jq
```

You should see: "model": "summary" and parsed_json with fields such as "patient information".

Step 5. Prove vLLM is NOT exposed (this must fail):

```
curl -s --max-time 3 localhost:8000/v1/models || echo "port 8000 closed - correct"
```

You should see: port 8000 closed - correct

CHECKPOINT: Step 4 returned parsed JSON and Step 5 printed "closed - correct".

If not: docker compose -f deploy/docker-compose.yml logs vllm | tail -50 shows vLLM's error.

SECTION 6B.7 - Create deploy/ec2_user_data.sh (what the AWS machine does on first boot)

WHERE: VS Code Explorer, folder deploy

GOAL: the boot script.

Step 1. Right-click the folder deploy.

Step 2. Click New File...

Step 3. Type ec2_user_data.sh and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
#!/bin/bash
# Runs ONCE, as root, the first time the EC2 machine boots (cloud-init "user data").
# deploy.sh replaces the __PLACEHOLDERS__ before sending it to AWS.
set -euxo pipefail                                      # -x prints every command into the boot log
REGION=__REGION__
REGISTRY=__REGISTRY__
BUCKET=__BUCKET__
TAG=__TAG__
APP_DIR=/opt/medscriptai
mkdir -p $APP_DIR/adapters $APP_DIR/hf-cache

# 1) Adapters from S3 (the machine's IAM role gives read access; no keys on the box).
aws s3 sync "s3://$BUCKET/adapters" $APP_DIR/adapters --region "$REGION"

# 2) Secrets from Secrets Manager into a root-only .env file.
secret() { aws secretsmanager get-secret-value --region "$REGION" --secret-id "medscriptai/$1" \
           --query SecretString --output text 2>/dev/null || true; }
cat > $APP_DIR/.env <<EOF
ADAPTERS_DIR=$APP_DIR/adapters
HF_HOME=$APP_DIR/hf-cache
HF_TOKEN=$(secret HF_TOKEN)
API_KEY=$(secret API_KEY)
LANGSMITH_API_KEY=$(secret LANGSMITH_API_KEY)
LANGSMITH_TRACING=true
VLLM_IMAGE=vllm/vllm-openai:v0.32.0
VLLM_ENABLE_CUDA_COMPATIBILITY=0
API_IMAGE=$REGISTRY/medscriptai-api:$TAG
AWS_REGION=$REGION
EOF
chmod 600 $APP_DIR/.env

# 3) Log Docker in to ECR and pull our API image.
aws ecr get-login-password --region "$REGION" | docker login --username AWS --password-stdin "$REGISTRY"
```

```
# 4) The compose files that deploy.sh uploaded to S3.
aws s3 cp "s3://$BUCKET/deploy/docker-compose.yml" $APP_DIR/ --region "$REGION"
aws s3 cp "s3://$BUCKET/deploy/compose.aws.yml" $APP_DIR/ --region "$REGION"

# 5) Start vLLM + API. vLLM downloads the 16 GB base model on first start.
cd $APP_DIR
docker compose -f docker-compose.yml -f compose.aws.yml --env-file .env up -d
```

Line	What it does (plain English)
`set -euxo pipefail`	stop on errors and print every command into /var/log/cloud-init-output.log
`REGION=__REGION__` etc.	placeholders that row 22 of Section 6B.8 (or deploy.sh) replaces with real values
`aws s3 sync "s3://$BUCKET/adapters" ...`	download the adapters using the machine's role (no keys on the box)
`secret() { aws secretsmanager get-secret-value ... }`	read one secret
`cat > $APP_DIR/.env <<EOF ... EOF` and `chmod 600`	write the settings file, readable only by root
`VLLM_IMAGE=vllm/vllm-openai:v0.32.0` and `VLLM_ENABLE_CUDA_COMPATIBILITY=0`	the AWS Deep Learning AMI has driver 595 / CUDA 13.2, so the default CUDA 13 image runs natively
`aws ecr get-login-password ... | docker login ...`	let Docker pull your private image
`aws s3 cp ".../docker-compose.yml" ...`	the same Compose files you tested on Hyperstack
`docker compose ... up -d`	start vLLM + API with CloudWatch logging

CHECKPOINT: ls deploy shows ec2_user_data.sh

SECTION 6B.8 - Deploy step by step (registry, storage, secrets, IAM, logs, firewall, GPU machine)

WHERE: VS Code Terminal on the Hyperstack VM, folder ~/medscriptai

GOAL: the same production deployment done twice: first FREE on FakeCloud, then for real on AWS.

Before you start (both columns): FakeCloud running (curl localhost:4566/_fakecloud/health); export API_KEY=$(grep ^API_KEY deploy/.env | cut -d= -f2); HF_TOKEN and LANGSMITH_API_KEY set (echo ${HF_TOKEN:0:3}). Blue column only: GPU quota approved (Section 2B.12).

Step 1. Write the trust policy file (WHO may use the role: EC2 machines):

```
cat > /tmp/medscriptai-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
{"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
```

Step 2. Do the rows of the table below one by one in your column. Then come back for Step 3.

Step	FakeCloud (free practice)	Real AWS (same step)
1 Target	`export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1`	`unset AWS_ENDPOINT_URL && export AWS_DEFAULT_REGION=us-east-1`
2 Who am I	`export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text) && echo $ACCOUNT_ID`	same (prints your real 12-digit ID)
3 Registry address	`export REGISTRY=localhost:4566`	`export REGISTRY=$ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com`
4 Create the image repository	`aws ecr create-repository --repository-name medscriptai-api --image-scanning-configuration scanOnPush=true`	same
5 Log Docker in	`aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY`	same; you should see Login Succeeded
6 Tag the image	`docker tag medscriptai-api:v1 $REGISTRY/medscriptai-api:v1`	same
7 Push	`docker push $REGISTRY/medscriptai-api:v1`	same (about 290 MB upload)
8 Verify	`aws ecr describe-images --repository-name medscriptai-api --query 'imageDetails[].imageTags'`	same; console: ECR > Repositories > medscriptai-api

9 Bucket	`export BUCKET=medscriptai-artifacts-$ACCOUNT_ID-us-east-1 && aws s3 mb s3://$BUCKET`	same (ignore "already exists" if you made it in Part 4)
10 Adapters + compose files to S3	`aws s3 sync outputs/adapters s3://$BUCKET/adapters --exclude "*/checkpoint-*" && aws s3 cp deploy/docker-compose.yml s3://$BUCKET/deploy/ && aws s3 cp deploy/compose.aws.yml s3://$BUCKET/deploy/`	same
11 Secrets	`for n in HF_TOKEN LANGSMITH_API_KEY API_KEY; do aws secretsmanager create-secret --name medscriptai/$n --secret-string "${!n}"; done`	same
12 IAM role	`aws iam create-role --role-name medscriptai-ec2-role --assume-role-policy-document file:///tmp/medscriptai-trust.json`	same
13 Permissions policy file	write it with the command in Step 3 below, then come back	same
14 Attach the policy	`aws iam put-role-policy --role-name medscriptai-ec2-role --policy-name medscriptai-runtime --policy-document file:///tmp/medscriptai-policy.json`	same
15 Instance profile	`aws iam create-instance-profile --instance-profile-name medscriptai-ec2-profile && aws iam add-role-to-instance-profile --instance-profile-name medscriptai-ec2-profile --role-name medscriptai-ec2-role`	same, then `sleep 10` (real IAM needs a few seconds)
16 Logs	`aws logs create-log-group --log-group-name /medscriptai/api && aws logs put-retention-policy --log-group-name /medscriptai/api --retention-in-days 14`	same
17 Alarm	`aws cloudwatch put-metric-alarm --alarm-name medscriptai-cpu-high --namespace AWS/EC2 --metric-name CPUUtilization --statistic Average --period 300 --evaluation-periods 2 --threshold 90 --comparison-operator GreaterThanThreshold`	same
18 Firewall	`export SG_ID=$(aws ec2 create-security-group --group-name medscriptai-sg --description "MedScriptAI API" --query GroupId --output text)`	same (uses your default VPC)
19 Allow only your IP	`export MY_IP=$(curl -s https://checkip.amazonaws.com) && for p in 22 8080; do aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port $p --cidr $MY_IP/32; done`	same. MY_IP is the Hyperstack VM's IP; to call the API from your laptop too, repeat with your laptop's IP
20 SSH key for EC2	`aws ec2 create-key-pair --key-name medscriptai-key --query KeyMaterial --output text > ~/.ssh/medscriptai-key.pem && chmod 400 ~/.ssh/medscriptai-key.pem`	same
21 Machine image (AMI)	`export AMI=$(aws ec2 describe-images --owners amazon --query 'Images[0].ImageId' --output text)` (FakeCloud's demo AMIs)	`export AMI=$(aws ec2 describe-images --owners amazon --filters "Name=name,Values=Deep Learning Base OSS Nvidia Driver GPU AMI (Ubuntu 24.04)*" "Name=state,Values=available" --query 'sort_by(Images,&CreationDate)[-1].ImageId' --output text)`
22 Boot script	`sed -e "s|__REGION__|us-east-1|g" -e "s|__REGISTRY__|$REGISTRY\|g" -e "s\|__BUCKET__\|$BUCKET|g" -e "s|__TAG__|v1|g" deploy/ec2_user_data.sh > /tmp/user-data.sh`	same
23 Launch	`aws ec2 run-instances --image-id $AMI --instance-type g6.xlarge --key-name medscriptai-key --security-group-ids $SG_ID --iam-instance-profile Name=medscriptai-ec2-profile --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":150,"VolumeType":"gp3"}}]' --user-data file:///tmp/user-data.sh --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=medscriptai-gpu}]' --query 'Instances[0].InstanceId' --output text`	same. BILLING STARTS NOW (about US$0.805/h + disk)
24 Get its IP	`aws ec2 describe-instances --filters Name=tag:Name,Values=medscriptai-gpu --query 'Reservations[].Instances[].[InstanceId,State.Name,PublicIpAddress]' --output text`	same
You should see	an i-... id, pending/running, and a fake IP such as 52.0.0.10. FakeCloud starts a small container to represent the machine; there is no GPU and no model	a real public IP; the boot script needs 10-15 minutes (it downloads about 30 GB)

25 Watch the boot	not applicable	`ssh -i ~/.ssh/medscriptai-key.pem ubuntu@EC2_IP` then `sudo tail -f /var/log/cloud-init-output.log` until docker compose reports Started
26 Test the API	not applicable (no model behind FakeCloud)	`curl -s http://EC2_IP:8080/ready` then the /v1/generate call from Section 6A.9 with your API_KEY
27 Read the logs	`aws logs describe-log-groups --query 'logGroups[].logGroupName'`	console: CloudWatch > Log groups > /medscriptai/api, or `aws logs tail /medscriptai/api --follow`
28 TEAR DOWN	do Section 6B.10 (it creates teardown.sh and runs it on FakeCloud)	after Section 6B.10: `TARGET=aws bash deploy/teardown.sh`, then check EC2 > Instances shows terminated

Step 3. The permissions policy for table row 13 (least privilege: only our repo, our bucket, our secrets, and logs):

```
cat > /tmp/medscriptai-policy.json <<EOF
{"Version":"2012-10-17","Statement":[
 {"Effect":"Allow","Action":["ecr:GetAuthorizationToken"],"Resource":"*"},
 {"Effect":"Allow","Action":["ecr:BatchGetImage","ecr:GetDownloadUrlForLayer"],"Resource":"arn:aws:ecr:us-east-
1:$ACCOUNT_ID:repository/medscriptai-api"},
 {"Effect":"Allow","Action":["s3:GetObject","s3:ListBucket"],"Resource":
["arn:aws:s3:::$BUCKET","arn:aws:s3:::$BUCKET/*"]},
 {"Effect":"Allow","Action":["secretsmanager:GetSecretValue"],"Resource":"arn:aws:secretsmanager:us-east-
1:$ACCOUNT_ID:secret:medscriptai/*"},
 {"Effect":"Allow","Action":["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"],"Resource":"*"}]}
EOF
```

New command pieces:

Piece	What it does
`get-login-password | docker login --password-stdin`	ECR gives a 12-hour token; piping it avoids typing it or saving it in history
`docker tag LOCAL REGISTRY/REPO:TAG`	a second name for the same image that says where to push it
`"${!n}"`	bash: the value of the variable whose NAME is in n
trust policy	WHO may use the role (EC2)
permissions policy	WHAT the role may do; every Resource is limited to MedScriptAI's own repo, bucket and secrets
instance profile	attaches the role to a machine; programs on it get temporary credentials automatically
`--cidr $MY_IP/32`	/32 = exactly one IP address
`VolumeSize 150, gp3`	150 GB SSD: the vLLM image (about 11 GB compressed) + the model (16 GB) need room
`--user-data file://...`	the boot script, run once at first start

Placeholder	Replace with	Example
EC2_IP	the PublicIpAddress from row 24	3.91.20.14

> **Watch out:** the most expensive mistake in this manual is forgetting row 28 on real AWS. Set a phone reminder when you launch.

CHECKPOINT (FakeCloud): rows 1-24 ran without errors. CHECKPOINT (AWS): row 26 returns an answer from your adapter.

> **If not:** "VcpuLimitExceeded" = GPU quota not approved yet; "InvalidAMIID" = run row 21 in us-east-1; no answer after 20 minutes = read /var/log/cloud-init-output.log on the machine.

SECTION 6B.9 - Create deploy/deploy.sh (the whole table as one script)

WHERE: VS Code Explorer, folder deploy

GOAL: one command that repeats Section 6B.8 rows 1-24 for either target.

Step 1. Right-click the folder deploy.

Step 2. Click New File...

Step 3. Type deploy.sh and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
#!/usr/bin/env bash
# One script, two targets:
#   TARGET=fakecloud bash deploy/deploy.sh   -> free local AWS emulator (localhost:4566)
#   TARGET=aws       bash deploy/deploy.sh   -> your real AWS account (costs money!)
# Run from the repo folder after: the API image is built and outputs/adapters/ exists.
set -euo pipefail

TARGET="${TARGET:-fakecloud}"
export AWS_REGION="${AWS_REGION:-us-east-1}"
export AWS_DEFAULT_REGION="$AWS_REGION"
APP=medscriptai
TAG="${TAG:-v1}"
MY_IP="${MY_IP:-$(curl -s https://checkip.amazonaws.com)}"   # only YOUR IP may reach SSH and the API

if [ "$TARGET" = "fakecloud" ]; then
  export AWS_ENDPOINT_URL=http://localhost:4566             # every aws command now talks to FakeCloud
  export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test  # FakeCloud accepts any keys
fi

ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
if [ "$TARGET" = "fakecloud" ]; then
  REGISTRY=localhost:4566
else
  REGISTRY="$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com"
fi
BUCKET="$APP-artifacts-$ACCOUNT_ID-$AWS_REGION"               # bucket names must be globally unique
echo "== target=$TARGET account=$ACCOUNT_ID region=$AWS_REGION registry=$REGISTRY"

echo "== 1/7 ECR repository + push the API image"
aws ecr describe-repositories --repository-names "$APP-api" >/dev/null 2>&1 \
  || aws ecr create-repository --repository-name "$APP-api" --image-scanning-configuration scanOnPush=true >/dev/null
aws ecr get-login-password | docker login --username AWS --password-stdin "$REGISTRY"
docker tag "$APP-api:$TAG" "$REGISTRY/$APP-api:$TAG"
docker push "$REGISTRY/$APP-api:$TAG"

echo "== 2/7 S3 bucket + upload adapters and reports"
aws s3 mb "s3://$BUCKET" 2>/dev/null || true
aws s3 sync outputs/adapters "s3://$BUCKET/adapters" --exclude "*/checkpoint-*" --only-show-errors
[ -d outputs/reports ] && aws s3 sync outputs/reports "s3://$BUCKET/reports" --only-show-errors
aws s3 cp deploy/docker-compose.yml "s3://$BUCKET/deploy/docker-compose.yml" --only-show-errors
aws s3 cp deploy/compose.aws.yml "s3://$BUCKET/deploy/compose.aws.yml" --only-show-errors

echo "== 3/7 Secrets Manager (HF token, LangSmith key, API key)"
for name in HF_TOKEN LANGSMITH_API_KEY API_KEY; do
  value="${!name:-}"
  [ -z "$value" ] && { echo "   skip $name (not set in your shell)"; continue; }
  aws secretsmanager create-secret --name "$APP/$name" --secret-string "$value" >/dev/null 2>&1 \
    || aws secretsmanager put-secret-value --secret-id "$APP/$name" --secret-string "$value" >/dev/null
  echo "   stored $APP/$name"
done

echo "== 4/7 IAM role the EC2 machine will use (least privilege)"
cat > /tmp/$APP-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":
{"Service":"ec2.amazonaws.com"},"Action":"sts:AssumeRole"}]}
EOF
cat > /tmp/$APP-policy.json <<EOF
{"Version":"2012-10-17","Statement":[
 {"Effect":"Allow","Action":["ecr:GetAuthorizationToken"],"Resource":"*"},
 {"Effect":"Allow","Action":
["ecr:BatchGetImage","ecr:GetDownloadUrlForLayer"],"Resource":"arn:aws:ecr:$AWS_REGION:$ACCOUNT_ID:repository/$APP-
api"},
 {"Effect":"Allow","Action":["s3:GetObject","s3:ListBucket"],"Resource":
["arn:aws:s3:::$BUCKET","arn:aws:s3:::$BUCKET/*"]},
 {"Effect":"Allow","Action":
["secretsmanager:GetSecretValue"],"Resource":"arn:aws:secretsmanager:$AWS_REGION:$ACCOUNT_ID:secret:$APP/*"},
 {"Effect":"Allow","Action":["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"],"Resource":"*"}]}
EOF
aws iam get-role --role-name "$APP-ec2-role" >/dev/null 2>&1 \
  || aws iam create-role --role-name "$APP-ec2-role" --assume-role-policy-document "file:///tmp/$APP-trust.json"
>/dev/null
aws iam put-role-policy --role-name "$APP-ec2-role" --policy-name "$APP-runtime" --policy-document "file:///tmp/$APP-
policy.json"
aws iam get-instance-profile --instance-profile-name "$APP-ec2-profile" >/dev/null 2>&1 || {
```

```
  aws iam create-instance-profile --instance-profile-name "$APP-ec2-profile" >/dev/null
  aws iam add-role-to-instance-profile --instance-profile-name "$APP-ec2-profile" --role-name "$APP-ec2-role"
  sleep 10                                                  # real IAM needs a few seconds to propagate
}

echo "== 5/7 CloudWatch: log group + CPU alarm"
aws logs create-log-group --log-group-name "/$APP/api" 2>/dev/null || true
aws logs put-retention-policy --log-group-name "/$APP/api" --retention-in-days 14
aws cloudwatch put-metric-alarm --alarm-name "$APP-cpu-high" --namespace AWS/EC2 --metric-name CPUUtilization \
  --statistic Average --period 300 --evaluation-periods 2 --threshold 90 --comparison-operator GreaterThanThreshold

echo "== 6/7 Security group: port 8080 (API) and 22 (SSH) only from $MY_IP"
SG_ID=$(aws ec2 describe-security-groups --filters "Name=group-name,Values=$APP-sg" \
        --query 'SecurityGroups[0].GroupId' --output text 2>/dev/null || echo None)
if [ "$SG_ID" = "None" ] || [ -z "$SG_ID" ]; then
  SG_ID=$(aws ec2 create-security-group --group-name "$APP-sg" --description "MedScriptAI API" \
          --query GroupId --output text)
  for port in 22 8080; do
    aws ec2 authorize-security-group-ingress --group-id "$SG_ID" --protocol tcp --port $port --cidr "$MY_IP/32"
>/dev/null
  done
fi

echo "== 7/7 Launch the GPU machine (g6.xlarge = 1x NVIDIA L4 24 GB)"
if [ "$TARGET" = "fakecloud" ]; then
  AMI=$(aws ec2 describe-images --owners amazon --query 'Images[0].ImageId' --output text)
else
  AMI=$(aws ec2 describe-images --owners amazon \
        --filters "Name=name,Values=Deep Learning Base OSS Nvidia Driver GPU AMI (Ubuntu 24.04)*"
"Name=state,Values=available" \
        --query 'sort_by(Images,&CreationDate)[-1].ImageId' --output text)
fi
sed -e "s|__REGION__|$AWS_REGION|g" -e "s|__REGISTRY__|$REGISTRY|g" -e "s|__BUCKET__|$BUCKET|g" -e "s|__TAG__|$TAG|g" \
  deploy/ec2_user_data.sh > /tmp/$APP-user-data.sh
aws ec2 describe-key-pairs --key-names "$APP-key" >/dev/null 2>&1 || {
  aws ec2 create-key-pair --key-name "$APP-key" --query KeyMaterial --output text > "$HOME/.ssh/$APP-key.pem"
  chmod 400 "$HOME/.ssh/$APP-key.pem"
}
INSTANCE_ID=$(aws ec2 run-instances --image-id "$AMI" --instance-type "${INSTANCE_TYPE:-g6.xlarge}" \
  --key-name "$APP-key" --security-group-ids "$SG_ID" \
  --iam-instance-profile Name="$APP-ec2-profile" \
  --block-device-mappings '[{"DeviceName":"/dev/sda1","Ebs":{"VolumeSize":150,"VolumeType":"gp3"}}]' \
  --user-data "file:///tmp/$APP-user-data.sh" \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$APP-gpu}]" \
  --query 'Instances[0].InstanceId' --output text)
echo "   instance $INSTANCE_ID starting (AMI $AMI)"
timeout 300 aws ec2 wait instance-running --instance-ids "$INSTANCE_ID" 2>/dev/null || sleep 5
IP=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" --query 'Reservations[0].Instances[0].PublicIpAddress' --
output text)
echo "DONE  ->  API will be at http://$IP:8080 in ~10-15 min (first boot downloads ~30 GB)."
echo "      SSH: ssh -i ~/.ssh/$APP-key.pem ubuntu@$IP   |   boot log: sudo tail -f /var/log/cloud-init-output.log"
```

Line	What it does (plain English)
`TARGET="${TARGET:-fakecloud}"`	the default is the free emulator; real AWS only if you type TARGET=aws
`MY_IP=...checkip.amazonaws.com`	your public IP, used to lock the firewall
`if [ "$TARGET" = "fakecloud" ]; then export AWS_ENDPOINT_URL=...`	the ONLY difference between the targets
`aws ecr describe-repositories ... || aws ecr create-repository ...`	"create if missing": safe to run twice (we tested a second run)
`aws s3 mb ... 2>/dev/null || true`	ignore "bucket already exists"
`value="${!name:-}"` + `continue`	skip secrets you did not export
`create-secret ... || put-secret-value ...`	create or update a secret
`aws iam get-role ... || aws iam create-role ...`	create the role once
`SG_ID=$(aws ec2 describe-security-groups --filters ...)`	re-use the firewall group on a second run
`sort_by(Images,&CreationDate)[-1].ImageId`	the newest matching AMI
`sed -e "s|__REGION__|$AWS_REGION|g" ...`	fills the placeholders of the boot script

`timeout 300 aws ec2 wait instance-running`	wait at most 5 minutes until the machine runs

CHECKPOINT: ls deploy shows deploy.sh

SECTION 6B.10 - Create deploy/teardown.sh and use both scripts

WHERE: VS Code Explorer (create), then VS Code Terminal

GOAL: delete everything with one command; repeat the deployment with one command.

Step 1. Right-click the folder deploy.

Step 2. Click New File...

Step 3. Type teardown.sh and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
#!/usr/bin/env bash
# Delete everything deploy.sh created, so the AWS bill stops.  TARGET=aws bash deploy/teardown.sh
set -uo pipefail                                          # no -e: keep going if one item is already gone
TARGET="${TARGET:-fakecloud}"
export AWS_REGION="${AWS_REGION:-us-east-1}" AWS_DEFAULT_REGION="${AWS_REGION:-us-east-1}"
APP=medscriptai
if [ "$TARGET" = "fakecloud" ]; then
  export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test
fi
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
BUCKET="$APP-artifacts-$ACCOUNT_ID-$AWS_REGION"

IDS=$(aws ec2 describe-instances --filters "Name=tag:Name,Values=$APP-gpu" "Name=instance-state-
name,Values=pending,running,stopped" \
      --query 'Reservations[].Instances[].InstanceId' --output text)
[ -n "$IDS" ] && aws ec2 terminate-instances --instance-ids $IDS >/dev/null && echo "terminated $IDS"
[ -n "$IDS" ] && timeout 600 aws ec2 wait instance-terminated --instance-ids $IDS 2>/dev/null
for try in 1 2 3 4 5 6; do                                # the SG can only go once the instance is gone
  aws ec2 delete-security-group --group-name "$APP-sg" >/dev/null 2>&1 && { echo "deleted security group"; break; }
  sleep 10
done
aws ec2 delete-key-pair --key-name "$APP-key" >/dev/null 2>&1 && echo "deleted key pair"
aws ecr delete-repository --repository-name "$APP-api" --force >/dev/null 2>&1 && echo "deleted ECR repo"
aws s3 rb "s3://$BUCKET" --force >/dev/null 2>&1 && echo "deleted bucket"
for name in HF_TOKEN LANGSMITH_API_KEY API_KEY; do
  aws secretsmanager delete-secret --secret-id "$APP/$name" --force-delete-without-recovery >/dev/null 2>&1 && echo
"deleted secret $name"
done
aws cloudwatch delete-alarms --alarm-names "$APP-cpu-high" 2>/dev/null && echo "deleted alarm"
aws logs delete-log-group --log-group-name "/$APP/api" 2>/dev/null && echo "deleted log group"
aws iam remove-role-from-instance-profile --instance-profile-name "$APP-ec2-profile" --role-name "$APP-ec2-role"
2>/dev/null
aws iam delete-instance-profile --instance-profile-name "$APP-ec2-profile" 2>/dev/null && echo "deleted instance
profile"
aws iam delete-role-policy --role-name "$APP-ec2-role" --policy-name "$APP-runtime" 2>/dev/null
aws iam delete-role --role-name "$APP-ec2-role" 2>/dev/null && echo "deleted role"
echo "TEARDOWN DONE ($TARGET)"
```

Line	What it does (plain English)
`set -uo pipefail` (no -e)	keep going if one resource is already gone
`describe-instances --filters "Name=tag:Name,Values=$APP-gpu"`	find our machines by their Name tag
`terminate-instances` then `wait instance-terminated`	delete the machine (stops compute billing) and wait
`for try in 1 2 3 4 5 6; do delete-security-group ...`	a firewall group can only be deleted after the machine is fully gone; we retry (found while testing)
`delete-repository --force`	delete the repo and its images
`aws s3 rb ... --force`	delete the bucket and all files in it
`delete-secret --force-delete-without-recovery`	delete immediately (no billed recovery window)
the four IAM lines	IAM pieces must be removed in this order

Step 6. Make the three scripts executable:

```
chmod +x ~/medscriptai/deploy/*.sh
```

Step 7. Clean up what you created by hand on FakeCloud:

```
cd ~/medscriptai && TARGET=fakecloud bash deploy/teardown.sh
```

You should see: lines such as terminated i-..., deleted security group, deleted ECR repo, deleted bucket, ... and TEARDOWN DONE (fakecloud)

Step 8. Deploy again with ONE command (FakeCloud):

```
TARGET=fakecloud bash deploy/deploy.sh
```

You should see: "== 1/7 ECR repository + push the API image" ... "== 7/7 Launch the GPU machine" and "DONE → API will be at http://52.0.0.10:8080 ..." (this exact flow ran during testing).

Step 9. Tear down again:

```
TARGET=fakecloud bash deploy/teardown.sh
```

Step 10. Real AWS (optional, costs money): run TARGET=aws bash deploy/deploy.sh, wait 10-15 minutes, test with row 26 of Section 6B.8, then run TARGET=aws bash deploy/teardown.sh the same day.

CHECKPOINT: both teardown runs end with TEARDOWN DONE; after a real-AWS run, EC2 > Instances shows terminated.

SECTION 6B.11 - (Optional) SageMaker endpoint objects

WHERE: VS Code Terminal, folder ~/medscriptai

GOAL: know the managed alternative and practise its API calls for free.

Step	FakeCloud (free practice)	Real AWS (same step)
1 Model	`aws sagemaker create-model --model-name medscriptai-model --primary-container Image=$REGISTRY/medscriptai-api:v1 --execution-role-arn arn:aws:iam::$ACCOUNT_ID:role/medscriptai-ec2-role`	needs a SageMaker-compatible container (GET /ping and POST /invocations on port 8080) and a role trusted by sagemaker.amazonaws.com; our API image is not that. AWS's Large Model Inference (LMI) containers with a vLLM backend are the usual choice (verify current names in the SageMaker docs)
2 Endpoint config	`aws sagemaker create-endpoint-config --endpoint-config-name medscriptai-cfg --production-variants VariantName=AllTraffic,ModelName=medscriptai-model,InstanceType=ml.g6.xlarge,InitialInstanceCount=1`	same shape
3 Endpoint	`aws sagemaker create-endpoint --endpoint-name medscriptai-endpoint --endpoint-config-name medscriptai-cfg`	same; billed per hour while InService
4 Status	`aws sagemaker describe-endpoint --endpoint-name medscriptai-endpoint --query EndpointStatus`	same
You should see	"InService" (FakeCloud emulates the control plane only; invoke-endpoint returned NotFound when we tested it)	"Creating" for 5-15 minutes, then "InService"
5 Delete	`aws sagemaker delete-endpoint --endpoint-name medscriptai-endpoint`	same, plus delete-endpoint-config and delete-model

Interview answer: EC2 + Docker Compose (what you built) gives full control and is cheapest for one always-on model; SageMaker adds managed autoscaling, safe deployments and monitoring, at a higher price and with stricter container rules.

CHECKPOINT: on FakeCloud describe-endpoint returned InService and delete-endpoint succeeded.

SECTION 6B.12 - Commit

Step 1. Run:

```
cd ~/medscriptai && git add . && git commit -m "Part 6B: Docker, Compose and cloud deployment"
```

Watch out: git status must NOT list deploy/.env (it is ignored). If it does, check .gitignore.

PART 6B RESULT CHECK

Check	Command	Expected
image	docker images medscriptai-api	v1, about 290 MB
container healthy	Section 6B.3 Step 3	healthy user=appuser
stack	docker compose -f deploy/docker-compose.yml ps	vllm healthy, api Up
vLLM not exposed	curl --max-time 3 localhost:8000/v1/models	fails
FakeCloud deploy	TARGET=fakecloud bash deploy/deploy.sh	DONE
clean teardown	TARGET=fakecloud bash deploy/teardown.sh	TEARDOWN DONE (fakecloud)
(AWS) live API	curl http://EC2_IP:8080/ready	"status": "ready"

6B.13 Common mistake at this level

The mistake: secrets baked into images or Git, vLLM's port open to the internet, and GPU machines left running.

How to spot it: docker history medscriptai-api:v1 shows no secrets; Section 6B.6 Step 5 fails as it should; after teardown, aws ec2 describe-instances shows nothing running.

PART 7 - ELITE: prove your results are real (how top labs work)

BEFORE YOU START: why this part exists

"Accuracy went from 61% to 64%" means nothing until you can answer four questions: (1) Is there test leakage? (2) Is 3 points on 500 questions real or luck? (3) Who graded it and can you trust the grader? (4) Can you roll a new version out and back safely? Top labs (Google's Med-PaLM 2, OpenAI's HealthBench, Stanford's MedHELM, AI2's Tulu 3) answer these before publishing. In this part you create five small scripts that answer them for YOUR project.

Question	Tool you build	Section
leakage?	04_contamination_check.py (13-gram near-duplicate check)	7.2
real or luck?	33_significance.py (paired bootstrap confidence interval)	7.3
trust the judge?	35_judge_agreement.py (Cohen's kappa vs your labels)	7.4
fields right?	36_summary_field_scores.py (criterion-level scoring)	7.5
safe rollout?	one environment variable + Compose (no new file)	7.6
documentation	34_report.py (model card)	7.7

7.1 Who defines the top level today

Who / what	Contribution you should know	Where
Tim Dettmers et al. (University of Washington), 2023	QLoRA: 4-bit NF4 + LoRA, fine-tuning big models on one GPU	https://arxiv.org/abs/2305.14314
Edward Hu et al. (Microsoft), 2021	LoRA: low-rank adapters	https://arxiv.org/abs/2106.09685
Rafael Rafailov, Archit Sharma, Eric Mitchell et al. (Stanford), 2023	DPO	https://arxiv.org/abs/2305.18290
Lewis Tunstall et al. (Hugging Face H4), 2023	Zephyr: SFT + DPO on AI feedback; the recipe this project follows	https://arxiv.org/abs/2310.16944
Nathan Lambert et al. (AI2), 2024	Tulu 3: open post-training (SFT, on-policy DPO, RL with verifiable rewards) with decontamination	https://arxiv.org/abs/2411.15124
Karan Singhal et al. (Google), Nature Medicine 2025	Med-PaLM 2: 86.5% on MedQA and physician pairwise evaluation	https://doi.org/10.1038/s41591-024-03423-7
Rahul K. Arora et al. (OpenAI), 2025	HealthBench: 5,000 conversations graded with 48,562 physician-written rubric criteria	https://arxiv.org/abs/2505.08775
Stanford CRFM / Stanford Medicine, 2025	MedHELM: 121 real clinical tasks (notes, decision support, communication, admin)	https://arxiv.org/abs/2505.23802
Lianmin Zheng et al. (LMSYS), 2023	LLM-as-a-judge biases and the position-swap trick	https://arxiv.org/abs/2306.05685
Seungone Kim et al., 2024	Prometheus 2: open judge models	https://arxiv.org/abs/2405.01535
Woosuk Kwon et al. (vLLM, UC Berkeley), 2023	PagedAttention, the memory manager behind vLLM	https://arxiv.org/abs/2309.06180
Ying Sheng et al. (S-LoRA), 2023	serving thousands of LoRA adapters on one base model	https://arxiv.org/abs/2311.03285
Daniel and Michael Han (Unsloth)	fast LoRA/QLoRA kernels	https://github.com/unslothai/unsloth
Brown et al. (OpenAI), 2020, GPT-3 paper	13-gram overlap contamination analysis	https://arxiv.org/abs/2005.14165

SECTION 7.2 - Leakage check (13-gram near-duplicates)

BEFORE YOU START: exact de-duplication (Section 4.4) misses near-copies: the same question with one word changed. We cut every question into all its 13-word windows ("13-grams") and flag a test question if at least half of its 13-grams come from ONE training question. A single shared 13-gram proves nothing: exam questions share stock phrases such as "which of the following is the most appropriate next step in management".

WHERE: VS Code Explorer (create), then VS Code Terminal with (.venv-eval)

GOAL: the number of near-duplicate test questions (should be 0 or very close).

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 04_contamination_check.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Elite check: is any TEST question a near-copy of a TRAINING question? (13-gram overlap, GPT-3 paper style)

Exact-match de-duplication (step 2) misses near-copies: the same question with one word changed.
Medical exams share stock phrases ("is brought to the emergency department because of a 2-day history of",
lab panels), so ONE shared 13-gram proves nothing. We flag a test question only when at least
50% of its 13-grams come from a SINGLE training question.
"""
import re
from collections import Counter, defaultdict
from common import SFT, read_jsonl

N = 13                       # n-gram length in words
THRESHOLD = 0.5              # share of the test question's n-grams found in one training question


def grams(text):
    text = text.split("\n\nOptions:")[0]                # compare the question stem, not the option labels
    words = re.findall(r"[a-z0-9]+", text.lower())
    return {" ".join(words[i:i + N]) for i in range(len(words) - N + 1)}


for task in ["qa", "notes", "summary"]:
    index = defaultdict(set)                            # n-gram -> ids of training rows containing it
    path = SFT / f"{task}_train.jsonl"
    if path.exists():
        for row_id, row in enumerate(read_jsonl(path)):
            for g in grams(row["messages"][1]["content"]):
                index[g].add(row_id)
    test = read_jsonl(SFT / f"{task}_test.jsonl")
    flagged = []
    for row in test:
        test_grams = grams(row["messages"][1]["content"])
        per_train_row = Counter(i for g in test_grams for i in index.get(g, ()))
        best = per_train_row.most_common(1)[0][1] if per_train_row else 0
        if test_grams and best / len(test_grams) >= THRESHOLD:
            flagged.append(row["messages"][1]["content"][:110].replace("\n", " "))
    print(f"{task:8} test rows={len(test):4}  near-duplicates of training rows={len(flagged):3} "
          f"({len(flagged) / max(len(test), 1):.1%})")
    for text in flagged[:3]:
        print("   e.g.", text)
```

Line	What it does (plain English)
`N = 13`	n-gram length, as in the GPT-3 analysis
`THRESHOLD = 0.5`	flag if at least half of a test question's 13-grams come from one training question
`def grams(text)`	lower-case words, drop the "Options:" block, return every 13-word window
`index = defaultdict(set)`	13-gram → ids of training rows that contain it
`per_train_row = Counter(...)`	for one test question: how many 13-grams it shares with each training row
`best / len(test_grams) >= THRESHOLD`	the share from the single most similar training row

Step 6. Run:

```
cd ~/medscriptai/scripts && source ../.venv-eval/bin/activate && python 04_contamination_check.py
```

You should see (real output on the full data):

```
qa       test rows= 500  near-duplicates of training rows=  1 (0.2%)
   e.g. Three days after admission to the intensive care unit for septic shock and bacteremia from a urinary tract inf
notes    test rows= 200  near-duplicates of training rows=  0 (0.0%)
summary  test rows= 200  near-duplicates of training rows=  0 (0.0%)
```

What we learned while building this (say it in interviews): (1) the first version counted ANY shared 13-gram and reported 13% "leakage" in QA, all false alarms from boilerplate; (2) the first full-data run then found REAL leakage: 14% (notes) and 18.5% (summary), because the clinical-notes dataset repeats 927 notes up to 5 times under different IDs. Removing repeated notes before splitting fixed it (0.0%). (3) One QA question (0.2%) remains: report it, or drop that test item and report results on 499.

Watch out: this only checks YOUR training data. Llama's pre-training data may already contain MedQA (it is public). That is why Med-PaLM 2, HealthBench and MedHELM also use new, private or expert-written test items.

CHECKPOINT: you can state the leakage numbers for all three tasks.

SECTION 7.3 - Is the gain real? (paired bootstrap)

BEFORE YOU START: the bootstrap. Your 500 test questions are one sample of all possible questions. The bootstrap draws 500 questions WITH replacement from your 500, computes the accuracy difference, and repeats this 10,000 times. The middle 95% of those differences is the confidence interval (CI). If the CI does not contain 0, the difference is unlikely to be luck. "Paired" = both models are scored on the same resampled questions, which removes question difficulty from the noise.

WHERE: VS Code Explorer (create), then VS Code Terminal with (.venv-eval)

GOAL: a 95% CI for "model B accuracy minus model A accuracy".

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 33_significance.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Elite check: is model B REALLY better than model A, or is it noise? Paired bootstrap on QA accuracy.

python scripts/33_significance.py --a qa --b qa-dpo
"""
import argparse
import random
from common import OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--a", required=True)
p.add_argument("--b", required=True)
p.add_argument("--rounds", type=int, default=10000)
args = p.parse_args()


def correct(row):
    pred = row["prediction"] or ""
    letter = pred.split("Answer:", 1)[1].strip()[:1].upper() if "Answer:" in pred else None
    return int(letter == row["answer_letter"])


a = [correct(r) for r in read_jsonl(OUT / "preds" / "qa" / f"{args.a}.jsonl")]
b = [correct(r) for r in read_jsonl(OUT / "preds" / "qa" / f"{args.b}.jsonl")]
n = min(len(a), len(b))
diffs = []
rng = random.Random(0)
for _ in range(args.rounds):
    idx = [rng.randrange(n) for _ in range(n)]          # resample the SAME questions for both models
    diffs.append(sum(b[i] - a[i] for i in idx) / n)
diffs.sort()
low, high = diffs[int(0.025 * args.rounds)], diffs[int(0.975 * args.rounds)]
observed = (sum(b[:n]) - sum(a[:n])) / n
print(f"accuracy {args.a}={sum(a[:n]) / n:.3f}  {args.b}={sum(b[:n]) / n:.3f}  diff={observed:+.3f}")
print(f"95% CI of the difference: [{low:+.3f}, {high:+.3f}]")
print("SIGNIFICANT" if low > 0 or high < 0 else "NOT significant: the CI contains 0, do not claim an improvement")
```

Line	What it does (plain English)
`def correct(row)`	1 if the predicted letter equals the gold letter, else 0
`a = [...]`, `b = [...]`	per-question correctness of the two models on the SAME questions
`idx = [rng.randrange(n) for _ in range(n)]`	one bootstrap sample: n random question indices, with replacement
`diffs.append(sum(b[i] - a[i] for i in idx) / n)`	the accuracy difference on that sample

`low, high = diffs[2.5%], diffs[97.5%]`	the 95% confidence interval
`SIGNIFICANT if low > 0 or high < 0`	the interval does not include 0

Step 6. Compare SFT with DPO, then base with SFT:

```
python 33_significance.py --a qa --b qa-dpo && python 33_significance.py --a base --b qa
```

You should see, for each comparison, three lines like:

```
accuracy qa=0.6xx  qa-dpo=0.6xx  diff=+0.0xx
95% CI of the difference: [-0.0xx, +0.0xx]
SIGNIFICANT          (or: NOT significant: the CI contains 0, do not claim an improvement)
```

Rule of thumb: on 500 questions a single accuracy has about +/- 4 points of uncertainty, so a 2-point gain is usually NOT significant; the paired test is tighter than comparing two separate intervals.

CHECKPOINT: you can write each comparison as "diff X, 95% CI [L, H], significant yes/no".

SECTION 7.4 - Can you trust the judge? (Cohen's kappa)

BEFORE YOU START: kappa. You and the judge both label the same 30 duels (qa-dpo, qa or tie). Raw agreement can look high by chance: if the judge always says "qa-dpo" and you say "qa-dpo" 80% of the time, you "agree" 80% while the judge learned nothing. Cohen's kappa removes chance agreement: 1 = perfect, 0 = no better than chance. We saw exactly this in testing: 0.80 agreement but kappa 0.00.

WHERE: VS Code Explorer (create), VS Code editor, VS Code Terminal with (.venv-eval)

GOAL: kappa between the judge and you on 30 pairwise comparisons.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 35_judge_agreement.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Elite check: can you trust the LLM judge? Compare its pairwise verdicts with YOUR labels (Cohen's kappa).

1) Open outputs/reports/human_labels.csv (created on first run) and fill the 'human' column
   with the model name you prefer, or 'tie', for each row (read the two answers in the preds files).
2) Run again: python scripts/35_judge_agreement.py --judge-file outputs/reports/judge_qa_pairwise_qa-dpo_vs_qa.json
"""
import argparse
import csv
import json
from collections import Counter
from pathlib import Path
from common import OUT

p = argparse.ArgumentParser()
p.add_argument("--judge-file", required=True)
p.add_argument("--n", type=int, default=30, help="how many items you will label by hand")
a = p.parse_args()

judge = json.loads(Path(a.judge_file).read_text())["per_item"][: a.n]
labels = OUT / "reports" / "human_labels.csv"
if not labels.exists():                                    # first run: make the sheet for you to fill
    with labels.open("w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["i", "human"])
        w.writerows([[i, ""] for i in range(len(judge))])
    print(f"fill the 'human' column in {labels}, then run this script again")
    raise SystemExit

human = {int(r["i"]): r["human"].strip() for r in csv.DictReader(labels.open()) if r["human"].strip()}
pairs = [(judge[i], h) for i, h in human.items() if i < len(judge)]
n = len(pairs)
observed = sum(j == h for j, h in pairs) / n                # raw agreement
jc, hc = Counter(j for j, _ in pairs), Counter(h for _, h in pairs)
expected = sum(jc[k] * hc[k] for k in set(jc) | set(hc)) / (n * n)   # agreement by pure chance
kappa = (observed - expected) / (1 - expected) if expected < 1 else 1.0
print(f"items={n}  agreement={observed:.2f}  Cohen's kappa={kappa:.2f}")
print("kappa > 0.6 = judge is usable; 0.4-0.6 = use with care; < 0.4 = do not trust this judge")
```

Line	What it does (plain English)
`judge = json.loads(...)["per_item"][: a.n]`	the judge's verdict per item (saved by 32_judge.py)
`if not labels.exists(): ... raise SystemExit`	the first run only creates an empty sheet for you
`human = {...}`	your labels; empty rows are ignored
`observed`	how often you and the judge agree
`expected`	agreement expected by chance from how often each of you uses each label
`kappa = (observed - expected) / (1 - expected)`	agreement beyond chance

Step 6. Create the label sheet:

```
python 35_judge_agreement.py --judge-file ../outputs/reports/judge_qa_pairwise_qa-dpo_vs_qa.json
```

You should see: "fill the 'human' column in .../outputs/reports/human_labels.csv, then run this script again"

Step 7. Show item 0 (then repeat with 1, 2, ... 29 by changing the number after i=):

```
python -c "import json;a=[json.loads(l) for l in open('../outputs/preds/qa/qa-dpo.jsonl')];b=[json.loads(l) for l in
open('../outputs/preds/qa/qa.jsonl')];i=0;print(a[i]['prompt'],'\n\nREFERENCE:',a[i]['reference'],'\n\nqa-dpo:',a[i]
['prediction'],'\n\nqa:',b[i]['prediction'])"
```

Step 8. In the Explorer open outputs > reports > human_labels.csv.

Step 9. For each row type qa-dpo, qa or tie after the comma (example line: 0,qa-dpo) and press Ctrl+S when done.

Step 10. Run again:

```
python 35_judge_agreement.py --judge-file ../outputs/reports/judge_qa_pairwise_qa-dpo_vs_qa.json
```

You should see: items=30 agreement=0.xx Cohen's kappa=0.xx and "kappa > 0.6 = judge is usable; 0.4-0.6 = use with care; < 0.4 = do not trust this judge".

Watch out: you are not a physician. A real medical product needs clinician labels (HealthBench used 262 physicians). For this project say: "judge-human agreement measured on 30 items by the author".

CHECKPOINT: you can report kappa and what it means.

SECTION 7.5 - Criterion-level scoring for the summary task

BEFORE YOU START: HealthBench grades every answer against a list of specific criteria. Our summary task allows a verifiable version: every field in the reference JSON (for example "patient information.age") is one criterion. Recall = how many reference fields the model filled (missed facts lower it). Precision = how many of the model's fields exist in the reference (invented fields lower it: a hallucination signal).

WHERE: VS Code Explorer (create), then VS Code Terminal with (.venv-eval)

GOAL: field precision/recall for base vs your summary adapter.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 36_summary_field_scores.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Elite check: criterion-level scoring for the summary task (rubric idea from HealthBench, made verifiable).

Every field path in the REFERENCE JSON is one criterion, e.g. "patient information.age".
recall    = reference fields the model filled       (missing facts lower it)
precision = model fields that exist in the reference (invented fields lower it = hallucination signal)
"""
import argparse
import json
from common import OUT, read_jsonl

p = argparse.ArgumentParser()
p.add_argument("--models", nargs="+", default=["base", "summary"])
a = p.parse_args()


def paths(value, prefix=""):
    """{"a": {"b": 1}, "c": [{"d": 2}]} -> {"a.b", "c.d"} (list items are merged)."""
```

```
    out = set()
    if isinstance(value, dict):
        for k, v in value.items():
            out |= paths(v, f"{prefix}{k.lower().strip()}.")
    elif isinstance(value, list):
        for v in value:
            out |= paths(v, prefix)
    elif value not in (None, "", "None"):
        out.add(prefix.rstrip("."))
    return out


def load(text):
    try:
        return json.loads((text or "").strip().removeprefix("```json").removesuffix("```").strip())
    except json.JSONDecodeError:
        return None


for model in a.models:
    rows = read_jsonl(OUT / "preds" / "summary" / f"{model}.jsonl")
    tp = fp = fn = invalid = 0
    for r in rows:
        ref, pred = paths(load(r["reference"]) or {}), load(r["prediction"])
        if pred is None:
            invalid += 1
            fn += len(ref)                    # an unreadable answer misses every criterion
            continue
        got = paths(pred)
        tp += len(ref & got)
        fp += len(got - ref)
        fn += len(ref - got)
    precision = tp / max(tp + fp, 1)
    recall = tp / max(tp + fn, 1)
    f1 = 2 * precision * recall / max(precision + recall, 1e-9)
    print(f"{model:10} field precision={precision:.3f} recall={recall:.3f} F1={f1:.3f} invalid_json=
{invalid}/{len(rows)}")
```

Line	What it does (plain English)
`def paths(value, prefix="")`	flattens nested JSON into field paths such as "patient information.age"
`elif value not in (None, "", "None")`	a field counts only if it has a real value
`def load(text)`	parse JSON (also inside a ```json block); None if invalid
`if pred is None: invalid += 1; fn += len(ref)`	an unreadable answer misses every criterion
`tp`, `fp`, `fn`	fields in both / invented by the model / missed by the model
`precision`, `recall`, `f1`	the standard definitions over all fields

Step 6. Run:

```
python 36_summary_field_scores.py --models base summary
```

You should see: two lines like "summary field precision=0.xxx recall=0.xxx F1=0.xxx invalid_json=x/200". Expect the fine-tuned adapter to have far fewer invalid JSON outputs and a higher recall than base.

Watch out: this checks that the right FIELDS are filled, not that each VALUE is correct. Combine it with the judge or with exact checks on simple fields (age, sex).

CHECKPOINT: you have field precision/recall for both models.

SECTION 7.6 - Zero-downtime rollout and rollback

BEFORE YOU START: in production you never "just replace" a model. You switch traffic to the new version, watch, and switch back in seconds if it is worse. Your API picks the adapter from an environment variable (MODEL_QA), and vLLM keeps all adapters loaded, so a switch only restarts the small API container, never the model server.

WHERE: VS Code Terminal and editor, Compose stack from Section 6B.6 running

GOAL: switch task qa from qa-dpo to qa and back, without restarting vLLM.

Step 1. Ask vLLM (from inside the api container, because vLLM's port is not published) which adapters it serves:

```
cd ~/medscriptai && docker compose -f deploy/docker-compose.yml exec api python -c "import urllib.request,json;
print([m['id'] for m in json.load(urllib.request.urlopen('http://vllm:8000/v1/models'))['data']])"
```

You should see: ['base', 'qa', 'notes', 'summary', 'qa-dpo']

Step 2. Open deploy/.env in the editor, add this line at the end, and press Ctrl+S:

```
MODEL_QA=qa
```

Step 3. Recreate ONLY the api container:

```
cd ~/medscriptai/deploy && docker compose --env-file .env up -d api && cd ..
```

Step 4. Check which adapter now answers qa:

```
curl -s -X POST localhost:8080/v1/generate -H "Content-Type: application/json" -H "X-API-Key: $(grep ^API_KEY
deploy/.env | cut -d= -f2)" -d '{"task":"qa","input":"Which vitamin deficiency causes scurvy?\n\nOptions:\nA) A\nB)
B12\nC) C\nD) D"}' | jq -r .model
```

You should see: qa

Step 5. Roll back: change the line in deploy/.env to MODEL_QA=qa-dpo, save, and repeat Step 3.

Step 6. Prove vLLM never restarted:

```
docker compose -f deploy/docker-compose.yml ps
```

You should see: vllm still "Up" for the same long time; only api was recreated.

Concept	Meaning
canary	send a small share of traffic to the new version first and compare metrics
rollback	return to the previous version; here one variable and a 2-second restart

CHECKPOINT: you switched and rolled back while vllm stayed up.

SECTION 7.7 - Report and model card

WHERE: VS Code Explorer (create), VS Code Terminal with (.venv-eval), then WinSCP and the Hub website

GOAL: outputs/reports/REPORT.md for your model card, README and interviews.

Step 1. Right-click the folder scripts.

Step 2. Click New File...

Step 3. Type 34_report.py and press Enter.

Step 4. Paste the grey box below.

Step 5. Press Ctrl+S.

```
"""Build outputs/reports/REPORT.md (paste it into your Hugging Face model card and your CV/portfolio)."""
import json
from pathlib import Path
from common import OUT

reports = OUT / "reports"
metrics = json.loads((reports / "metrics.json").read_text())
cols = ["n", "accuracy", "format_ok", "json_valid", "rougeL", "bertscore_f1"]
lines = ["# MedScriptAI evaluation report", "", "## Automatic metrics", "",
         "| task/model | " + " | ".join(cols) + " |", "|" + "---|" * (len(cols) + 1)]
for name, m in metrics.items():
    lines.append(f"| {name} | " + " | ".join(str(m.get(c, "-")) for c in cols) + " |")
lines += ["", "## LLM-as-a-judge", ""]
for path in sorted(reports.glob("judge_*.json")):
    lines.append(f"- **{path.stem.removeprefix('judge_')}**: `{json.dumps(json.loads(path.read_text()))}`")
lines += ["", "Judge: Qwen2.5-14B-Instruct. Pairwise results use position swapping (a win must win in both orders).",
          "", "Built with Llama. Educational use only; not a medical device and not medical advice."]
(reports / "REPORT.md").write_text("\n".join(lines) + "\n")
print("\n".join(lines))
```

Line	What it does (plain English)
`metrics = json.loads((reports / "metrics.json").read_text())`	all automatic metrics
`cols = [...]`	the columns of the table

`lines.append(f"| {name} | ...")`	one table row per task/model ("-" where a metric does not apply)
`for path in sorted(reports.glob("judge_*.json"))`	every judge result
final lines	judge description, "Built with Llama", not-for-clinical-use statement

Step 6. Run:

```
python 34_report.py
```

You should see: a Markdown table (# MedScriptAI evaluation report ...) printed and saved as outputs/reports/REPORT.md.

Step 7. In WinSCP, drag /home/ubuntu/medscriptai/outputs/reports from the right panel to your laptop.

Step 8. On your Hub adapter page click Edit model card, paste REPORT.md, and add: base model, data sources and licences, training settings (r=16, alpha=16, LR 2e-4, 1 epoch; DPO beta 0.1, LR 5e-6), leakage results (7.2), significance results (7.3), judge kappa (7.4), limitations, "Built with Llama", "not for clinical use". Click Commit changes.

CHECKPOINT: the model card shows your numbers.

7.8 What elite teams would try next

- **RL with verifiable rewards (GRPO).** For multiple-choice QA the reward is checkable (correct letter = 1). TRL's GRPOTrainer and Unsloth support it (verify current docs).
- **Other preference losses.** DPO vs SimPO (TRL loss_type="sigmoid_norm"), IPO, KTO, ORPO on the same pairs, each with a significance test.
- **Iterative on-policy DPO.** Regenerate pairs with the DPO model and train a second round; stop when the win rate stops moving.
- **Serving efficiency.** FP8 or AWQ base model, prefix caching for the shared system prompt, speculative decoding; measure tokens per second and cost per 1,000 requests.
- **Safety evaluation.** A red-team set (self-harm, dosing, emergencies) with expected refusals or escalation; track refusal rate next to helpfulness.
- **Privacy and regulation.** Real notes contain patient data: de-identify, keep traces private, and know that software making clinical decisions is regulated (for example FDA "software as a medical device").

SECTION 7.9 - Commit

Step 1. Run:

```
cd ~/medscriptai && git add . && git commit -m "Part 7: leakage, significance, judge agreement, field scores, report"
```

PART 7 RESULT CHECK

Check	Where	Expected
leakage	04_contamination_check.py	qa about 0.2%, notes 0.0%, summary 0.0%
significance	33_significance.py	a CI for each comparison and a clear yes/no
judge reliability	35_judge_agreement.py	kappa with interpretation
field scores	36_summary_field_scores.py	precision/recall/invalid_json for base and summary
rollout	Section 7.6	switch + rollback with vllm still up
model card	Hub page	REPORT.md + limitations + "Built with Llama"

7.10 Common mistake at this level

The mistake: elite-looking analyses that are not tied to a decision: a kappa nobody reads, a CI computed on a different test set than the headline number, a leakage check run after the results are published.

How to spot it: write the question first ("is DPO better than SFT on QA accuracy?"), then the analysis, then the decision ("ship qa-dpo" or "keep qa").

PART 8 - Tools and Resources Map (checked October 2026)

GitHub stars were read on 21 October 2026 and change daily. Prices marked (verify) could not be confirmed on the official page.

Tool	What it does	Free / Open-source / Paid	Exact cost + free-tier limits	Who uses it	Platform	Official link (+ stars)	Best for	My verdict
Unsloth 2026.10.7	2x faster, lower-VRAM LoRA/QLoRA/GRPO training	open source (Apache-2.0) + paid Pro	free	individuals, startups, Kaggle winners	Linux/Windows, NVIDIA (AMD/Apple partial)	https://github.com/unslothai/unsloth (78k)	single-GPU QLoRA	default choice for one GPU
TRL 1.15/1.16	SFT, DPO, GRPO, reward trainers	open source (Apache-2.0)	free	Hugging Face, AI2, most open post-training papers	Python	https://github.com/huggingface/trl (19.6k)	the standard post-training API	learn its configs deeply
PEFT 0.21	LoRA and other adapters	open source	free	industry-wide	Python	https://github.com/huggingface/peft (21.8k)	adapters	always used under TRL/Unsloth
Transformers 5.x	model and tokenizer library	open source	free	everyone	Python	https://github.com/huggingface/transformers (168k)	loading any model	core skill
bitsandbytes 0.50	4-bit/8-bit quantisation, 8-bit optimizers	open source	free	QLoRA users	Linux/Windows	https://github.com/bitsandbytes-foundation/bitsandbytes (8.5k)	QLoRA	needed, rarely touched
distilabel 1.5.3	synthetic data and AI-feedback pipelines	open source (Apache-2.0)	free	Argilla/HF community, Zephyr-style datasets	Python	https://github.com/argilla-io/distilabel (3.4k)	generate + rate pipelines	last release Jan 2025: pin versions or write plain async OpenAI calls
vLLM 0.32	high-throughput inference, multi-LoRA	open source (Apache-2.0)	free	Meta, Mistral, AWS, many startups	Linux, NVIDIA/AMD/TPU	https://github.com/vllm-project/vllm (94.6k)	serving open models	industry default

SGLang	inference engine (RadixAttention, structured output)	open source	free	xAI, LMSYS, cloud providers	Linux	https://github.com/sgl-project/sglang (37.6k)	very high throughput, agents	strong alternative to vLLM
Axolotl / LLaMA-Factory	config-driven fine-tuning frameworks	open source	free	researchers, startups	Linux	https://github.com/axolotl-ai-cloud/axolotl (12.6k), https://github.com/hiyouga/LLaMA-Factory (75.6k)	multi-GPU / no-code training	good once you understand TRL
FastAPI 0.156	Python web API framework	open source (MIT)	free	Microsoft, Netflix, Uber, countless startups	Python	https://github.com/fastapi/fastapi (103k)	ML APIs	default for Python services
LangSmith	tracing, evaluation, prompt management	paid SaaS with free tier	Developer plan US$0: 1 seat, about 5k base traces/month; overage about US$2.50-5.00 per 1k traces (sources disagree, verify) [11]	LangChain users, many enterprises	web + SDK	https://smith.langchain.com (SDK 1.1k)	quick tracing	fine for demos; check data-privacy terms
Langfuse	open-source LLM tracing/evals	open source (MIT) + cloud	self-host free; cloud free tier (verify)	startups wanting self-hosting	web + SDK	https://github.com/langfuse/langfuse (36.1k)	self-hosted tracing	best when data must stay inside
Docker / Compose	containers	free (Docker Desktop paid for large companies)	Engine free	everyone	Linux/Win/Mac	https://docs.docker.com	packaging	must-know
FakeCloud 0.51	local AWS emulator, 106 services incl. ECR push/pull, EC2, SageMaker control plane	open source (AGPL-3.0)	free, no account [5]	developers testing AWS code	Linux/macOS/Docker	https://github.com/faiscadev/fakecloud (761)	free AWS practice and integration tests	perfect for learning; not a GPU

LocalStack	the best-known AWS emulator	open source core + paid tiers	free tier needs an account/auth token (verify)	many companies' CI	Docker	https://github.com/localstack/localstack (65k)	team CI	industry standard alternative
moto	Python library that mocks AWS	open source	free	Python unit tests	Python	https://github.com/getmoto/moto (8.7k)	unit tests	lightweight
AWS (ECR, S3, EC2, SageMaker, CloudWatch)	production cloud	paid	g6.xlarge about US$0.805/h on demand in us-east-1 [12]; ECR storage about US$0.10/GB-month (verify)	most enterprises	web/CLI	https://aws.amazon.com	real deployment	learn the core 6 services
Hyperstack	GPU cloud VMs	paid	A100-80G-PCIe about US$1.35/h, L40 about US$1.00/h, RTX A6000 about US$0.50/h on demand [1] (verify)	startups, researchers	web/API	https://www.hyperstack.cloud	cheap single-GPU training	good value; remember hibernate
lm-evaluation-harness	standard benchmark runner (MedQA, MMLU, ...)	open source (MIT)	free	EleutherAI, HF Open LLM Leaderboard	Python	https://github.com/EleutherAI/lm-evaluation-harness (14.3k)	comparable benchmark numbers	add it after this project
Stanford HELM / MedHELM	holistic evaluation framework	open source	free	Stanford, research	Python	https://github.com/stanford-crfm/helm (3k)	rigorous medical evaluation	read the MedHELM paper
Prometheus-eval	open judge models	open source	free	researchers	Python	https://github.com/prometheus-eval/prometheus-eval (1.1k)	judging without paid APIs	good second judge
DeepEval	LLM test framework (pytest-style)	open source + cloud	free core	startups	Python	https://github.com/confident-ai/deepeval (19k)	CI tests for LLM apps	nice for API regression tests

uv	fast Python package/env manager	open source	free	fast-growing industry default	all	https://github.com/astral-sh/uv (91k)	environments	use everywhere
VS Code + Remote-SSH	editor on remote machines	free	free	everyone	all	https://code.visualstudio.com (194k)	working on GPU VMs	default
WinSCP	SFTP/SCP file manager	free (GPL)	free	Windows users	Windows	https://winscp.net (3.9k)	moving files	simple and reliable

Best free setup: Google Colab or Kaggle free GPU (T4, 15-16 GB; no bf16, slower) + Unsloth + TRL + FakeCloud + LangSmith Developer + Hugging Face Hub. You can do Parts 4-5 on a T4 with --batch 2 --grad-accum 8, but not the 14B teacher.

Best paid setup: 1x H100 80 GB (Hyperstack about US$1.90-3.00/h [1]) for everything; AWS or GCP for deployment with Terraform; Langfuse self-hosted or LangSmith Plus for tracing; W&B or MLflow for experiment tracking.

Best setup within this manual's budget (about US$30-40): Hyperstack 1x A100-80G-PCIe only while working + hibernate + FakeCloud for every AWS rehearsal + one 2-3 hour real AWS deployment with immediate teardown.

Five open-source projects worth studying:

- **huggingface/alignment-handbook** (5.7k stars): the exact Zephyr SFT + DPO recipes and configs; compare them with yours. https://github.com/huggingface/alignment-handbook
- **allenai/open-instruct** (3.9k stars): AI2's Tulu 3 post-training code (SFT, DPO, RLVR) with decontamination tooling. https://github.com/allenai/open-instruct
- **epfLLM/meditron** (2.2k stars): an open medical LLM with documented data and evaluation. https://github.com/epfLLM/meditron
- **openai/simple-evals** (4.7k stars): includes the HealthBench grading code and rubric approach. https://github.com/openai/simple-evals
- **vllm-project/vllm** (94.6k stars): read docs/features/lora.md and the LoRA code path to understand multi-adapter batching. https://github.com/vllm-project/vllm

PART 9 - Keep It Working

Task	How
Save your work	push adapters and datasets to the Hub (Sections 4.10, 5.12); git add/commit/push your code changes; aws s3 sync outputs/reports to S3
Push your code to GitHub (portfolio)	1) github.com > New repository > name medscriptai > Create (no README). 2) on the VM: git remote add origin https://github.com/YOUR_GITHUB_USER/medscriptai.git. 3) git branch -M main. 4) git push -u origin main (user name + a GitHub personal access token as password: github.com > Settings > Developer settings > Personal access tokens)
Back up the VM	your important files are on the Hub and in Git; the VM is disposable. To be extra safe, Hyperstack > Volumes > create a Shared Storage Volume and copy outputs/ there (see Hyperstack docs)
Stop costs	Hibernate the Hyperstack VM (Section 2A.13); TARGET=aws bash deploy/teardown.sh; delete unused Hub repos is optional (free)
Restart after hibernate	Restore the VM; update the IP in ~/.ssh/config and WinSCP; on the VM: mkdir -p /ephemeral/hf-cache; restart FakeCloud (Section 2B.11 Step 6); cd deploy && docker compose --env-file .env up -d
Resume an interrupted training run	checkpoints are in outputs/checkpoints/RUN; change trainer.train() to trainer.train(resume_from_checkpoint=True) in 10_train_sft.py and run the same command again
Update libraries safely	never upgrade in place: create .venv-train2, install the new versions, run 03_validate_jsonl.py and a --max-steps 20 training test, compare, then switch. Unsloth releases often pin new transformers/trl versions
Update vLLM	change VLLM_IMAGE to the new tag, restart, re-run 30_generate_preds.py on 50 rows and compare outputs with the old version
Undo a code change	git diff (see changes), git restore FILE (undo one file), git log + git revert COMMIT (undo a pushed commit)
Free disk space	docker system prune (removes stopped containers and dangling images); du -sh $HF_HOME/hub/* to find big cached models, then delete only the folders you no longer need
Rotate secrets	create a new HF/LangSmith key, aws secretsmanager put-secret-value, restart the API; delete the old key on the website

PART 10 - Troubleshooting

Symptom	Likely cause	Exact fix (steps)
Hyperstack connection problems (timeouts, publickey, VS Code stuck, WinSCP key)	see the dedicated table	MS Part 2A, Section 2A.14
SyntaxError / invalid character when running a script you created	the paste was cut, or curly quotes came from an edited copy	open the file, Ctrl+A, Delete, paste again from the grey box, Ctrl+S; check with python -m py_compile scripts/FILE.py
ModuleNotFoundError: No module named 'common'	script run from the wrong folder	cd ~/medscriptai/scripts first (scripts import common.py from their own folder)
ssh: Connection timed out	SSH access/firewall not enabled or wrong IP after restore	1) Hyperstack > VM > Firewall: Enable SSH access. 2) Copy the current PUBLIC IP. 3) Update HostName in ~/.ssh/config
torch.cuda.is_available() is False	CPU build of PyTorch installed	cd ~/medscriptai && uv pip install --python .venv-train -r requirements/train.txt --torch-backend cu126 --reinstall
CUDA out of memory during training	batch too big, or vLLM/teacher still running on the GPU	1) nvidia-smi to see who uses memory. 2) docker rm -f medscriptai-vllm medscriptai-teacher. 3) or add --batch 4 --grad-accum 4
Two <|begin_of_text|> tokens / quality worse than expected	chat template applied twice	use 10_train_sft.py as given (removeprefix(tokenizer.bos_token)); print train[0]["text"] and check it starts with <|start_header_id|>
vLLM: "CUDA driver version is insufficient for CUDA runtime version"	image CUDA newer than the driver	1) keep -e VLLM_ENABLE_CUDA_COMPATIBILITY=1 with the -cu129 image (data-centre GPU). 2) or deploy with the R595 / CUDA 13.2 OS image and VLLM_IMAGE=vllm/vllm-openai:v0.32.0
vLLM returns 404 "The model `qa` does not exist"	adapter not registered or wrong name	1) curl localhost:8000/v1/models. 2) check outputs/adapters/sft-qa/adapter_config.json exists. 3) restart serve/start_vllm.sh
Requests for different adapters are slow / queue	--max-loras left at the default 1	start vLLM with --max-loras 4 (already in start_vllm.sh and docker-compose.yml)
distilabel: No module named 'requests' / datasets errors	wrong environment or unpinned versions	cd ~/medscriptai && uv pip install --python .venv-synth -r requirements/synth.txt, then source .venv-synth/bin/activate
Docker build: 429 Too Many Requests pulling the base image	anonymous pull rate limit	1) wait 10 minutes, or 2) docker build --build-arg BASE_IMAGE=python:3.12-slim ... (other registry), or 3) docker login to Docker Hub
AWS: VcpuLimitExceeded when launching g6.xlarge	GPU quota still 0	Service Quotas > Amazon EC2 > Running On-Demand G and VT instances > request 8 vCPUs (Section 2B.12); wait for approval
aws CLI: Could not connect to the endpoint URL http://localhost:4566	FakeCloud not running (e.g. after hibernate)	1) nohup fakecloud --storage-mode persistent --data-path ~/fakecloud-data > ~/fakecloud.log 2>&1 &. 2) curl localhost:4566/_fakecloud/health
Real AWS command accidentally went to FakeCloud (or the opposite)	AWS_ENDPOINT_URL still set / not set	echo $AWS_ENDPOINT_URL before every blue-column step; unset AWS_ENDPOINT_URL for real AWS
teardown: DependencyViolation deleting the security group	instance still shutting down	run TARGET=aws bash deploy/teardown.sh again after 1-2 minutes (the script already retries 6 times)

PART 11 - Practice Projects (goals and success criteria only, no solutions)

11.1 Easy - a fourth adapter: patient-friendly explanations

Goal: an adapter "explain" that rewrites a clinical note in plain language for patients (reading age about 12).

Requirements: generate 2,000 training pairs with distilabel from the notes data (teacher writes the plain version); add a SYSTEM["explain"] prompt; train with 10_train_sft.py; serve it as a fifth adapter; add task "explain" to the API.

Success criteria: the API returns explanations for 20 unseen notes; a readability score (for example Flesch-Kincaid grade) is at least 3 grades lower than the source note; the judge rates faithfulness at least 4/5 on 80% of items.

11.2 Medium - RL with verifiable rewards for QA

Goal: improve qa-dpo with GRPO using the correct letter as the reward.

Requirements: use TRL's GRPOTrainer (or Unsloth's GRPO support) on the 1,891 DPO prompts; reward = 1 for the correct letter, 0 otherwise, plus a small format reward; evaluate exactly as in Sections 6A.7 and 7.3.

Success criteria: a QA accuracy gain over qa-dpo whose 95% paired bootstrap interval excludes 0, with format_ok still above 99%.

11.3 Hard - production hardening on AWS

Goal: a deployment a security reviewer would accept.

Requirements: infrastructure as code (Terraform or AWS CDK) for everything in deploy.sh; HTTPS with an Application Load Balancer and an ACM certificate; the API in a private subnet; secrets only from Secrets Manager; CloudWatch alarm on 5xx rate and p95 latency with e-mail alerts; a canary that sends 10% of qa traffic to a new adapter; a load test (for example locust) report; teardown with one command; everything rehearsed on FakeCloud first.

Success criteria: terraform apply and destroy both succeed on FakeCloud and AWS; p95 latency under 3 s at 5 requests/second on g6.xlarge for qa; zero secrets in Git, images or user data; the canary can be rolled back in under 1 minute.

PART 12 - Cheat Sheet

Part 2 - workspace

```
ssh-keygen -t ed25519 -f $env:USERPROFILE\.ssh\hyperstack_ed25519 -C "medscriptai"
ssh -i $env:USERPROFILE\.ssh\hyperstack_ed25519 ubuntu@VM_IP
nvidia-smi
echo 'export HF_HOME=/ephemeral/hf-cache' >> ~/.bashrc
curl -LsSf https://astral.sh/uv/install.sh | sh
mkdir -p ~/medscriptai/{scripts,serve,api,deploy,tests,requirements,data,outputs}
uv venv .venv-train --python 3.12
uv pip install --python .venv-train -r requirements/train.txt --torch-backend cu126
uv venv .venv-synth --python 3.12 && uv pip install --python .venv-synth -r requirements/synth.txt
uv venv .venv-eval --python 3.12 && uv pip install --python .venv-eval -r requirements/eval.txt --torch-backend cu126
curl -fsSL https://fakecloud.dev/install.sh | bash -s -- --version v0.51.0
nohup fakecloud --storage-mode persistent --data-path ~/fakecloud-data > ~/fakecloud.log 2>&1 &
export AWS_ENDPOINT_URL=http://localhost:4566 AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-
east-1
unset AWS_ENDPOINT_URL        # back to real AWS
tmux new -s NAME   |   Ctrl+B then D (detach)   |   tmux attach -t NAME
```

Part 4 - data and first SFT (.venv-train, folder scripts)

```
python 01_download_data.py
python 02_build_sft_jsonl.py
python 03_validate_jsonl.py
python 10_train_sft.py --task qa --max-train 2000
python 11_chat_test.py --task qa --n 3
python 12_push_to_hub.py --folder ../outputs/adapters/sft-qa --repo YOUR_HF_USERNAME/Llama-3.1-8B-MedScriptAI-qa-lora
aws s3 sync outputs/adapters/sft-qa s3://$BUCKET/adapters/sft-qa
```

Part 5 - synthetic data, three adapters, evaluation

```
bash serve/start_teacher.sh                              # teacher/judge on :8001
python 20_synth_explanations.py --limit 50               # (.venv-synth)
python 10_train_sft.py --task qa --train-file ../data/sft/qa_train_synth.jsonl --max-train 12000
python 10_train_sft.py --task notes && python 10_train_sft.py --task summary
GPU_UTIL=0.35 bash serve/start_vllm.sh                   # student + adapters on :8000
python 30_generate_preds.py --task qa --models base qa   # (.venv-eval)
python 31_eval_metrics.py
python 32_judge.py --task notes --mode score --models base notes --n 100
```

Part 6A/6B - DPO, serving, API, Docker, deployment

```
python 21_build_dpo_pairs.py --task qa --limit 1500      # (.venv-synth)
python 22_train_dpo.py --task qa                         # (.venv-train)
curl -s -X POST localhost:8000/v1/load_lora_adapter -H "Content-Type: application/json" -d
'{"lora_name":"NAME","lora_path":"/adapters/FOLDER"}'
curl -s -X POST localhost:8000/v1/unload_lora_adapter -H "Content-Type: application/json" -d '{"lora_name":"NAME"}'
python 32_judge.py --task qa --mode pairwise --models qa-dpo qa --n 200
VLLM_URL=http://localhost:8000/v1 API_KEY=dev-key PYTHONPATH=../scripts uvicorn app:app --port 8080
docker build -f api/Dockerfile -t medscriptai-api:v1 .
cd deploy && docker compose --env-file .env up -d
aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY
docker tag medscriptai-api:v1 $REGISTRY/medscriptai-api:v1 && docker push $REGISTRY/medscriptai-api:v1
TARGET=fakecloud bash deploy/deploy.sh      |   TARGET=aws bash deploy/deploy.sh
TARGET=fakecloud bash deploy/teardown.sh    |   TARGET=aws bash deploy/teardown.sh
```

Part 7 - elite checks

```
python 04_contamination_check.py
python 33_significance.py --a qa --b qa-dpo
python 35_judge_agreement.py --judge-file ../outputs/reports/judge_qa_pairwise_qa-dpo_vs_qa.json
python 36_summary_field_scores.py --models base summary
python 34_report.py
```

Key numbers and settings

Setting	Value	Why
LoRA rank / alpha / dropout	16 / 16 / 0	scale 1.0; fast in Unsloth
target modules	q,k,v,o,gate,up,down	all linear layers
SFT LR / epochs / batch	2e-4 / 1 / 16 effective	QLoRA standard
DPO beta / LR / batch	0.1 / 5e-6 / 16 pairs	stable DPO
max sequence length	2048 (train), 4096 (serve)	data max about 1,600 tokens
DPO loss at step 0	0.693 = ln 2	policy equals reference
vLLM LoRA flags	--enable-lora --max-loras 4 --max-lora-rank 16 --max-cpu-loras 8	default max-loras is 1
Memory rule of thumb	2 bytes/param (bf16), ~0.5-0.7 bytes/param (4-bit)	8B: 16 GB vs ~6 GB
Hyperstack states	SHUTOFF = still billed; HIBERNATED = GPU not billed, /ephemeral may be wiped	Section 2A.13
FakeCloud	http://localhost:4566, account 123456789012, any keys	Section 2B.11

PART 13 - Self-Test

1. What happens if you remove .removeprefix(tokenizer.bos_token) from 10_train_sft.py?
2. Do this: compute how many optimizer steps one epoch takes with 7,040 notes examples, --batch 8 and --grad-accum 2.
3. What happens if you start vLLM without --max-loras and three users call qa, notes and summary at the same moment?
4. What happens to the DPO loss at step 0, and why?
5. Do this: write the aws command that lists the images in the medscriptai-api ECR repository, for FakeCloud.
6. What happens if you hibernate the Hyperstack VM with the Hugging Face cache on /ephemeral, and what must you do after Restore?
7. Your QA accuracy goes from 0.612 to 0.628 on 500 questions and the paired 95% CI is [-0.010, +0.042]. What may you claim?
8. What happens if the LLM judge always answers "A" in pairwise mode with position swapping?
9. Do this: change the API so task qa is answered by the SFT adapter instead of the DPO adapter, without editing Python code.
10. Why does the EC2 instance profile policy list specific ARNs instead of "Resource": "*" for ECR images, S3 and secrets?

PART 14 - Sources

[1] Hyperstack GPU pricing: https://www.hyperstack.cloud/gpu-pricing (and https://computeprices.com/providers/hyperstack/gpus/a100pcie)

[2] Hyperstack getting started (billing starts when ACTIVE; console flow): https://docs.hyperstack.cloud/docs/getting-started/

[3] Hyperstack VM states and billing; hibernation: https://docs.hyperstack.cloud/docs/billing/states-and-billing and https://docs.hyperstack.cloud/docs/virtual-machines/hibernation

[4] Hyperstack operating system images: https://docs.hyperstack.cloud/docs/virtual-machines/images

[5] FakeCloud: https://fakecloud.dev and https://github.com/faiscadev/fakecloud

[6] Hyperstack ephemeral storage (/ephemeral, wiped on hibernate): https://docs.hyperstack.cloud/docs/storage/ephemeral

[7] FakeCloud install and ECR docs: https://fakecloud.dev/docs/getting-started/install and https://fakecloud.dev/docs/services/ecr

[8] vLLM LoRA adapters (flags, runtime load/unload): https://docs.vllm.ai/en/latest/features/lora.html

[9] TRL SFTTrainer: https://huggingface.co/docs/trl/main/en/sft_trainer

[10] TRL DPOTrainer: https://huggingface.co/docs/trl/main/en/dpo_trainer

[11] LangSmith pricing (secondary sources, verify on the official page): https://www.langchain.com/pricing, https://www.zenml.io/blog/langsmith-pricing, https://www.metacto.com/blogs/the-true-cost-of-langsmith-a-comprehensive-pricing-integration-guide

[12] AWS g6.xlarge / g5.xlarge pricing (aggregators): https://calculator.holori.com/aws/ec2/g6.xlarge/us-east-1, https://www.doit.com/compute/spot/us-east-1/g6.xlarge

[13] AWS Deep Learning Base OSS Nvidia Driver GPU AMI (Ubuntu 24.04) release notes: https://docs.aws.amazon.com/dlami/latest/devguide/aws-deep-learning-base-gpu-ami-ubuntu-24-04.html

[14] Unsloth documentation: https://unsloth.ai/docs and https://github.com/unslothai/unsloth

[15] Llama 3.1 Community License: https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct and https://www.llama.com/llama3_1/license/

[16] distilabel on PyPI (1.5.3, January 2025): https://pypi.org/project/distilabel/

[17] Datasets: https://huggingface.co/datasets/GBaker/MedQA-USMLE-4-options, https://huggingface.co/datasets/openlifescienceai/medmcqa, https://huggingface.co/datasets/qiaojin/PubMedQA, https://huggingface.co/datasets/AGBonnet/augmented-clinical-notes

[18] Qwen2.5-14B-Instruct: https://huggingface.co/Qwen/Qwen2.5-14B-Instruct

[19] LangSmith SDK: https://github.com/langchain-ai/langsmith-sdk and https://docs.smith.langchain.com

[20] Papers: QLoRA https://arxiv.org/abs/2305.14314, LoRA https://arxiv.org/abs/2106.09685, DPO https://arxiv.org/abs/2305.18290, Zephyr https://arxiv.org/abs/2310.16944, Tulu 3 https://arxiv.org/abs/2411.15124, UltraFeedback https://arxiv.org/abs/2310.01377, LLM-as-a-judge https://arxiv.org/abs/2306.05685, Prometheus 2 https://arxiv.org/abs/2405.01535, PagedAttention https://arxiv.org/abs/2309.06180, S-LoRA https://arxiv.org/abs/2311.03285, GPT-3 https://arxiv.org/abs/2005.14165, BERTScore https://arxiv.org/abs/1904.09675

[21] Medical evaluation: Med-PaLM 2 https://doi.org/10.1038/s41591-024-03423-7, HealthBench https://arxiv.org/abs/2505.08775, MedHELM https://arxiv.org/abs/2505.23802

[22] AWS documentation: ECR https://docs.aws.amazon.com/ecr/, EC2 user data https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html, Service Quotas https://docs.aws.amazon.com/servicequotas/, Secrets Manager https://docs.aws.amazon.com/secretsmanager/

[23] uv: https://docs.astral.sh/uv/ · VS Code Remote-SSH: https://code.visualstudio.com/docs/remote/ssh · WinSCP: https://winscp.net/eng/docs/start

ANSWER KEY

1. The trainer adds <|begin_of_text|> again, so every example starts with two BOS tokens. Training still runs, but the model learns from a sequence it never sees at inference (vLLM sends one BOS), which can hurt quality. The CPU smoke test caught exactly this.

2. 7,040 / (8 × 2) = 440 optimizer steps.

3. The default --max-loras is 1, so only one adapter can be active in a GPU batch; requests for the other two adapters wait for the next batches. Latency rises even though the GPU has capacity. With --max-loras 4 all three run in the same batch.

4. About 0.693 (= ln 2). At step 0 the policy equals the reference (the frozen "ref" copy of the SFT adapter), so both log-ratios are 0, the margin is 0 and -log(sigmoid(0)) = ln 2.

5. aws --endpoint-url http://localhost:4566 ecr describe-images --repository-name medscriptai-api (or set AWS_ENDPOINT_URL and drop the flag).

6. GPU billing stops, the root disk is kept, but /ephemeral is wiped, so the cache is lost (it re-downloads automatically). After Restore: run mkdir -p /ephemeral/hf-cache, update the IP if it changed, and restart FakeCloud.

7. Only that the observed difference is +1.6 points and NOT statistically significant: the interval contains 0. Do not claim an improvement; collect more test data or a bigger effect.

8. With swapping, A-then-B gives "A" (model 1) and B-then-A gives "A" (model 2), so the orders disagree and every item becomes a tie. The win rates are 0/0 and ties 100%: a clear signal of a broken or position-biased judge.

9. Set MODEL_QA=qa in deploy/.env and run docker compose --env-file .env up -d api in the deploy folder (only the API container restarts).

10. Least privilege: if the machine is compromised, the attacker can only read MedScriptAI's own image, bucket and secrets, not every image, bucket and secret in the account. "Resource": "*" is kept only for ecr:GetAuthorizationToken (it has no resource) and logs.